

핵심 요약 노트 이론편

정보 처리 기사 실기

목 차

1. 소프트웨어 구축	1
2. 데이터베이스	6
3. 운영체제	9
4. 네트워크	11
5. 정보보안	14
6. 기타 용어	16
7. SQL문 활용	18

안녕하세요. **꿈꾸는라이언**입니다.

먼저 정처기 필기 합격을 진심으로 축하드립니다!

이제 정처기 실기를 앞두고 이 요약노트를 찾으셨을텐데요,

처음 준비하시는 분들을 위해 **간단한 실기 공부 팁**을 공유드리려 해요~

꼭 읽어주시고 참고하셔서 우리 모두 정처기 1트 합격해봐요!

1. **실기도 기출이 가장 중요해요!** 물론 필기만큼이나 기출 출제율이 높은 건 아니에요. 그럼에도 기출이 아예 출제되지 않는 것도 아닙니다! 예를 들어 **최근 23년 3회 실기에서는 5문항이 이전 기출에서 유사하게 출제되었는데요**, 전체 20문항인 실기 시험을 생각하면 기출 연계율이 결코 낮은게 아니죠! 사실 기출에서 ‘단 한 문제’라도 출제된다면 수험생 입장에서는 너무나 감사한 일이고 당연히 기출을 우선으로 공부하셔야 해요. 20년도 개정 이후로 출제된 실기 기출이 지금까지 200 여 문항 정도가 있을텐데요. 개정 이후 전체 기출 정리가 시간 관계 상 **어렵다면 최신 기준 2~3년치 기출만이라도 정리하고 시험에 응시하는걸 적극 권장드려요!**
2. **프로그래밍 언어를 반드시 공부하셔야 돼요!** 20년에 출제된 예전 기출을 보시면 프로그래밍 언어는 4~5문제로 전체 20문항 중에서 그 비중이 적었어요. 실제 당시에는 프로그래밍 언어를 모르더라도 합격에는 문제가 없었죠. 그런데 안타깝지만 최신 기출 동향을 살펴보면 프로그래밍 언어 문제가 8~10문항으로 거의 절반에 달하고 있습니다! 이 말인 즉슨 프로그래밍 언어를 아예 모른다면 실기 합격에 필요한 60점을 물리적으로 받을 수 없어요. π 원래 해당 요약노트에도 프로그래밍 언어를 간단하게라도 요약하고자 했습니다만 최신 기출 문제를 보면 난이도가 점점 높아지고 있고 단순 암기가 아닌 종합적인 ‘이해’를 해야만 풀 수 있는 문제들이 자주 출제되고 있어 단 몇 페이지로 방대한 프로그래밍 언어를 요약하기가 매우 어려웠습니다. **그래서 정처기 실기 중 프로그래밍 언어 부분은 별도 기출풀이집을 제작해서 기출 풀이를 통한 이론 정리로 대체하려고 해요!** 전문성을 갖추기 위해 **현업 개발자 지인과 함께 제작했는데요**, **꿈꾸는라이언 스토어에서 별도 무료 샘플 제공 및 별도 판매 중이니 한번 참고해보세요!** 추가로 정처기에 출제되는 프로그래밍 언어 관련해서는 유튜브에만 관련 키워드 검색하셔도 이론부터 실제 기출 풀이까지 상세한 강의를 쉽게 찾아볼 수 있으니 꿈꾸는라이언 요약노트와 함께 반드시 프로그래밍 언어도 같이 병행해서 공부 부탁드릴게요!
3. **해당 요약노트는 프로그래밍 언어를 제외한 나머지 이론 내용과 SQL문에 대한 내용을 요약한 자료예요!** 필기 요약노트를 먼저 보신 분들은 상당 부분 겹치는 내용이 많은 것을 아실텐데요, 그럴 수밖에 없는게 필기와 실기 이론 부분의 출제 범위가 다르지 않거든요. 다만 총 5과목으로 나뉘서 과목별로 출제되는 필기와 다르게 실기의 경우 실제 시험에서는 과목별로 나뉘서 출제되지 않습니다. 기출을 보시면 아시겠지만 시험 과목 순서와 상관없이 이론 내용들이 프로그래밍 언어와 함께 무작위로 출제되고 있어요. π 이로 인해 시험 범위 순서대로 암기하신 수험생 분들은 실제 시험 때 관련 키워드가 어느 과목의 무슨 파트에서 출제되었는지 혼란스러울 수 있고요. 따라서 정처기 출제 기준으로 공식 가이드된 12개 과목의 순서가 아닌 각 내용별 연관성을 고려하여 **총 7개의 파트**로 다시 나뉘 방대한 시험 범위를 암기하시는데 편하게 정리했어요!

정처기 자격증이 필요하신 수험생들의 빠른 1트 합격을 위해 합격에 필요한 핵심 키워드들을 놓치지 않으면서 비전공자인 저도 이해할 수 있도록 최대한 꼼꼼하게 요약해봤어요! 필기보다 실기가 아무래도 낯선 프로그래밍 언어로 익숙하지 않아 많이 힘드시겠지만 끝까지 포기하지 마시고 최선을 다하신다면 모두들 좋은 결과로 마무리하실거라 믿어요 :D

이 <정처기 실기 요약노트 - 이론편>으로 공부하시면서 궁금하신 사항이나 의견 있으시면 언제든지 편하게 메일이나 구매하셨던 스토어 톡톡문의로 연락주시고요~ 문의주신 모든 분들께 제가 아는 선에서 성심껏 빠르게 답변해드릴게요! 그럼 정처기 합격을 시작으로 각자 계획하셨던 목표와 꿈 모두 이루시길 진심으로 바라겠습니다!

합격 미리 축하드립니다!! ^__^



■ 소프트웨어 생명 주기 (Software Development Life Cycle, SDLC)

① 프로젝트 계획 ▶ ② 요구 분석 ▶ ③ 설계 ▶ ④ 구현 ▶ ⑤ 테스트 ▶ ⑥ 유지 보수

폭포수	선형 순차적 개발 / 고전적, 전통적 개발 모형 / Step-by-Step
프로토타입	고객의 need 파악 위해 전본/시제품 을 통해 최종 결과 예측 인터페이스 중심 / 요구사항 변경 용이
나선형 (Spiral)	폭포수 + 프로토타입 + 위험 분석 기능 추가 (위험 관리/최소화) 점진적 개발 과정 반복 / 정밀하며 유지보수 과정 필요 X ★ 계획 수립 → 위험 분석 → 개발 및 검증 → 고객 평가
애자일 (Agile)	일정한 짧은 주기(Sprint 또는 Iteration) 반복하며 개발 진행 → 고객 요구사항에 유연한 대응 (고객 소통/상호작용 중심) Ex. XP(eXtreme Programming), Scrum, FDD (기능중심), 린 (LEAN), DSDM (Dynamic System Development Method)

하향식 설계 (Top-down): **절차 지향** (순차적) / 최상위 컴포넌트 설계 후 하위 기능 부여
→ 테스트 초기부터 사용자에게 시스템 구조 제시 가능

상향식 설계 (Bottom-up): **객체 지향** / 최하위 모듈 먼저 설계 후 이들을 결합하고 검사
→ 인터페이스 구조 변경 시 상위 모듈도 같이 변경 필요하여 **기능 추가 어려움**

* **Component** : 명백한 역할을 가지며 재사용되는 모든 단위 / 인터페이스 통해 접근 가능

■ 익스트림 프로그래밍 (eXtreme Programming, XP)

- 고객의 요구사항을 유연하게 대응하기 위해 고객 참여와 신속한 개발 과정을 반복

- 5가지 핵심 가치 : 용기 / 단순성 / 의사소통 / 피드백 / 존중

※ **피드백** : 시스템의 상태와 사용자의 지시에 대한 효과를 보여줘서 사용자가 명령에 대한 진행 상황과 표시된 내용을 해석할 수 있게 도와줌

- 기본 원리 : 전체 팀 / 소규모 릴리즈 / 테스트 주도 개발 / 지속적인 통합 / 공동 소유권 (Collective ownership) / 짝(pair) 프로그래밍 / 디자인 개선 (리팩토링) / 애자일(Agile) 방법론 활용
상식적 원리 및 경험 추구, **개발 문서보단 소스코드에 중점** (문서화 X)

① 프로젝트 계획

▶ 하향식 비용 산정 기법

→ 개인적 / 주관적 판단 가능

- 전문가 감정 기법 : 조직 내 두 명 이상의 전문가에게 비용 산정을 의뢰하는 기법

- 델파이 기법 : **한 명의 조정자**와 여러 전문가의 의견을 종합하여 산정
→ '전문가 감정 기법'의 주관적 편견 보완

▶ 상향식 비용 산정 기법

- 프로젝트 세부 작업 단위 별로 비용 정산 후 전체 비용을 산정하는 방법

[종류]

. **LOC (Source Line of Code)** : 코드 라인 총 수 / 생산성 / 개발 참여 인원 등으로 계산
→ 낙관치(a), 비관치(b), 기대치(c)를 측정/예측하여 **비용 산정** = $\frac{a + 4c + b}{6}$

. **개발 단계별 인일 수 (Effort Per Task)** : LOC 기법 보완 / 생명 주기 각 단계별로 산정

▶ 수학적 비용 산정

① **COCOMO (Constructive Cost Model)** : 보험 (Boehm) 제안 / 원시코드 라인 수 기반
→ 비용 견적 강도 분석 및 비용 견적의 유연성이 높아 널리 통용됨
→ 같은 프로젝트라도 성격에 따라 비용이 다르게 산정

유형	조직형 (Organic)	중,소규모 SW용 / 5만 라인(50KDSI) 이하
	반분리형 (Semi-detached)	30만 라인 (300KDSI) 이하의 트랜잭션 처리 시스템
	내장형 (Embedded)	30만 라인 (300KDSI) 이상의 최대형 규모 SW 관리

② **PUTNAM** : SW 생명주기 전 과정에 사용될 **노력의 분포**를 이용한 비용 산정

Rayleigh Norden 곡선의 노력 분포도를 기초로 함

★ **SLIM** : Rayleigh-Norden 곡선 / Putnam 모형 기초로 개발된 자동화 추정 도구

③ **Function Point (FP)** : SW 기능 증대 요인에 **가중치 부여** 후 합산하여 기능점수 산출
→ SW 기능 증대 요인 : 자료 입력 (입력 양식) / 정보 출력 (출력 보고서) / 명령어 (사용자 질의수) / 데이터 파일 / 인터페이스
★ **ESTIMACS** : FP 모형을 기반으로 하여 개발된 자동화 추정 도구

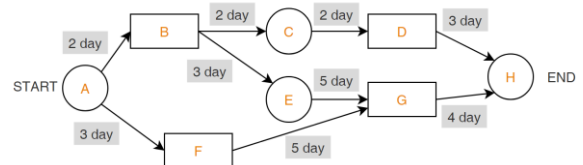
■ 개발 일정 산정

- **WBS (Work Breakdown Structure)** : 프로젝트 목표 달성을 위한 활동과 업무를 세분화
→ 전체 프로젝트를 분할 / 수행 업무 식별, 일정 및 비용

▶ 네트워크 차트

PERT (Program Evaluation and Review Technique)	. 프로젝트 작업 상호관계를 네트워크로 표현 . 원 노드(작업)와 간선(화살표)으로 구성 (@불확실한 상황) → 간선에는 각 작업별 낙관치/기대치/비관치를 기재
CPM (Critical Path method)	. 미국 Dupont 회사에서 화학공장 유지/관리 위해 개발 . 노드(작업) / 간선(작업 전후 의존 관계) / 박스(이정표) 구성 . 간선(화살표)의 흐름에 따라 작업 진행 (@확실한 상황)

[예시] CPM 네트워크 - **임계 경로** : 14일 (A-B-E-G-H) ← 경로상 가장 오래 걸리는 시간



▶ **간트 차트** : 각 작업의 시작/종료 일정을 막대 바(Bar) 도표를 이용하여 표현
시간선(Time-line) 차트 (수평 막대 길이 = 작업기간)
작업 경로는 표현 불가 / 계획 변화에 대한 적응성이 낮음

② 요구사항 분석

■ **요구사항** : 어떠한 문제를 해결하기 위해 필요한 조건 및 제약사항을 요구
소프트웨어 개발/유지 보수 과정에 필요한 기준과 근거 제공

▶ 요구사항의 유형

기능적 요구사항	실제 시스템 수행에 필요한 기능 관련 요구사항 ex. 금융 시스템은 조회/인출/입금/송금 기능이 있어야 한다.
비기능적 요구사항	성능, 보안, 품질, 안정성 등 실제 수행에 보조적인 요구사항 Ex. 모든 화면이 3초 이내에 사용자에게 보여야 한다.

▶ 요구사항 개발 프로세스 ★순서 중요

① 도출/추출	이해관계자들이 모여 요구사항 정의 (식별하고 이해하는 과정) Ex. 인터뷰, 설문, 브레인스토밍, 청취, 프로토타이핑, 유스케이스
② 분석	사용자 요구사항에 타당성 조사 / 비용 및 일정에 대한 제약 설정 Ex. 관찰, 개념 모델링, 정형 분석, 요구사항 정의 문서화
③ 명세	요구사항 체계적 분석 후 승인가능하도록 문서화
④ 확인/검증	요구사항 명세서가 정확하고 완전하게 작성되었는지 검토

▶ 요구사항 분석 도구

요구사항 분석 CASE (Computer Aided SW Engineering)	. SADT : SoftTech사에서 개발 / 구조적 분석 및 설계분석 . SREM : 실시간 처리 SW 시스템에서 요구사항 명확한 기술 목적 . PSL/PSA : 문제 기술언어 및 요구사항 분석 보고서 출력 . TAGS : 시스템 공학 방법 응용에 대한 자동 접근 방법
HIPO (Hierarchy Input Process Output)	하향식 설계 방식 / 가시적, 총체적, 세부적 다이어그램으로 구성 기능과 자료의 의존 관계 동시 표현 / 이해 쉽고 유지보수 간단

▶ 구조적 분석 모델

- **데이터/자료 흐름도 (DFD, Data flow diagram)** :

프로세스 (Process) / 자료 흐름 (Flow) / 자료 저장소 (Data store) / 단말 (Terminator)
원 화살표 평행선 사각형

→ 구조적 분석 기법에 이용 / 시간 흐름 명확한 표현 불가 / 버블(bubble) 차트

- **자료 사전 (DD, Data Dictionary)** : 자료 흐름도에 기재된 모든 자료의 상세 정의/설명

=	정의	[]	택일 / 선택	()	생략
+	구성	**	설명 / 주석	{ }	반복

- 소단위 명세서 / 개체 관계도 (ERD, Entity Relationship Diagram) / 상태 전이도



※ 객체지향 분석 모델

- . Booch (부치): 미시적, 거시적 개발 프로세스를 모두 사용 (클래스/객체 분석 및 식별)
 - . Jacobson (제이콥슨): Use case를 사용 (사용자, 외부 시스템이 시스템과 상호작용)
 - . Coad-Yourdon: E-R 다이어그램 사용 / 객체의 행위 모델링
 - . Wirfs-Brock: 분석과 설계 구분 없으며 고객 명세서 평가 후 설계 작업까지 연속 수행
 - . Rumbaugh (럼바우): 가장 일반적으로 사용, 객체/동적/기능 모델로 구분
- 객체 모델링 (Object) → 객체 다이어그램 / 객체들 간의 관계 규정/정의
 동적 모델링 (Dynamic) → 상태 다이어그램 / 시스템 동적인 행위 기술
 기능 모델링 (Function) → 자료 흐름도(DFD) / 다수의 프로세스들 간의 처리 과정 표현

▶ 요구사항 명세

정형 명세	수학적 원리 / 정확하고 간결한 요구사항 표현 가능 어려운 표기법으로 사용자 이해 어려움 (VDM, Z, Petri-net, CSP)
비정형 명세	자연어, 그림 중심 / 쉬운 자연어 사용으로 의사소통 용이하나 작성자에 따라 모호한 내용으로 일관성 떨어짐 (FSM, Decision Table, E-R 모델, State Chart)

③ 소프트웨어 설계

■ 소프트웨어 설계 원리

- 분할과 정복**: 여러 개의 작은 서브시스템으로 나눠서 각각을 완성
- 모듈화 (Modularity)**: 시스템 기능을 모듈 단위로 분류하여 성능/재사용성 향상
 - 모듈 크기 ▲ → 모듈 개수 ▼ → 모듈간 통합비용 ▼ (but, 모듈 당 개발 비용 ▲)
 - 모듈 크기 ▼ → 모듈 개수 ▲ → 모듈간 통합비용 ▲
- 추상화 (Abstraction)**: 불필요한 부분은 생각하고 필요한 부분만 강조해 모델화
 - 문제의 포괄적인 개념을 설계 후 차례로 세분화하여 구체화 진행
 - ① **과정 추상화**: 자세한 수행 과정 정의 X, 전반적인 흐름만 파악가능하게 설계
 - ② **데이터(자료) 추상화**: 데이터의 세부적 속성/용도 정의 X, 데이터 구조를 표현
 - ③ **제어 추상화**: 이벤트 발생의 정확한 절차/방법 정의 X, 대표 가능한 표현으로 대체
- 단계적 분해 (Stepwise refinement)**: 하향식 설계 전략 (by Niklaus Wirth)
 - 추상화의 반복에 의한 세분화 / 세부 내역은 가능한 뒤로 미루어 진행
- 정보 은닉 (Information Hiding)**
 - 한 모듈 내부에 포함된 절차/자료 관련 정보가 숨겨져 다른 모듈의 접근/변경 불가
 - 모듈을 독립적으로 수행할 수 있어 요구사항에 따라 수정/시험/유지보수가 용이함

■ 아키텍처 패턴

Layer	시스템을 계층으로 구분/구성하는 고전적 방식 (OSI 참조 모델)
Client-server	하나의 서버 컴포넌트와 다수의 클라이언트 컴포넌트로 구성 클라이언트와 서버는 요청/응답 제의 시 서로 독립적 * 컴포넌트(Component): 독립적 업무/기능 수행 위한 실행코드 기반 모듈
Pipe-Filter	데이터 스트림 절차의 각 단계를 필터 컴포넌트로 캡슐화 후 데이터 전송 / 재사용 및 확장 용이 / 필터 컴포넌트 재배치 가능 단방향으로 흐르며, 필터 이동 시 오버헤드 발생 변환, 버퍼링, 동기화 적용 (ex. UNIX 셸 - Shell)
Model-view Controller	모델 (Model): 서브시스템의 핵심 기능 및 데이터 보관 뷰 (View): 사용자에게 정보 표시 컨트롤러 (Controller): 사용자로부터 받은 입력 처리 → 각 부분은 개별 컴포넌트로 분리되어 서로 영향 X → 하나의 모델 대상 다수 뷰 생성 ▶ 대화형 애플리케이션에 적합
Master-slave	마스터에서 슬레이브 컴포넌트로 작업 분할/분리/배포 후 슬레이브에서 처리된 결과물을 다시 돌려 받음 (병렬 컴퓨팅)
Broker	컴포넌트와 사용자를 연결 (분산 환경 시스템)
Peer-to-peer	피어를 한 컴포넌트로 산정 후 각 피어는 클라이언트가 될 수도, 서버가 될 수도 있음 (펄티스레드 방식)
Event-bus	소스가 특정 채널에 이벤트 메시지를 발행 시 해당 채널을 구독한 리스너들이 메시지를 받아 이벤트를 처리함
Blackboard	컴포넌트들이 검색을 통해 블랙보드에서 원하는 데이터 찾을
Interpreter	특정 언어로 작성된 프로그램 코드를 해석하는 컴포넌트 설계

■ UML (Unified Modeling Language) → 구성요소 : 사물, 관계, 다이어그램

- 고객/개발자 간 원활한 의사소통을 위해 표준화된 대표적 객체지향 모델링 언어
- Rumbaugh, Booch, Jacobson 등 객체지향 방법론의 장점 통합

※ **인터페이스**: 클래스/컴포넌트가 구현해야하는 오퍼레이션 세트를 정의하는 모델 요소

- 사물 (Things)**: 구조(개념, 물리적 요소) / 행동 / 그룹 / 주해(부가적 설명, 제약조건)
- 관계 (Relationship)**

연관 관계 (Association)	2개 이상의 사물이 서로 관련
집합 관계 (Aggregation)	하나의 사물이 다른 사물에 포함 (전체-부분 관계)
포함 관계 (Composition)	집합 관계 내 한 사물의 변화가 다른 사물에게 영향
일반화 관계 (Generalization)	한 사물이 다른 사물에 비해 일반/구체적인지 표현 (한 클래스가 다른 클래스를 포함하는 상위 개념일 때)
의존 관계 (Dependency)	사물 간 서로에게 영향을 주는 관계 (한 클래스가 다른 클래스의 기능을 사용할 때)
실체화 관계 (Realization)	한 객체가 다른 객체에게 오퍼레이션을 수행하도록 지정 / 서로를 그룹화할 수 있는 관계

3) 다이어그램 (Diagram)

구조, 정적 다이어그램 (클래스/컴포넌트)	클래스 (Class)	클래스 사이의 관계 및 속성 표현
	객체 (Object)	인스턴스를 객체와 객체 사이의 관계로 표현
	컴포넌트 (Component)	구현 모델인 컴포넌트 간의 관계 표현
	배치 (Deployment)	물리적 요소(HW/SW)의 위치/구조 표현
	복합체 구조 (Composite Structure)	클래스 및 컴포넌트의 복합체 내부 구조 표현
행위, 동적 다이어그램 (유시커상활타상)	패키지 (Package)	UML의 다양한 모델요소를 그룹화하여 묶음
	유스케이스 (Use case)	사용자의 요구를 분석 (사용자 관점) → 사용자(Actor) + 사용 사례 (Use Case)
	시퀀스 (Sequence)	시스템/객체들이 주고받는 메시지 표현 → 구성항목: 액터* / 객체 / 생명선 / 메시지 제어 삼각형
	커뮤니케이션 (Communication)	객체들이 주고받는 메시지와 객체 간의 연관관계까지 표현
	상태 (State)	다른 객체와의 상호작용에 따라 상태가 어떻게 변화하는지 표현
	활동 (Activity)	객체의 처리 로직 및 조건에 따른 처리의 흐름을 순서로 따라 표현
	타이밍 (Timing)	객체 상태 변화와 시간 제약 명시적으로 표현
	상호작용 개요 (Interaction Overview)	상호작용 다이어그램 간 제어 흐름 표현

▶ UI 설계 (User Interface): 직관성, 유효성(사용자의 목적 달성), 학습성, 유연성

- 설계 지침: 사용자 중심 / 일관성 / 단순성 / 결과 예측 / 가시성 / 표준화 / 접근성 / 명확성 / 오류 발생 해결

- CLI (Command Line), GUI (Graphical), NUI (Natural), VUI (Voice), OUI (Organic)

텍스트 그래픽 말/행동 음성 사물과 사용자 상호작용

- UI 설계 도구

Wireframe	기획 초기 단계에 대략적인 레이아웃을 설계
Story Board	최종적인 산출문서 (와이어프레임-UI, 콘텐츠 구성, 프로세스 등)
Prototype	와이어프레임 / 스토리보드에 인터랙션 적용 실제 구현된 것처럼 테스트가 가능한 동적인 형태 모형 *인터랙션: UI를 통해 시스템을 사용하는 일련의 상호작용 (동적효과)
Mockup	실제 화면과 유사한 정적인 형태 모형
Use case	사용자 측면 요구사항 및 목표를 다이어그램으로 표현



④ 소프트웨어 구현

■ 소프트웨어 개발 프레임워크

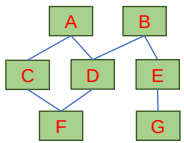
- 개발해야 할 애플리케이션 일부분이 이미 내장된 클래스 라이브러리에 구현
- 동일 로직 반복 최소화 / 재사용성 확대 / 생산성 및 유지보수성 향상
- 모듈화 / 재사용성 / 확장성 / 제어의 역흐름 (프레임워크가 어플리케이션 흐름 제어)

스프링 프레임워크 (Spring Framework)	. JAVA 플랫폼을 위한 오픈 소스 경량형 프레임워크 . 동적 웹 사이트 개발 / 전자정부 표준 프레임워크
전자정부 프레임워크	. 우리나라 공공부문 정보화 사업 시 효율적인 정보 시스템 구축 지원 위해 필요한 기능/아키텍처를 제공
닷넷 프레임워크 (.NET Framework)	. MS에서 개발한 Windows 프로그램 개발 . 공통 언어 런타임(CLR)이라는 가상 머신 상에서 작동

※ API (Application Programming Interface)

- 소프트웨어 간 인터페이스 (서로 다른 소프트웨어/서비스 간 상호 작용 용이)
- 운영체제 및 프로그래밍 언어의 기능을 프로그램에서 사용 가능하도록 구현
- 개발 비용 저감 / 중복 작업 최소화 / 유지 관리 용이 / 비즈니스 확장

- ▶ 팬인 (Fan-in): 자신을 사용하는 모듈의 수 → 팬인이 높으면 재사용 측면 설계 우수
but, 단일 장애점 발생 가능성 높아 집중적인 관리/테스트 필요
- ▶ 팬아웃 (Fan-out): 자신이 사용하는 모듈의 수 → 불필요한 호출 가능성 (단순화 필요)



Q. D의 팬인의 개수는? 2개 (A, B)

Q. D의 팬아웃의 개수는? 1개 (F)

■ 응집도 (Cohesion)

- 개발 모듈이 독립적인 기능으로 정의되어 있는 정도 / 응집도 ▲ → 품질 ▲

높은 응집도

기능적 (Function)	모듈 내부의 모든 기능 요소가 단일 문제와 연관되어 수행
순차적 (Sequential)	모듈 내 출력 데이터를 다음 활동의 입력 데이터로 사용
통신적 (Communication)	동일한 입/출력을 사용하여 서로 다른 기능을 수행
절차적 (Procedural)	모듈 내 구성 요소들이 다수 관련 기능을 순차적으로 수행
시간적 (Temporal)	특정 시간 내 처리되는 기능을 모아 하나의 모듈로 작성
논리적 (Logical)	유사한 성격의 처리 요소들로 하나의 모듈이 형성
우연적 (Coincidental)	각 구성 요소들이 서로 관련없는 요소로만 구성

낮은 응집도

■ 결합도 (Coupling) - 개별 모듈 간 상호 의존하는 정도 / 결합도 ▼ → 품질 ▲

높은 결합도

내용 (Content)	한 모듈이 다른 모듈의 내부 기능 및 자료를 직접 참조/수정
공통/공유 (Common)	공유되는 공통 데이터를 여러 모듈이 사용 (전역 변수 참조)
외부 (External)	한 모듈에서 선언한 데이터를 외부의 다른 모듈에서 참조
제어 (Control)	. 한 모듈이 다른 모듈의 상세한 처리 절차를 알고 있어 이를 통제하는 경우나 처리 기능이 두 모듈에 분리되어 설계 . 처리 대상 값뿐만 아니라 처리 방식의 제어 요소도 전달
스탬프 (Stamp)	. 두 모듈이 동일한 자료 구조(배열, 오브젝트)를 조회 . 자료 구조 및 포맷 변화 시 조회하고 있는 모든 모듈에 영향
자료 (Data)	모듈 간의 인터페이스가 자료 요소로만 구성

낮은 결합도

- ※ 모듈의 독립성: 모듈 간 과도한 상호작용을 막고 하나의 독립적 기능만을 수행
- 높은 독립성: 높은 응집도 / 낮은 결합도

■ 객체 지향 (Object-oriented): 객체와 속성, 클래스와 멤버, 전체와 부분으로 나눠 분석

객체 (Object)	고유 식별자 / 하나의 독립된 존재 / 일정한 기억장소 보유 상태(state) = 객체가 가질 수 있는 조건, 속성 값에 의해 정의 행위(연산, Method) = 객체가 반응할 수 있는 메시지 집합
클래스 (Class)	공통 속성과 연산(행위)을 갖는 객체들의 집합 / 데이터 추상화 단위 ※ 인스턴스 (Instance): 클래스에 속한 각각의 객체 ※ Operation: 클래스의 동작 / 객체에 대해 적용될 메서드 정의
캡슐화 (Encapsulation)	데이터와 데이터 처리 함수를 하나로 묶음 세부 내용 은폐(정보 은닉) → 외부 접근 제한 결합도 낮음 / 재사용 용이 / 인터페이스 단순 / 오류 파급효과 낮음
상속 (Inheritance)	상위 클래스의 속성과 연산을 하위 클래스가 물려받는 것 ※ 다중 상속: 단일 클래스가 두 개 이상의 상위 클래스로부터 상속
다형성 (Polymorphism)	하나의 메시지에 각 객체 별 고유 특성에 따라 여러 형태의 응답

오버라이딩 (Overriding): 상위클래스로부터 상속받은 메서드를 하위클래스에서 재정의
→ 단, 메서드 이름 / 매개변수 / 반환 타입은 동일해야 함

오버로딩 (Overloading): 메서드 이름은 동일하나 매개변수 개수 또는 타입을 다르게 지정

※ 객체지향 설계 5대 원칙 (SOLID)

- . 단일 책임 원칙 (SRP, Single Responsibility Principle)
→ 모든 클래스/객체는 하나의 책임만 / 완전한 캡슐화
- . 개방 폐쇄의 원칙 (OCP, Open Closed Principle)
→ 확장에는 Open하고, 수정에는 Close되어야 한다.
- . 리스코프 교체 원칙 (LSP, Liskov Substitution Principle)
→ 상위 클래스의 행동 규약을 하위 클래스가 위반하면 안된다.
- . 인터페이스 분리 원칙 (ISP, Interface Segregation Principle)
→ 클라이언트가 비사용 메서드에 의존하지 않아야 한다.
- . 의존성 역전 원칙 (DIP, Dependency Inversion Principle)
→ 의존 관계 수립 시 변화하기 어려운 것에 의존해야 한다.
추상성이 높은 상위 클래스

■ 디자인 패턴: GoF (Gang of Four) 처음 제안하여 구체화

- 서브시스템에 속하는 컴포넌트들과 그 관계를 설계하기 위한 참조 모델
- 객체 지향 프로그래밍 설계 시 자주 발생하는 문제에 대한 반복적 해결 방법
- cf) 아키텍처 패턴: 전체 시스템의 구조를 설계

[생성 패턴]

Abstract Factory	구체적인 클래스에 의존하지 않고, 서로 연관되거나 의존적인 객체들이 조합된 인터페이스 제공
Builder	객체 생성 단계를 캡슐화/분리하여 객체를 조립하여 생성 → 동일한 객체 생성 절차에서 서로 다른 표현 결과를 제공
Factory Method	상위클래스에서 객체 생성 인터페이스를 정의하지만, 인스턴스를 만드는 클래스는 서브 클래스에서 결정하도록 분리
Prototype	원본/원형 객체를 복제하는 방식으로 객체를 생성
Singleton	클래스에서 하나의 객체만 생성 가능하며, 해당 객체를 어디서든 참조할 수 있지만 여러 프로세스가 동시에 참조는 불가

[구조 패턴]

Adapter	비호환 인터페이스에 호환성 부여하도록 변환
Bridge	구현부에서 추상층을 분리 후 각자 독립적으로 변형/확장 가능
Composite	트리 구조로 부분/전체 계층 표현, 복합/단일 객체를 구분없이 사용
Decorator	상속 사용없이 객체 간 결합을 통해 객체 기능을 동적으로 추가/확장
Façade	상위에 인터페이스 구성하여 서브클래스의 기능을 복잡하게 표현하지 않고 단순한 인터페이스로 구현
Flyweight	인스턴스를 공유하여 메모리 절약 (클래스 경량화)
Proxy	접근이 힘든 객체를 연결하는 인터페이스 역할 (대리 객체 수행)



[행위 패턴]

Chain of Responsibility	처리가능한 객체가 둘 이상 존재하여 한 객체 내 처리 불가 시 다음 객체로 이관
Command	요청 명령어들을 추상/구체 클래스로 분리 후 단순화/캡슐화
Interpreter	언어에 문법 표현 정의
Iterator	컬렉션 객체의 내부 구조를 숨기고 요소들을 순차적으로 접근 → 접근이 빈번한 객체에 대해 동일 인터페이스 사용 가능
Mediator	객체들간 복잡한 상호작용을 캡슐화하여 객체로 정의 후 중재
Memento	객체를 이전의 특정 시점의 상태로 저장하고 복원 (캡슐화 유지)
Observer	한 객체 상태 변화 시 상속되어 있는 객체들에 변화 전달
State	객체의 상태에 따라 동일한 동작을 다르게 처리
Strategy	동일 계열 알고리즘을 개별적으로 캡슐화하여 상호 교환
Template Method	여러 클래스에서 공통 사용 메서드를 상위 클래스에서 정의하고, 하위 클래스마다 다르게 구현해야하는 세부 사항을 개별 구현
Visitor	각 클래스 데이터 구조로부터 처리/연산 기능을 분리하여 별도의 클래스를 만들고, 해당 클래스 메서드가 각 클래스를 돌아다니며 특정 작업을 수행 → 객체 구조 변경 X / 새로운 연산 기능만 추가

⑤ 소프트웨어 테스트

■ 애플리케이션 테스트 기본 원리

- 테스트는 기본적으로 결함이 존재함을 밝히는 것 (무결함을 증명할 수는 없음)
→ 완벽한 테스트는 근원적으로 불가능 (무한 경로, 무한 입력 불가)
- 결함 집중 : **파레토(Pareto) 법칙** - 20%의 모듈에서 전체 결함 80% 발생
- **살충제 패러독스** : 동일한 테스트 케이스에 의한 반복 테스트는 새로운 버그 발견 X
- 오류-부재의 궤변 : 결함이 없다 해도 사용자의 요구사항 미충족 시 품질 저하
- Brooks의 법칙 : 지연되는 프로젝트에 인력 추가 투입 시 더 지연

■ 애플리케이션 테스트 분류

① 프로그램 실행 여부

정적 테스트	프로그램 실행 X / 명세서, 소스 코드만 분석 Ex. 동료 검토, 워크 스루, 인스펙션, 코드 검사
동적 테스트	프로그램 실행 후 오류 검사 ex. 화이트/블랙박스 테스트

② 테스트 기반 테스트

명세 기반	사용자의 요구사항에 대한 명세를 빠짐없이 테스트 케이스로 구현하는지 확인 → ex. 동등 분할 / 경계값 분석 (블랙박스)
구조 기반	SW 내부 논리 흐름에 따라 테스트 케이스 작성/확인 ex. 구문 기반 / 결정 기반 / 조건 기반 (화이트박스)
경험 기반	테스터의 경험을 기반으로 수행 ex. 에러 추정, 체크리스트, 탐색적 테스트

③ 목적 기반 테스트

회복 (Recovery)	시스템에 인위적 결함 부여 후 정상으로 회복되는 과정 확인
안전 (Security)	외부 불법 침입으로부터 시스템을 보호할 수 있는지 확인
강도 (Stress)	과부하 시 SW 정상 구동 여부 확인
성능 (Performance)	실시간 성능 및 전체적인 효율성 진단 (응답 시간, 업무 처리량)
구조 (Structure)	SW 내부 논리적 경로 및 소스 코드 복잡도 평가
회귀 (Regression)	SW 내 변경 또는 수정된 코드에 새로운 결함이 없음을 확인
병행 (Parallel)	변경 및 기존 SW에 동일한 데이터 입력 후 결과 비교

④ 시각(관점) 기반 테스트

검증 (Verification)	개발자의 시각에서 제품의 생산 과정 테스트 Ex. 단위/통합/시스템 테스트
확인 (Validation)	사용자의 시각에서 생산된 제품의 결과 테스트 Ex. 인수 테스트 (알파 / 베타)

■ 화이트박스 테스트 (White Box Test)

- 모듈 안의 내용(작동) 직접 볼 수 있으며, 내부의 **논리적인 모든 경로를 테스트**
- 소스 코드의 모든 문장을 한 번 이상 수행 / **논리적 경로 점검** (선택, 반복 수행)
- 테스트 데이터 선택하기 위해 **검증 기준 커버리지(Coverage)** 정함

[화이트박스 테스트 검증 기준]

구문(문장) 커버리지 (Statement Coverage)	프로그램 내 모든 명령문 을 적어도 한 번 수행
결정(분기) 커버리지 (Branch Coverage)	프로그램 내 전체 결정의 분기 가 적어도 한 번씩은 실행되어 참/거짓 결과 수행
조건 커버리지 (Condition Coverage)	결정 명령문 내의 각 개별 조건식 이 적어도 한 번은 참/거짓 결과 수행
조건/결정 커버리지 (Condition/Decision)	전체 조건식뿐만 아니라 개별 조건식도 참 한 번 이상, 거짓 한 번 이상 결과 수행
변경 조건/결정 커버리지 (MC/DC)	Modified Condition/Decision Coverage 각 개별조건식이 독립적으로 전체 조건식의 결과에 영향
다중 조건 커버리지 (Multiple Condition)	결정 포인트 내 모든 개별조건식의 모든 가능한 논리적 조합을 고려하여 100% 커버리지 보장

[화이트박스 테스트 종류]

기초 경로 검사 (Base Path Testing)	. 대표적 화이트박스 테스트 기법 (동적 테스트) . 테스트 케이스 설계자가 절차적 설계의 논리적 복잡성을 측정할 수 있게 해주는 테스트 기법 . 측정 결과는 실행 경로의 기초를 정의하는 지침으로 사용
제어 구조 검사 (Control Structure Testing)	. 조건 검사 : 프로그램 모듈 내 논리적 조건 테스트 . 루프 검사 : 프로그램 반복(Loop) 구조 테스트 . 자료 흐름 검사 : 변수의 정의와 변수 사용의 위치 테스트

■ 블랙박스 테스트 (Black Box Test)

- 모듈 내부의 내용 알 수 없음 / 소프트웨어 인터페이스에서 실시되는 테스트
- SW 각 기능이 완전히 작동되는 것을 입증하는 테스트로 '**기능 테스트**' 라고 함

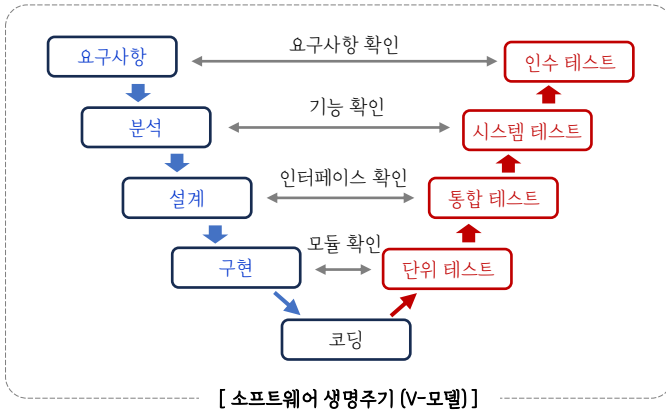
[블랙박스 테스트 종류]

동치 분할 검사 (Equivalence Partition)	. 프로그램 입력 조건에 타당한 입력 자료와 타당하지 않은 입력 자료의 개수를 균등하게 진행
경계값 분석 (Boundary Value)	. 입력 조건의 경계값을 테스트 케이스로 선정 (ex. 범위 구간의 양끝 : ~이상/이하/초과/미만)
원인-효과 그래프 검사 (Cause-Effect Graphing)	. 입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계적으로 분석 후 효용성이 높은 테스트 케이스 선정
오류 예측 검사 (Error Guessing)	. 과거의 경험이나 확인자의 감각으로 테스트 진행
비교 검사 (Comparison)	. 여러 버전의 프로그램에 동일한 결과가 출력되는지 확인

■ 요구사항 검증 (Requirements Verification)

- 실제로 고객이 원하는 시스템을 제대로 정의했는지 점검하는 과정
- 시스템 개발 완료 후 문제 발생 시 막대한 재작업 비용 발생하기에 검증 중요 (실제 요구사항 반영 여부 / 문서 상 요구사항이 서로 상충되지 않는지 점검)
 - ① 동료 검토 (Peer review) : 작성자가 내용 설명 후 동료들이 결함 검토
 - ② 워크 스루 (Walk through) : 요구사항 명세서 미리 배포 후 짧은 검토 회의 진행
 - ③ 인스펙션 (Inspection) : 작성자 제외한 다른 전문가들이 결함 검토

※ 검증 체크리스트 : 기능성 (Functionality) / 완전성 (Completeness) / 일관성 (Consistency) / 명확성 (Unambiguity) / 검증 가능성 (Verifiability) / 추적 가능성 (Traceability) / 변경 용이성 (Easily Changeable)



■ 개발단계에 따른 애플리케이션 테스트

→ 소프트웨어를 이루는 기본 단위 (독립적 기능)

① 단위 테스트 (Unit test): 최소 단위(모듈/컴포넌트) 기반 테스트

주로 구조 기반 테스트 진행 / 기능성 테스트 최우선

② 통합 테스트 (Integration test): 인터페이스 간 시스템이 정상 실행되는지 확인

- 단위 테스트 후 모듈을 통합하는 과정에서 발생하는 오류 및 결함을 찾는 테스트 기법

하향식 (Top-down)	. 상위 모듈에서 하위 모듈 방향으로 통합 . 깊이 우선(Depth first) 통합법 / 넓이 우선(Breadth first) 통합법 . 초기부터 사용자에게 시스템 구조 보여줌 . 스텝(Stub): 모듈의 기능을 단순히 수행하는 도구 (시험용 모듈)
상향식 (Bottom-up)	. 하위 모듈에서 상위 모듈 방향으로 통합 . 하나의 주요 제어 모듈과 관련된 종속 모듈의 그룹인 클러스터(Cluster)와 드라이버(Driver) 사용 / 스텝(Stub) 미사용

③ 시스템 테스트 (System test): 개발된 SW의 컴퓨터 시스템 내 작동여부 점검

실제 사용 환경과 유사한 테스트 환경 ▶ 기능적 및 비기능적 테스트 구분
블랙박스 화이트박스

④ 인수 테스트 (Acceptance test): 사용자의 요구사항 충족 여부 확인

- 알파 테스트: 통제된 환경에서 사용자가 개발자와 함께 확인
- 베타 테스트: 통제되지 않은 환경에서 개발자 없이 여러 명의 사용자가 검증

■ 테스트 관련 기타 용어

테스트 시나리오 (Test Scenario)	. 테스트 케이스 적용/동작 순서에 따라 여러 테스트 케이스를 묶은 집합
테스트 오라클 (Test Oracle)	. 테스트 결과의 참/거짓 판단 위해 사전에 정의된 참 값을 대입

- . 참 오라클 (True): 모든 테스트 케이스의 입력값에 기대 결과 제공
- . 샘플링 오라클 (Sampling): 특정 테스트 케이스 입력값에 기대 결과 제공
- . 추정 오라클 (Heuristic): 특정 테스트 케이스 입력값에 기대 결과 제공
+ 나머지 입력값에 대해선 추정 결과 제공
- . 일관성 오라클 (Consistent): 테스트 케이스의 수행 전/후의 결과값 동일 여부 확인

■ 테스트 하네스 (Test Harness)

테스트 드라이버 (Test Driver)	시험 대상의 하위 모듈 호출 / 모듈 테스트 수행 후의 결과 도출 → 상향식 통합 테스트에서 사용
테스트 스텝 (Test Stub)	제어 모듈이 호출하는 하위 모듈의 역할 단순 수행 → 하향식 테스트에 사용
테스트 스위트 (Test Suites)	시스템에 사용되는 테스트 케이스의 집합 (컴포넌트 / 모듈)
테스트 케이스 (Test Case)	사용자의 요구사항 준수 여부 확인 위해 설계된 테스트 항목 명세서 (입력값, 실행 조건, 기대 결과 등)
테스트 스크립트 (Test Script)	자동화된 테스트 실행 절차에 대한 명세서
목 오브젝트 (Mock Object)	사용자의 행위 조건부 입력 시 계획된 행위를 수행하는 객체

⑥ 소프트웨어 유지 보수

■ 애플리케이션 성능 개선

- 성능 분석 지표: 처리량 / 응답 시간 / 경과 시간 / 자원 사용률
- 소스코드 최적화를 통한 코드 스멜 제거 후 품질 개선

※ 코드 스멜 (Code Smell): 소스코드 내 존재하는 잠재적 문제점

- 예시) 중복 코드 / 긴 메서드 / 큰 클래스 / 클래스 동시 수정
- 스파게티 코드: 소스코드 로직이 복잡하게 얽혀있는 구조
- 외계인 코드: 오래되어 유지보수 작업이 어려운 코드

▶ 클린 코드 작성 원칙: 가독성 / 단순성 / 의존성 배제 / 중복성 최소화 / 추상화

▶ 소스 코드 품질 분석 도구

정적 분석 도구 (Static Analysis)	프로그램 실행 없이 코딩 표준/스타일/결함 등을 분석 → PMD, Checkstyle, SonarQube, Cppcheck, Ccm, Cobertura
동적 분석 도구 (Dynamic Analysis)	프로그램 실행하여 코드 내 메모리 누수 및 스레드 결함 발견 → Avalanche, Valgrind, Valance

■ 소프트웨어 형상 관리 (SCM: Software Configuration Management)

- 개발 과정에서 SW 변경사항을 관리하기 위한 일련의 활동 / 개발 전체 단계 적용
- 중요성: 변경사항 추적/통제, 무절제한 변경 방지, 개발 진행 이력 확인
- 형상 관리 역할: 배포본 관리 용이 / 불필요한 소스 수정 제한 / 여러 개발자 동시 개발

형상 식별	관리 대상에 이름/번호 부여 후 계층 구조로 구분 → 수정/추적 용이
형상 통제	식별된 형상 항목에 대한 변경 요구 검토 (기준선 반영될 수 있게)
형상 감사	기준선(Base line)의 무결성 평가 위해 확인/검증/검열 과정 진행
형상 기록	형상 식별/통제/감사 작업 결과를 기록/관리하고 보고서 작성

※ 제품 SW의 형상 관리 역할: 배포본 관리 용이 / 소스 수정 제한

동일 프로젝트에 대해 여러 개발자 참여 후 동시 개발 가능

■ 소프트웨어 버전 관리 도구

공유 폴더	클라이언트/서버	분산 저장소
공유 폴더에 복사	서버에서 버전 일괄 관리	원격 → 지역 저장소
RCS, SCCS, PVCS, QVCS	CVS, SVN, CVSNT, CMVC	Git, GNU arch, DCVS, Bitkeeper, Bazaar

※ 형상관리 도구

CVS	서버/클라이언트 구성, 다수의 인원이 동시 버전 관리 가능
SVN (Subversion)	CVS 개선 톨 / 모든 개발은 trunk 디렉터리에서 수행 Commit 수행 시 revision 1씩 증가
Git	서버(원격) 저장소와 개발자(지역) 저장소가 독립적 → Commit 실수 발생해도 서버에 영향 없음 (분산된 P2P 모델)

■ ISO 12207 - 소프트웨어 관련 생명주기

기본 생명주기	획득, 공급, 개발, 운영, 유지보수 프로세스
지원 생명주기	문서화, 형상관리, 품질보증, 검증, 확인, 합동검토, 감사
조직 생명주기	관리, 기반구조, 개선, 교육훈련

■ CMMI(Capability Maturity Model Integration): SW 개발 조직의 업무 능력 평가 모델

1) 초기	프로세스 X	작업자 능력에 따라 성공 여부 결정
2) 관리	규칙화 프로세스	특정된 프로젝트 내의 프로젝트 정의/수행
3) 정의	표준화 프로세스	조직의 표준 프로세스 활용 업무 수행
4) 정량적 관리	예측 가능 프로세스	프로젝트 정량적 관리/통제
5) 최적화	지속 개선 프로세스	프로세스 역량 향상 위한 지속적인 개선



■ 데이터베이스 (DBMS, DataBase Management System)

- 특정 조직 내 필요한 데이터들의 모임, 공용으로 소유/유지/이용하는 공용 데이터

■ 스키마 (Schema)

외부 스키마 (서브 스키마)	. 사용자 관점의 스키마 → 하나의 DB에 여러 개의 외부 스키마 존재 . 사용자, 프로그램마다 다양한 형태의 논리적 구조로 존재
개념 스키마	. 사용자와 DB 관리자 관점의 스키마 / DB의 전체적인 논리적 구조 . 일반적으로 하나의 DB에는 하나의 개념 스키마가 존재 → 데이터 개체/관계/제약조건/접근권한/무결성 규칙 명세
내부 스키마	. DB 설계자/개발자 관점의 스키마 . 개념 스키마를 물리적 저장장치 에 구현하는 방법을 정의 → 물리적 구조 / 내부 레코드의 물리적 순서

■ 데이터 독립성

- 논리적 독립성: 응용 프로그램 영향없이 데이터베이스 논리 구조 변경

- 물리적 독립성: 응용 프로그램 및 논리 구조 영향없이 데이터의 물리적 구조 변경

■ 데이터 언어

DDL (Data Definition Language)	데이터베이스 구조/제약 조건 정의
DML (Data Manipulation ~)	데이터 처리/조작에 사용되는 언어
DCL (Data Control ~)	데이터의 보안, 권한, 무결성, 권한 검사, 병행 제어를 위한 언어

■ 데이터베이스 설계 순서 ← ★순서 외우기

요구조건 분석	데이터베이스의 사용 용도 / 요구사항 / 요구 조건 명세서 작성
↓	
개념적 모델	. 현실 세계의 인식을 추상적 개념 세계로 표현 . 독립적인 개념 스키마 모델링 / 트랜잭션 모델링 / E-R 모델
↓	
논리적 모델	. 개념적 모델을 컴퓨터가 처리할 수 있는 구조로 표현 (관계 모델) . 종속적인 논리 스키마 설계 / 트랜잭션 인터페이스 설계 / 정규화 . 논리적 데이터 베이스 구조로 Mapping / 스키마의 평가 및 정제
↓	
물리적 모델	. 데이터의 실제 저장 방법 및 접근 경로 표현 (레코드 형식/순서) . 목표 DBMS 종속적인 물리적 구조 데이터로 변환 / 반정규화 . 오브젝트, 접근방법, 인덱스, 뷰 용량 설계 / 트랜잭션 작성 → 저장 레코드의 형식/순서/접근 경로 설계
↓	
구현	DBMS에서 SQL로 작성한 명령문 실행 후 데이터베이스 실제 생성

※ 데이터 모델링: 현실 내 복잡한 데이터 구조를 단순/추상화하여 체계적으로 표현

■ 데이터 모델 구성 요소

- 구조 (Structure): 논리적 개체 간 관계, 데이터 구조, 정적 성질 표현

- 연산 (Operation): 실제 데이터를 처리하는 작업 명세로 데이터베이스 조작 도구

- 제약 조건 (Constraint): DB에 저장될 수 있는 실제 데이터의 논리적 제약 조건

■ 개체-관계 모델 (E-R Model / Entity-Relation Model)

개체 (Entity)	독립적이고 구별 가능한 모든 것 . 개체 타입: 개체를 구성하는 속성들의 집합 . 개체 인스턴스: 실제 값을 가지면서 실체화된 개체 . 개체 세트: 개체 인스턴스들의 집합
속성 (Attribute)	개체가 가진 고유의 특성 / 가장 작은 논리적 단위 (원)
관계 (Relationship)	두 개체 간의 의미 있는 연관성 및 연결 구조 (마름모) → 일대일 (1:1) / 일대다 (1:N) / 다대다 (N:M)

■ E-R 다이어그램 표기법

	개체 집합		속성		다중값 속성
	관계 집합		기본키		개체-속성 연결

※ 점선: '관계-속성 연결'

■ 관계형 데이터베이스 용어

- 릴레이션(Relation)이라는 표(Table)로 표현

- 속성(Attribute) = 열 = 필드 / 튜플(Tuple) = 행 = 레코드

차수(Degree): 속성의 개수 / 기수(Cardinality): 튜플의 개수

- 도메인(Domain): 하나의 속성에서 취할 수 있는 원자값들의 집합 (ex. 성별: 남/여)
Atomic value

- 릴레이션 인스턴스: 릴레이션에 실제로 저장된 데이터 집합

※ 릴레이션 특징

- 튜플과 속성은 유일하며 순서는 무의미함

- 튜플은 삽입/삭제 등에 의해 계속 변함 / 튜플은 서로 상이한 값을 가짐

- 속성의 값은 분해 불가 (원자성) / 동일할 수 있음

- 튜플을 식별하기 위해 속성(필드)의 일부를 Key로 설정

- 속성은 Null 값을 가질 수 있으나, 기본키에 해당되는 속성은 Null 값을 가질 수 없음

■ 키 (Key)

후보키 (Candidate Key)	. 기본키로 사용 가능한 속성 / 모든 릴레이션에는 후보키 존재 . 모든 튜플에 대해 유일성/최소성 만족시켜야 함 ※ 유일성: 하나의 키 값으로 하나의 튜플 유일하게 식별 ※ 최소성: 모든 레코드 식별하는데 최소한의 속성으로만 구성
기본키 (Primary Key)	. 후보키 중에서 선택되어, 중복된 값과 NULL 값 가질 수 없음 . 유일성/최소성 만족하며 튜플 식별하기 위해 반드시 필요한 키
대체키 (Alternate Key)	. 후보키가 둘 이상일 때 기본키를 제외한 나머지 후보키
슈퍼키 (Super Key)	. 한 릴레이션 내 속성들의 집합으로 구성된 키 . 모든 튜플에 대해 유일성은 만족하지만, 최소성은 만족 불가
외래키 (Foreign Key)	. 다른 릴레이션의 기본키를 참조하는 속성 or 속성들의 집합 . 참조되는 릴레이션 기본키와 대응되어 릴레이션 간 참조 관계

■ 무결성 (Integrity)

개체 무결성	. 기본키를 구성하는 어떤 속성도 NULL/중복 값 가질 수 없음 . 기본키의 속성 값이 NULL값이 아닌 원자 값을 갖는 성질
도메인/속성 무결성	. 릴레이션 내 튜플들이 각 속성의 도메인에 지점된 값만 가짐
참조 무결성	. 외래키는 NULL 또는 참조 릴레이션의 기본키 값과 동일 . 릴레이션은 참조할 수 없는 외래키 값을 가질 수 없음
사용자 정의 무결성	. 속성 값들의 사용자가 정의한 제약 조건에 만족
데이터 무결성 강화	. 데이터 특성에 맞는 적절한 무결성을 정의하고 강화 . 강화 방법: 제약조건, 어플리케이션, 데이터베이스 트리거

■ 접근 통제 기술 (AC, Access Control)

임의 접근 통제 (DAC) Discretionary AC	. 사용자의 신원/신분에 따라 접근 권한 부여 . 데이터 소유자가 접근 통제 권한 지정/제어 . 객체 생성자가 모든 권한 갖고, 다른 사용자에게 허가 . SQL 명령어: GRANT / REVOKE
강제 접근 통제 (MAC) Mandatory AC	. 주체와 객체의 등급을 비교 후 시스템이 접근 권한 부여 . DB 객체별로 보안 등급 부여 및 사용자별로 인가 등급 부여 . 자신보다 보안 높은 객체에 읽기/수정/등록 불가하나 등급 같은 객체에는 모두 가능, 자신보다 낮은 객체에는 읽기 가능
역할 기반 접근 통제 (RBAC) Role-based AC	. 사용자의 역할에 따라 접근 권한 부여 (중앙관리자가 지정) . 다중 프로그래밍에 최적화

■ 뷰 (View)

- 기본 테이블에 기반을 둔 이름을 가지는 가상 테이블 (기본 테이블과 동일 형태)

- 저장장치 내 가상으로, 논리적으로 존재 (실존하는 물리적 존재 X)

- 기본 테이블 및 뷰 삭제 시 해당 테이블/뷰를 기초로 정의된 다른 뷰도 자동 삭제

▷ 장점: 논리적 데이터 독립성 제공 / 데이터 자동 보안 제공 / 데이터 관리 용이

▷ 단점: 독립적 인덱스 보유 불가 / ALTER 변경 불가 / 삽입, 삭제, 갱신, 연산 제약



■ 정규화

- 이상(Anomaly) 현상이 발생하지 않도록 **중복성/중속성을 최소화**하기 위한 작업
- **논리적 설계 단계**에서 수행하며, 속성 수가 적은 테이블로 분할되어 관리가 용이해짐
- 데이터 구조 안정성 최대화 / 데이터 삽입 시 릴레이션 재구성 필요 최소화

※ 이상현상 종류 - 삽입 이상: 데이터 삽입 시 불필요한 데이터가 함께 삽입
 삭제 이상: 튜플 삭제 시 필요한 데이터도 함께 삭제
 갱신 이상: 일부만 수정되어 데이터 불일치 → 정보 모순 발생

■ 정규화 과정

제 1 정규형	모든 도메인(Domain)이 원자 값만 으로 되어 있음
제 2 정규형	기본키가 아닌 속성이 기본키에 대한 완전 함수적 종속 만족 부분적 함수 종속을 제거 한 정규형
제 3 정규형	기본키가 아닌 모든 속성이 기본키에 대해 이행적 함수 종속 관계 를 만족하지 않아야 함 ($X \rightarrow Y, Y \rightarrow Z$ 이면 $X \rightarrow Z$)
BCNF (보이스/코드)	릴레이션 R에서 모든 결정자가 후보키 인 정규형
제 4 정규형	릴레이션 R에 다치 종속 이 성립하는 경우 R의 모든 속성이 A에 함수적 종속 관계를 만족하는 정규형
제 5 정규형	릴레이션 R의 모든 조인 종속이 R의 후보키를 통해서만 성립

※ **반정규화**: 정규화 과정 진행 후, **시스템 성능 향상** 및 운영 편의성을 위해 의도적으로
 데이터 중복/통합/분리를 허용(**무결성 저해**)하여 정규화 원칙을 위반

■ 함수적 종속

완전 함수적 종속	. Full Functional Dependency / 종속자가 기본키에만 종속
부분 함수적 종속	. Partial Functional Dependency / 기본키가 여러 속성으로 구성 되어 있을 때 기본키를 구성하는 속성 중 일부만 종속
이행적 함수 종속	. Transitive Functional Dependency / $X \rightarrow Y, Y \rightarrow Z$ 이면 $X \rightarrow Z$

■ 관계 대수 - 원하는 정보의 검색 과정을 정의하는 **절차적 언어**

① 순수 관계 연산자

σ	Select (선택)	조건을 만족하는 튜플들의 부분 집합 (수평 연산)
π	Project (추출)	속성들의 부분 집합, 중복 제거 (수직 연산)
$\bowtie$	Join (조인)	두 개의 릴레이션을 하나로 합쳐 새로운 릴레이션 형성
$\div$	Division (나누기)	A의 속성이 B의 속성 값을 모두 가진 튜플에서 ($A \supset B$) B가 가진 속성을 제외한 나머지 속성들만 추출

- . **세타 조인**: 두 릴레이션 속성 값을 비교 후 조건을 만족하는 튜플만 반환
- . **동등 조인**: 조건이 정확하게 '=' 등호로 일치하는 결과를 반환
- . **자연 조인**: 동등 조인의 결과에서 중복된 속성을 제거한 결과를 반환

② 일반 집합 연산자

U	Union (합집합)	두 릴레이션의 합 추출 / 중복은 제거
$\cap$	Intersection (교집합)	두 릴레이션의 중복되는 값만 추출
-	Difference (차집합)	A 릴레이션에서 B 릴레이션 간 중복되지 않는 값을 추출
X	Cartesian Product (교차곱)	두 릴레이션의 가능한 모든 튜플의 집합 → 속성/컬럼끼리 더하기 / 튜플끼리는 곱하기

■ 관계 해석 - 원하는 정보가 무엇이라는 것만 정의하는 **비절차적 언어**

논리 연산자	$\vee$	OR	원자식 간 “또는” 관계로 연결
	$\wedge$	AND	원자식 간 “그리고” 관계로 연결
	$\neg$	NOT	원자식에 대한 부정
정량자	$\forall$	전칭 정량자	모든 가능한 튜플 “For All”
	$\exists$	존재 정량자	어떤 튜플 하나라도 존재 “There Exists”

■ 트랜잭션 (Transaction)

- 데이터베이스 상태를 변환시키는 **하나의 논리적 기능을 수행하기 위한 작업 단위**

원자성 (Atomicity)	트랜잭션 연산이 정상적으로 수행되거나 (Commit), 아니면 어떠한 연산도 수행되지 않아야 함 (Rollback)
일관성 (Consistency)	시스템의 고정 요소는 트랜잭션 수행 전/후로 동일해야 함
독립성 (Isolation)	개별 트랜잭션은 다른 트랜잭션의 간섭 및 영향 받지 않아야 함
영속성 (Durability)	완료된 트랜잭션 결과는 영구적으로 기록되어야 함

. **COMMIT**: 트랜잭션 정상 종료 후 변경된 내용을 DB에 반영하는 명령어

. **ROLLBACK**: 트랜잭션 비정상 종료 후 모든 변경 작업을 취소하고 이전 상태로 원복

. **REDO**: 데이터베이스 비정상 종료 시 트랜잭션 시작(start)과 완료(commit)에 대한
기록이 있는 트랜잭션들의 작업을 재작업 (이전 값을 이후 값으로 변경)

. **UNDO**: 데이터베이스 비정상 종료 시 시작은 있지만 완료 기록이 없는 트랜잭션들이
작업한 내용을 모두 취소 (이후 값을 이전 값으로 변경)

■ 데이터 회복 기법

Immediate Update

. **즉시 갱신 기법**: 트랜잭션 실행 상태에서 변경되는 내용을 바로 데이터베이스에 적용
→ 변경되는 모든 내용을 로그(Log)에 기록하여 장애 시 해당 로그 토대로 복원

→ Redo / Undo 모두 수행

Deferred Update

. **지연 갱신 기법**: 트랜잭션 수행 후 부분 완료될 때까지는 데이터베이스에 바로
적용하지 않고, 지연시킨 후 부분 완료 시 로그(Log)의 내용을 토대로 저장
→ 트랜잭션이 실패할 경우 Undo 수행 없이 Redo 만 수행

Check point

. **검사 시점 기법**: 트랜잭션 수행 중간에 검사시점 (Checkpoint) 지정하여
검사 시점까지 부분 수행 후 완료된 내용을 중간중간 데이터베이스에 저장

Shadow paging

. **그림자 페이지 기법**: 로그 미사용 / 데이터베이스를 동일 크기의 페이지로 분할 후
각 페이지마다 복사하여 그림자 페이지 보관
→ 변경 내용은 원본 페이지에만 적용하여 장애 발생 시 해당 페이지 사용/회복

■ 로킹 단위 (Locking): 로킹의 대상이 되는 객체의 크기

- 로킹: 데이터베이스 병행 제어 위해 트랜잭션(transaction)이 접근하고자 하는
데이터를 잠가(lock) **다른 트랜잭션이 접근하지 못하도록 하는 병행 제어 기법**

- 대상: 데이터베이스 / 파일 / 레코드

. 로킹 단위 ▼ → 로크의 수 ▲ → 로킹 오버헤드 ▲ → 병행성(공유도) ▲

. 로킹 단위 ▲ → 로크의 수 ▼ → 로킹 오버헤드 ▼ → 병행성(공유도) ▼

※ 병행제어 기법 종류

. **로킹 기법**: 일관성과 무결성을 유지하기 위한 트랜잭션의 순차적 진행을 보장
(병행 수행 트랜잭션들 간 동일 데이터 접근 차단)

. **낙관적 검증**: 일단 트랜잭션을 수행하고, 트랜잭션 종료 시 검증을 수행

. **타임 스탬프 기법**: 타임 스탬프를 부여해 보여된 시간에 따라 트랜잭션 수행

. **다중버전 동시성 제어(MVCC)**: 타임스탬프를 비교해 직렬가능성이 보장되는
적절한 버전을 선택해 접근하도록 함

■ 분산 데이터베이스

- 논리적으로는 하나의 시스템에 존재하나, 물리적으로는 연결된 다수의 컴퓨터에
분산되어 있는 데이터 베이스

- 구성요소: 분산 처리기 (분산된 지역 컴퓨터) / 분산 데이터 베이스 / 통신 네트워크

- 목표: 위치 투명성 (Location) / 중복 투명성 (Replication) / 병행 투명성 (Concurrency)
분할 투명성 (Division) / 장애 투명성 (Failure)

→ 사용자 입장에서 데이터베이스의 위치/중복/병행/분할/장애 여부 인식할 필요 X



■ 데이터베이스 관련 용어

인덱스 (Index)	데이터베이스 테이블 검색 속도 향상을 위한 저장 위치 자료 → 데이터의 빠른 조회를 위한 위치 정보(포인터) 포함
클러스터 (Cluster)	자주 사용하는 테이블 데이터를 동일 위치에 저장하여 데이터 접근 효율 향상
데이터베이스 이중화	서비스 장애에 대비하여 데이터베이스 중복(복제)하여 관리 . Eager 기법: 모든 이중화 데이터 즉시 업데이트 . Lazy 기법: 변경 사항을 새로운 트랜잭션으로 각 노드에 전달
파티셔닝 (Partitioning)	대용량 데이터베이스를 여러 섹션으로 분할 (조회 속도 향상) . 범위 분할: 파티션 키 값을 범위로 분할 . 해시 분할: 해시 함수 적용 후 반환된 값으로 파티션 분할 . 합성 분할: 범위 분할 후 해시 함수 적용하여 다시 분할
복구 시간 목표 (RTO)	Recovery Time Objective / 서비스 중단 후 복원까지 최대 시간 → 서비스를 사용할 수 없는 최대 허용 시간
복구 시점 목표 (RPO)	Recovery Point Objective / 마지막 데이터 복구 후 허용되는 최대 시간 → 허용 가능한 최대 데이터 손실량 결정

■ 알고리즘

- 선형 검색: 하나씩 순서대로 비교하며 원하는 결과값을 찾아내는 검색
- 이진 검색: 검색 수행 전에 데이터 집합이 정렬되어야 함

■ 알고리즘 시간 복잡도

복잡도	알고리즘	설명
$O(1)$	해시 함수	자료 크기 무관 일정한 속도
$O(\log_2 N)$	이진 탐색	로그형 복잡도
$O(n)$	순차 탐색	선형 복잡도: 입력 자료를 하나씩 처리 (정비례)
$O(N \log_2 N)$	힙 / 합병(병합) 정렬	선형 로그형 복잡도
$O(N^2)$	선택 / 버블 / 삽입 퀵 정렬	대부분의 경우 힙/합병 보다 복잡도가 큼 $N^2 > N \log_2 N$ ($N > 2$)

■ 알고리즘 설계 기법

Divide and Conquer	문제를 최소 단위로 나누어 풀고, 각각을 다시 병합
Dynamic Programming	더 작은 문제의 연장선으로 여기고 과거에 구한 해를 활용
Greedy	현 시점에서 가장 최적의 방법을 선택
Backtracking	모든 조합을 시도하여 문제의 답을 선택 → 문제의 부모 노드로 되돌아간 후 다른 자손 노드 검색

■ 정렬 알고리즘

삽입	이미 정렬된 파일에 새로운 레코드를 순서에 맞게 삽입하여 정렬
선택	N개의 레코드 중 최소값을 찾아 첫번째에 배치, 나머지 N-1개 레코드 중 최소값 찾아 두번째 배치, 이를 반복하여 정렬
버블	인접한 두 개의 레코드 키 값 비교 후 크기에 따라 레코드 위치 교환
퀵	자료 이동 최소화 후 하나의 파일을 부분적으로 나눠가면서 정렬
2-way 합병	이미 정렬된 2개의 파일을 한 개의 파일로 합병하여 정렬
셸	입력 파일을 매개변수 값으로 서브파일 구성 후 각 서브파일을 삽입 정렬 방식으로 순서 배열하는 과정을 반복
힙	완전 이진 트리를 이용한 정렬

■ 정렬 방법

① 선택 정렬 (Selection sort)

- 최소값을 찾아 첫번째 숫자와 위치 교환, 이후 정렬된 값을 제외한 최소 숫자를 정렬되지 않은 숫자 중 첫번째 숫자와 다시 위치 교환, 이를 반복

시작:

9	6	7	3	5
---	---	---	---	---

 →

3	6	7	9	5
---	---	---	---	---

 →

3	5	7	9	6
---	---	---	---	---

→

3	5	6	9	7
---	---	---	---	---

 →

3	5	6	7	9
---	---	---	---	---

② 삽입 정렬 (Insert sort)

- 두번째 숫자와 첫번째 숫자 비교하여 크기 순으로 정렬, 이후 3번째 숫자를 앞서 정렬된 숫자들 사이에 크기 순에 맞게 재정렬, 이를 반복

시작:

9	6	7	3	5
---	---	---	---	---

 →

6	9	7	3	5
---	---	---	---	---

 →

6	7	9	3	5
---	---	---	---	---

→

3	6	7	9	5
---	---	---	---	---

 →

3	5	6	7	9
---	---	---	---	---

③ 버블 정렬 (Bubble sort)

- 왼쪽부터 인접한 두 숫자 간 크기 비교하여 위치 교환,
→ 1 Pass 마다 정렬되는 숫자들 중 가장 큰 숫자를 오른쪽으로 밀어서 배치

시작:

9	6	7	3	5
---	---	---	---	---

 →

6	7	3	5	9
---	---	---	---	---

 →

6	3	5	7	9
---	---	---	---	---

→

3	5	6	7	9
---	---	---	---	---

 →

3	5	6	7	9
---	---	---	---	---



■ 운영체제 종류

윈도우 (Window)	마이크로소프트사에서 개발한 운영체제
유닉스 (Unix)	미국 AT&T 벨연구소에서 개발한 운영체제 → 커널(Kernel) / 셸 (Shell)로 구성
리눅스 (Linux)	리누스 토발즈가 유닉스를 기반으로 개발한 운영체제
MacOS / iOS	애플이 유닉스 기반으로 개발한 운영체제
Android	구글이 리눅스 커널 기반으로 개발한 개방형 모바일 운영체제

※ UNIX 구성요소

커널 (Kernel)	. 하드웨어 보호 / 프로그램 및 하드웨어 간 인터페이스 역할 . 프로세스 관리, 기억 장치 관리, 파일 관리, 입출력 관리, 데이터 전송 및 변환, 셸 프로그램 실행을 위한 프로세스 및 메모리 관리
셸 (Shell)	. 사용자의 명령 인식/해석 후 커널로 전달, 명령을 수행 (반복적인 명령 프로그램을 만드는 프로그래밍 기능 제공) . 시스템과 사용자 간 인터페이스 역할 . 초기화 파일을 이용해 사용자 환경 설정

※ UNIX Shell 환경 변수 : set, env, printenv, setenv

■ 주기억장치

RAM (Random Access Memory)	휘발성 (전원 off 시 data 없어짐) / 실행 중인 프로그램 저장
	속도 집적도 전력소모 재충전 용도
	SRAM (정적) 빠름 낮음 많음 불필요 캐시 DRAM (동적) 느림 높음 적음 필요 주기억
ROM (Read Only Memory)	- 전력공급 없어도 내용 사라지지 않음 (비휘발성) - CMOS SETUP으로 값 수정 가능 - H/W와 S/W의 중간인 일종의 펌웨어 (Firmware) - 업데이트만으로 시스템 성능 향상 - 부팅 시 가장 먼저 동작하여 자체 진단 (POST) 실행 - BIOS에는 날짜, 전원관리, 부팅순서, 기본글꼴, 칩셋, 시스템 암호 (Windows 암호 X), PnP (Plug and Play), 하드디스크 타입, 안티바이러스 등의 정보 포함
	Mask ROM 제조 과정에서 저장된 ROM, 임의 수정 불가
	PROM 한번만 기록 가능, 이후는 읽기만 가능
	EPROM 자외선 신호를 이용한 ROM
	EEPROM 전기 신호를 이용한 ROM (플래쉬 메모리) - BIOS ROM으로 활용 및 업데이트 가능

■ 기타 메모리

레지스터 (Register)	CPU(중앙처리장치) 내부에서 처리할 명령어나 연산의 중간 값 등을 일시적으로 저장하는 휘발성 메모리 (메모리 중 가장 빠른 속도)
캐시 메모리	SRAM을 이용하여 CPU(빠름)와 주기억장치(느림) 사이의 속도 차이 해결 (버퍼 메모리 일종) → 캐시 적중률이 높을수록 시스템 처리 속도 증가
가상 메모리	보조기억장치를 주기억장치처럼 사용하는 메모리 (기억용량 확대) 큰 프로그램 실행 가능 / 매핑-mapping 방식 필요
플래시 메모리	MP3나 디지털카메라 등에서 사용되는 일종의 EEPROM (비휘발성) (ex. USB / SD card / NAND Flash / SATA Memory)
버퍼 메모리	데이터 일시적으로 저장해 두 장치 간 데이터 전송 속도 차이 해결
연상(연관) 메모리	저장된 데이터의 일부를 이용하여 기억장치에 접근 후 데이터 확인 내용 참조 (매핑-mapping 방식) / 주소가 아닌 내용을 참조

★H/W 처리 속도: 레지스터 > 캐시 메모리 > 주기억 장치 > 보조기억장치

■ 기억장치 관리 전략

반입 전략 (Fetch)	요구 반입	특정 프로그램/데이터의 참조를 '요구'할 때 적재
	예상 반입	미래의 참조가 '예상'되는 데이터를 미리 적재
배치 전략 (Placement)	최초 적합 (First Fit)	사용 가능한 '첫 번째' 분할 영역에 데이터 배치
	최적 적합 (Best Fit)	단편화를 '최소화'하는 분할 영역에 데이터 배치
	최악 적합 (Worst Fit)	단편화를 '최대화'하는 분할 영역에 데이터 배치
교체 전략 (Replacement)	새로운 데이터를 배치하고자 할 때 기존 사용 중 데이터를 교체 ex. FIFO, OPT, LRU, LFU, NUR ...	

■ 페이지 분할 기법

1) 페이지 기법 (Paging)

- 가상기억장치에 보관된 프로그램과 주기억장치의 영역을 **동일 크기**로 분할 후 나눠진 프로그램(페이지)을 동일하게 나눠진 주기억장치의 영역(페이지 프레임)에 배치하여 실행하는 기법 (내부 단편화 발생 가능성 존재)

▶ 작은 페이지 크기 : 페이지 단편화 감소 / 페이지 이동시간 감소 / 워킹 셋 유지
맵 테이블 크기 커지며 매핑 속도 감소 / 입출력 시간 증가

▶ 큰 페이지 크기 : 맵 테이블 크기 작아지며 매핑 속도 증가 / 입출력 시간 감소
페이지 단편화 증가 / 페이지 이동시간 증가 / 불필요 데이터 적재

2) 세그먼테이션 기법 (Segmentation)

- 가상기억장치에 보관된 프로그램을 **다양한 크기**의 논리적 단위로 분할 후 (Segment) 주기억 장치에 배치 및 실행 (외부 단편화 발생 가능성 존재)

※ Paging 기법의 경우, 프로그램을 **동일 일정 크기**로 분할함

■ 페이지 교체 알고리즘

OPT	. 앞으로 가장 오랫동안 사용하지 않을 페이지를 교체 . 페이지 부재 횟수가 가장 적게 발생 / 효율적 교체 알고리즘
FIFO	. First-in First-out . 가장 먼저 들어와 가장 오래 있었던 페이지를 교체
LRU	. Least Recently Used . 최근에 가장 오랫동안 사용하지 않은(오래 전에 사용된) 페이지를 교체
LFU	. Least Frequently Used . 사용 빈도가 가장 적은 페이지를 교체 / 활발한 페이지는 교체 X

[기출 문제] 20년도 4회 필기

4개의 페이지를 수용할 수 있는 주기억장치가 있으며, 초기에는 모두 비어 있다고 가정한다. 다음의 순서로 페이지 참조가 발생할 때, FIFO 페이지 교체 알고리즘을 사용할 경우 페이지 결함 발생 횟수는?

▶ 페이지 참조 순서 : 1, 2, 3, 1, 2, 4, 5, 1

참조 페이지	1	2	3	1	2	4	5	1
페이지 프레임 (4개)	1	1	1	①	1	1	5	5
		2	2	2	②	2	2	1
			3	3	3	3	3	3
						4	4	4
부재 발생	✓	✓	✓			✓	✓	✓

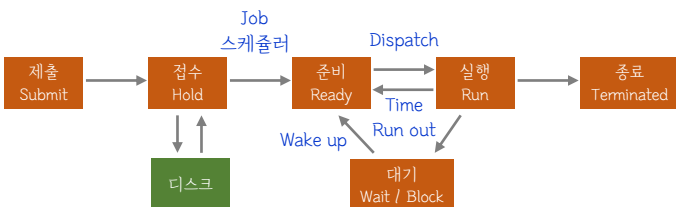
4개의 페이지가 다 찰 때까지 진행
(앞서 중복 숫자가 있으면 넘어감 = 부재 미발생)

전체 페이지가 다 차서 새로운 참조 값을 기존 페이지에 교체해야 하는 상황
이때, FIFO 기준에 따라 교체된지 오래된 페이지부터 신규 참조 값을 대체



■ 프로세스 (Process)

- 프로세서(CPU)에 의해 처리되는 사용자/시스템 프로그램
- 프로세스 제어 블록 (PCB, Process Control Block)에 프로세스 식별자, 상태 등의 중요 정보를 저장 / 각 프로세스 실행 시 고유 PCB 생성, 작업 완료 후 PCB 제거



- ※ Dispatch : 준비 상태에서 대기 중인 프로세스가 실행 상태로 전이되는 과정
- ※ Wake up : 입출력 작업이 완료되어 프로세스가 대기 상태에서 준비 상태로 전이
- ※ Context Switching : 이전 프로세스의 상태 레지스터 내용 보관하고 (문맥 교환) 다음 프로세스의 레지스터를 적재하는 과정

■ 스레드 (Thread) : 프로세스 내 작업 단위 / 프로그램 단위

- 단일: 하나의 프로세스에 하나의 스레드 / 다중: 하나의 프로세스에 복수의 스레드
- 동일 프로세스 환경 내 서로 다른 독립적 다중 스레드 수행 가능

[장점] 한 프로세스 내 복수의 스레드 생성하여 작업 병행성 향상

응답 시간 단축 / 기억 장소 낭비 최소화 / 프로세스 간 통신 향상

운영체제 성능 및 프로그램 처리 효율 향상 / 접근 가능한 기억장치 사용

사용자 수준	. 사용자가 만든 라이브러리를 이용한 스레드 운용 . 속도 빠르나 구현 어려움
커널 수준	. 운영체제의 커널에 의한 스레드 운용 . 구현 쉬우나 속도가 느림

■ 스케줄링 : 여러 프로세스의 처리 순서를 결정하는 기법

1) 선점형 : 운영체제가 실행 중인 프로세서로부터 CPU를 강제로 빼앗음

→ 프로세서 별 CPU 처리 시간에 순서 조정으로 효율적 운영 가능 / 높은 오버헤드

Round Robin	. 우선순위 없이 시간 단위로 CPU 할당 (공평한 자원 분배)
SRT (Shortest Remaining Time)	. 비선점형 SJF 기법을 선점 스케줄링 형식으로 변경 . 잔여 실행 시간이 가장 짧은 프로세스에 CPU 우선 할당
다단계 큐	. 프로세스를 그룹화 후 각 그룹마다 다른 준비 상태 큐를 사용 . 하위 준비 상태의 큐 실행 간 상위 준비 상태 큐 유입 시 상위 프로세서에 CPU 우선 할당
다단계 피드백 큐	. 프로세스 생성 시 가장 높은 우선 순위 준비 큐에 등록되며 등록된 프로세스는 FCFS 순서로 CPU를 할당받아 실행 . 단계가 내려갈수록 시간 할당량(Time Quantum)이 증가

2) 비선점형 : 기존 프로세서가 점유하고 있는 CPU를 빼앗을 수 없음

→ 낮은 처리율 / 적은 오버헤드

FCFS (First Come First Serve)	. 프로세서 도착 순서에 따라 처리 순서 결정 (=FIFO)
SJF (Shortest Job First)	. 준비 상태의 프로세서 중 실행 시간이 가장 짧은 것부터 처리
HRN (Highest Response ratio Next)	. 우선 순위를 계산하여 프로세스의 처리 순서 결정 → 우선 순위 = (대기시간 + 서비스 시간) / 서비스 시간 . 실행 시간이 긴 프로세서가 불리한 SJF 기법을 보완

■ 교착 상태 (Dead Lock) : 둘 이상의 프로세스들이 서로 점유하고 있는 자원을 요구하며 무한정 대기하고 있는 상태

[교착 상태 필요 충분 조건]

상호 배제 (Mutual Exclusion)	한 번에 한 개의 프로세스만이 공유 자원을 사용
점유 및 대기 (Hold and Wait)	최소한 하나의 자원을 점유하면서 다른 프로세스에 할당된 자원을 추가로 점유하기 위해 대기하는 프로세스 존재
비선점 (Non-preemption)	다른 프로세스에 할당된 자원은 끝날때까지 강제로 선점 불가
환형 대기 (Circular Wait)	프로세스들이 원형으로 구성되어 있어 앞/뒤 프로세스 자원을 요구

[교착 상태 해결 방법]

예방 (Prevention)	. 교착발생 4가지 조건 중 하나 이상을 사전에 제거하여 예방
회피 (Avoidance)	. 발생된 교착상태를 적절히 피해가는 방법 ※ 은행원 알고리즘 (Banker's Algorithm): 은행에서 모든 고객의 요구가 충족되도록 현금을 할당하는 데서 유래
발견 (Detection)	. 교착상태에 있는 프로세스 및 자원을 발견 / 점검
회복 (Recovery)	. 교착상태 유발 프로세스 종료 / 프로세스 내 할당된 자원 선점

※ 기억장치 관련 용어

스래싱 (Thrashing)	- 프로세스 처리 시간보다 페이지 교체 시간이 더 많아지는 상태 - 전체 시스템 성능 저하 / 다중 프로그래밍 다발 시 스래싱 발생 [방지 방법] 다중 프로그래밍 적정 수준 유지 / 일부 프로세스 중단 페이지 부재 빈도 조절 / 워킹 셋 유지
지역성 (Locality)	- 프로세스 실행 간 주기억장치 참조 시 일부 페이지만 집중 참조 . 시간 지역성 : 한 페이지를 일정 시간 동안 집중적으로 액세스 → 반복/순환(Loop), 스택(Stack), 부 프로그램(Sub Routine) . 공간 지역성 : 프로그램 실행 시 인접한 페이지들도 집중 참조 → 배열순회(Array Traversal), 순차적 코드 실행
워킹 셋 (Working Set)	- 일정 시간 동안 자주 참조하는 페이지들을 주기억장치에 상주시켜 페이지 부재 및 교체 현상을 최소화 → 기억장치 사용 안정화
페이지 부재	- 프로세스 실행 중 필요한 페이지가 주기억장치에 누락(부재)



■ 정보 전송 방식

- 단방향: 한쪽 방향으로만 전송 가능 (TV, 라디오)
- 반이중: 한쪽에서 송신하면 다른 쪽에서는 수신만 가능 (무전기)
- 전이중: 동시에 송/수신 가능 (전화기)

스타형/성형 중앙 집중형		중앙노드와 1:1 (P2P) 연결 / 고장 발견, 유지 보수, 확장이 쉬움 중앙노드의 제어장치가 통신망의 처리능력, 신뢰성 결정
버스형		한 개의 회선에 여러 단말 장치 연결, 회선 양 끝 종단장치 필요 단말 장치 하나가 고장나도 전체 영향 없기에 신뢰성 높음 설치/제거 용이, 기밀성 낮고 통신 회선의 제한이 있음
링/루프 원/환형		인접한 단말기를 서로 연결 / 양방향 전송 가능 통신망 하나가 고장 시 전체 통신망 마비 단말 장치 추가/제거, 기밀보호 어려움
트리/계층 분산형		나뭇가지 모양으로 계층적 연결 / 분산처리 시스템 구성 방식 확장이 많으면 트래픽 (통신량)이 과중됨
망/매쉬 그물/완전형		모든 지점의 단말기를 서로 연결 / 빠른 응답 시간, 높은 연결성 통신량이 많을 경우 유리함 (공중 데이터 통신망) 회선 장애 발생 시 다른 경로로 전송 / 높은 보안성 및 안정성 단말 장치 추가/제거 어려움 (낮은 확장성)

■ 동기 / 비동기 전송

구분	동기식	비동기식
전송 단위	데이터 블록 단위	문자 단위
동기 제어 방식	클럭 동기	Start bit, Stop bit
통신 속도	고속	저속
회로 복잡도	복잡	단순
휴지 시간	블록 간 휴지 시간 없음 → 높은 전송 효율	문자 간 휴지 시간 존재 가능 → 낮은 전송 효율
예시	데이터 통신망, 전화 교환망	편지, 이메일
전송 방식	문자 동기 방식 (SYN, STX, ETX) 비트 동기 방식	-

※ 비동기 전송 모드 (ATM, Asynchronous Transfer Mode)

- 자료를 일정한 크기(Cell)로 정한 후 순서대로 전송 / 멀티미디어 서비스 적합
- Cell : 고정 길이 패킷 크기 **53 Byte** (페이로드 48 Byte + 헤더 5 Byte)
- 단순한 처리 방식 / 고속망 적합 / 실시간 및 비실시간 서비스 제공 가능
- 고정 비트레이트(CBR) 및 가변 비트레이트(VBR) 모두 처리 가능

■ IPv4 / IPv6 특징

IPv4	IPv6
10,97,135,183 (0~255 = 256개 = 8 Bit)	2544::D6::4B::13F:2C4F 0000~FFFF (각 자리를 · 표로 구분)
10진수 (0~9)	16진수 (0~9 + A~F)
8 Bit x 4개 = 32 Bit	16 Bit x 8개 = 128 Bit
유니, 멀티, 브로드캐스트	유니, 멀티, 애니캐스트

※ IPv6는 주소의 확장성/용통성/연동성이 높으며, IPv4 대비 전송 속도가 빠름,
인증성/기밀성, 데이터 무결성으로 보안 문제 해결 가능
패킷 크기 제한 없으며, 등급/서비스별 패킷 구분 가능 → 품질 보장 용이

※ 캐스팅 전송 방식

유니캐스트	일 대 일 통신 (단일 송신 ↔ 단일 수신)
멀티캐스트	일 대 다 통신 (단일 송신 ↔ 다중 수신)
브로드캐스트	호스트가 속해 있는 네트워크 전체를 대상으로 패킷을 전송
애니캐스트	일 대 일 통신 (수신 가능한 노드 중 가장 가까운 한 노드와 통신)

■ 프로토콜 (Protocol): 정보 교환을 위해 무엇을, 어떻게, 언제 통신할 것인지

(구문, 의미, 순서)를 정한 **통신 규약**

[기능] **흐름 제어**: 패킷 흐름(전송량/속도)을 조정하여 송수신 안정성 유지

동기화: 송/수신이 같은 상태를 유지

오류 제어: 전송 중 발생한 오류를 검출, 복원/정정

※ 흐름제어 방식

Stop and Wait	. 수신 측 확인 후 다음 패킷 전송 / 한 번에 하나의 패킷만 전송
Sliding Window	. 수신 측 확인 없이 사전에 정해진 패킷 수만큼 연속적으로 전송 . 한 번에 여러 패킷을 동시에 전송 가능 . 긍정 수신 시 윈도우 크기 증가 / 부정 신호 시 윈도우 크기 감소
Slow start	. 정상 패킷 전송 시 혼잡 윈도우 크기를 패킷마다 1씩 증가시키나 혼잡 현상 발생 시 혼잡 윈도우 크기를 1로 줄임
Congestion Avoidance	. 네트워크 내 패킷의 지연이 너무 높아 트래픽이 붕괴되지 않도록 패킷의 흐름을 제어하는 트래픽 제어

※ 전송 오류 제어 방식

- **전진 오류 수정** (FEC, Forward Error Correction)

: 수신측에서 **스스로 오류 검출 후 수정**하는 방식 (송신측에 별도 통보 X)

ex. 해밍 코드 / 상승 코드

- **후진 오류 수정** (BEC, Backward Error Correction)

: 오류 발생 시 **송신측에 재전송 요구**

ex. 패리티 검사 / CRC (Cyclic Redundancy Check) / 블록 합

※ 오류 검출 방식

해밍 코드	. 수신측에서 오류 검출 후 자동 수정 / 1비트 오류 수정 가능 . 검출 가능 최대 오류 수 = 해밍 거리 - 1
상승 코드	. 순차적/한계값 디코딩 사용 / 여러 비트의 오류 수정 가능
패리티 검사	. 7~8개 비트로 구성된 데이터 블록 끝에 특정 패리티 비트 (짝수/홀수)를 추가하여 오류 검출
순환 중복 검사 (CRC)	. 다항식을 통해 산출된 값으로 오류 검사 (집단 오류 해결) . 데이터 뒤 오류 검출코드 FCS (Frame Check Sequence) 추가
블록 합	. 짝수개 비트 오류 검출 불가한 패리티 검사를 개선한 방법 . 데이터 블록의 수평/수직 패리티 비트 추가

■ **Oauth**: 다양한 플랫폼의 특정 사용자 데이터에 접근하기 위해 비밀번호를 제공하지 않고 제 3자 클라이언트가 사용자의 **접근 권한을 위임** 받을 수 있는 표준 프로토콜 (사용자 인증에 사용되는 표준 인증 방법 / 공개 API로 구현)

■ **IPC** (Inter-Process Communication): 실행 프로세스 간 통신을 가능하게 하는 기술

- 종류: 파이프, 메시지 큐, 공유 메모리, 세마포어, 소켓

■ **AD-hoc Network**: 기지국/엑세스 포인트와 같은 네트워크 장치가 필요하지 않고, 멀티 홉 라우팅 기능에 의해 무선 인터페이스가 가지는 통신 거리상의 제약 극복
→ 긴급 구조, 긴급 회의, 전쟁터에서의 군사 네트워크에 활용

■ IEEE 802 LAN 표준안

802.2	논리 링크 제어 (LLC)	802.6	도시형 통신망 (MAN)
802.3	CSMA/CD	802.8	Fiber Optic LANS
802.4	Token Bus	802.9	종합 음성/데이터 네트워크
802.5	Token Ring	802.11	무선 LAN



■ OSI 참조 모델 (7계층)

		TCP/IP 4계층
응용 계층 (Application)	응용 프로그램이 OSI 환경에 접속 가능한 서비스	<응용 계층> Telnet / FTP HTTP / POP SMTP DHCP / SNMP DNS
표현 계층 (Presentation)	응용-세션 간 코드/데이터 변환, 데이터 암호화/압축	
세션 계층 (Session)	컴퓨터 간 세션/연결을 생성/유지/종료하여 적절한 통신 상태 유지	
전송 계층 (Transport)	종단(END-TO-END) 간 투명한 데이터 전송/전달 주소 설정 / 다중화 / 오류 제어 / 흐름 제어 수행	<전송 계층> TCP / UDP
네트워크 계층 (Network)	개방 시스템 간 네트워크 연결을 관리 / 데이터 교환 경로 설정(Routing), 트래픽 제어, 패킷 정보 전송	<인터넷 계층> IP / ICMP ARP / RARP
데이터 링크 계층 (Data link)	송/수신 간 속도 차이 해결 위한 흐름 제어 기능 프레임 동기화 / 오류 제어 기능 / 노드 간 프레임 전송 → 물리적 주소 (MAC) 결정	<네트워크 액세스> HDLC / PPP LLC
물리 계층 (Physical)	전송에 필요한 기계적, 전기적, 기능적 특성을 정의 데이터를 0과 1의 전기적 신호로 변환	

■ HDLC 프로토콜 : High Level Data Link Control

- 대표적 데이터링크 프로토콜 / **비트 동기 방식**
- BSC (프레임에 전송 제어 문자 삽입) / LAP-B (패킷교환망) / LAP-D (ISDN에서 사용)
- PPP (전화회선을 이용한 PC 간 연결) / LLC (LAN 프로토콜)

[HDLC 링크 구성] 불균형 (1:다 구성) / 균형(1:1 구성)

*주국 : 데이터 전송 명령 수행 / *종국 : 명령에 응답 / *혼합국 : 명령과 응답 모두 처리

[HDLC 전송 모드]

정규 응답 모드 : NRM (Normal Response Mode)	. 주국과 종국의 관계 (불균형) → 종국에서 데이터 전송 시 주국 허락 필요
비동기 균형 모드 : ABM (Asynchronous Balanced Mode)	. 대등한 혼합국 간의 관계 (균형) → 양쪽에서 명령과 응답 모두 전송 가능
비동기 응답 모드 : ARM (Asynchronous Response Mode)	. 불균형 모드 / 종국이 주국의 허락 없이 데이터 전송 및 송신 가능하나 제어 기능은 주국만 허용

[HDLC 프레임 종류]

정보 프레임 (제어부가 0으로 시작)	. 사용자 데이터 및 일부 제어 정보의 전달 ① Seq(송신용 순서번호) / ② Next(응답용 순서번호) / ③ P/F(주국 컴퓨터가 종국 컴퓨터의 데이터 전송 허용) 구성
감독 프레임 (제어부가 10으로 시작)	. 확인 응답, 데이터링크의 흐름 제어, 오류 제어 용도 . 전송 목적이 아니라 Seq 값 필요 없음 (Next만 존재) . 2비트로 구성되어 4가지 Type(0~3)으로 구성
비번호 프레임 (제어부가 11로 시작)	. 순서가 없는 프레임 / 링크의 동작 모드 설정 및 관리 . 2비트 Type + 3비트 Modifier = 총 5비트 구성

■ 네트워크 계층 프로토콜

IP	. Internet Protocol / 전송할 데이터에 주소 지정하고 경로를 설정 . 비연결형 데이터그램 방식 / 신뢰성 보장 X / 패킷을 분할 및 병합 . 헤더 체크섬(header checksum) 제공 / 데이터 체크섬은 제공 X
ICMP	. Internet Control Message Protocol / 헤더 8Byte . IP와 함께 통신 간 오류 처리와 전송 경로 변경 등 제어 메시지 관리
ARP	. Address Resolution Protocol . 호스트 IP주소를 네트워크 접속 장비의 물리적 주소(MAC)로 바꿈
RARP	. Reverse Address Resolution Protocol . ARP 반대로 물리적 주소(Mac Address)를 IP주소로 변환하는 기능
IGMP	. Internet Group Management Protocol . 멀티캐스트 지원하는 호스트/라우터 간 멀티캐스트 그룹 유지
RIP	. Routing Information Protocol . 최소 Hop count (최단거리) 경로로 라우팅하는 프로토콜
NAT	. Network Address Translation (네트워크 주소 변환) . 사설 네트워크에 속한 IP를 공인 IP 주소로 변환하는 기술 → 1개의 정식 IP 주소에서 다량의 가상 사설 IP 주소를 할당/연결

■ 전송 계층 프로토콜

TCP	. Transmission Control Protocol / 양방향 서비스 제공 . 순서 제어/오류 제어/흐름 제어 기능 → 높은 신뢰성 . 프로토콜 헤더는 기본적으로 20~60 Byte . 패킷 단위의 스트림 위주 전달
UDP	. User Datagram Protocol / 비연결형 서비스 제공 . TCP 대비 단순한 헤더 구조로 오버헤드 적고, 전송속도 빠름 (제어 X) . 실시간 전송 유리 / 신뢰성보다는 속도가 중요한 네트워크에 활용
RTCP	. Real-time Control Protocol / RTP 패킷의 전송 품질 제어 . 세션에 참여한 각 참여자들에게 주기적으로 제어 정보 전송

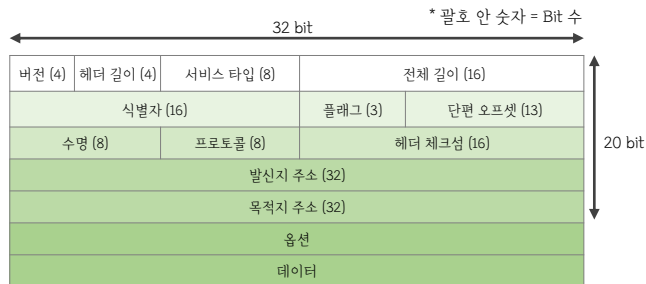
■ 응용 계층 프로토콜

※ 괄호 안 숫자 : 포트 번호

Telnet [23]	다른 컴퓨터 접속 후 원격 서비스 제공 / 가상의 터미널 기능 수행
FTP [21]	File Transfer Protocol / 원격 파일 전송 프로토콜 (컴퓨터-인터넷)
HTTP [80]	Hyper Text Transfer Protocol / WWW 내 HTML 문서 송수신 위한 프로토콜 / GET과 POST 메서드를 통해 메시지 주고 받음
SMTP [25]	Simple Mail Transfer Protocol / 전자 우편 교환 서비스
DNS	Domain Name System / 도메인 네임을 IP 주소로 매핑하는 시스템
SNMP	Simple Network Management Protocol TCP/IP 네트워크 관리 프로토콜 (네트워크 기기 정보 전송 규약)
DHCP	IP주소 및 TCP/IP 프로토콜의 기본 설정을 클라이언트에게 자동 제공

■ IP 헤더 구조

버전 (Version)	사용 중인 프로토콜 버전 (ex. IPv4 / IPv6)
헤더 길이 (Header Length)	IP 헤더의 길이를 바이트 단위로 표현 (20 ~ 60 Byte)
서비스 유형 (Service Type)	요구 서비스 품질 지정 (패킷의 전송 우선순위 제공)
전체 패킷 길이 (Packet Length)	IP 헤더 + 데이터 포함된 전체 IP 패킷 길이
식별자 (Identification)	전송 패킷을 식별하기 위한 부여된 일련번호 (16 Bit)
플래그 (Flags, DF/MF)	단편화(분할) 상태를 표현
단편 오프셋 (Fragmentation Offset)	단편화(분할)된 패킷들의 순서 지정
수명 (Time to live, TTL)	패킷이 네트워크를 통과할 수 있는 남은 라우터의 최대 개수
프로토콜 타입 (Protocol)	데이터에 포함되는 상위 프로토콜 종류 표현 Ex. 1 = ICMP / 2 = IGMP / 6 = TCP / 17 = UDP
헤더 체크섬 (Header Checksum)	헤더 필드의 오류 발생 유무 검사
발신지/목적지 주소 (Source / Destination IP Address)	패킷을 보낸/받는 IP 주소





■ 정보 보안 3대 요소

- 기밀성** : 시스템 내 정보 자원은 인가된 사용자에게만 접근 허용
- 무결성** : 오직 인가된 사용자가 시스템 내 정보 수정 가능
- 가용성** : 인가된 사용자는 권한 범위 내에서 언제든지 자원 접근 가능
+ 인증 (사용자의 신분 확인) / 부인방지 (부인할 수 없도록 송·수신 증거 제공)

■ AAA (Triple-A)

- 인증 (Authentication)** : 사용자가 네트워크 접속 전에 시스템에서 사용자 신분 확인
- 권한부여 (Authorization)** : 검증된 사용자에게 사용가능한 접근 권한 확인
- 계정관리 (Accounting)** : 사용자의 자원 사용에 대한 정보 수집 (과금, 감사, 보고서)

■ 양방향 암호화 방식

	대칭키 / 비밀키 / 개인키		비대칭키 / 공개키
특징	동일한 키로 데이터를 암호화/복호화		암호화 키는 DB 사용자에게 공개 복호화 키는 비밀키로 관리자만
	블록 (Block) 2 bit 이상 연산	스트림 (Stream) 1 bit 씩 연산	
종류	DES, AES, SEED, ARIA IDEA, LEA	RC4, LFSR	소인수분해 : RSA, Rabin 이산대수 : Diffie-Hellman, DSA 타원곡선 : ECC
키 개수	N(N-1)/2 개		2N 개
장점	알고리즘 단순 암호/복호화 속도 빠름		키 분배 용이 관리해야할 키의 수 적음
단점	관리해야할 키의 수 많음		알고리즘 복잡 암호/복호화 속도 느림

※ 양방향 암호화 종류

DES	IBM에서 개발, 미국 NBS에서 국가 표준으로 발표 블록 크기 64비트 / 키 길이 56비트 / 16라운드 (페이스펠 구조)
AES	미국 표준 기술 연구소 (NIST) 발표 (키 길이 128/192/256 비트) DES 대체 목적 / 레인달(Rijndael) 기반 암호화 / SNP 암호 방식
SEED	국내 개발 개인키 (128 및 256 비트 사용)
ARIA	국가보안기술연구소(NSRI) 주도 개발 (128/192/256 비트)
IDEA	스위스 개발 PES 개선한 암호키 (64비트 블록 / 128비트 키)
LEA	국내 NSRI 개발 (128비트의 데이터 블록을 암호화)
Skipjack	미국 NSA 개발 / 64비트 입출력, 80비트의 키, 32라운드 (전화기)
RC4	Ron Rivest가 설계한 스트림 암호화 (옥렛 단위 기반)
LFSR	선형 연산을 통한 다음 상태 생성 / 스트림 기반 난수 생성 활용
RSA	MIT 공개키 암호화 / 소인수분해 어려움 큰 소수 숫자 활용
Rabin	Rabin 개발 (소인수분해) / RSA보다 빠름
Diffie-Hellman	두 사용자가 사전에 어떤 비밀 교환 없이도 공통키를 교환 가능
DSA	미국 표준 디지털 서명 알고리즘
ECC	타원곡선 이론에 기반한 공개 키 암호 방식

■ 단방향 암호화 방식 - 해시 함수 (Hash)

- 해시 알고리즘(해시 함수)로 불리며, 해시 함수로 변환된 값/키를 해시값/키라고 함
- 임의의 길이의 입력 데이터를 받아 **고정된 길이의 해시 값**을 변환한다. (단방향 함수)
- 종류: SHA 시리즈 / HAVAL / MD4 / MD5 / N-NASH / SNEFRU

중첩법 (폴딩법)	레코드 키를 여러 부분으로 나누고, 나눈 부분의 각 숫자를 더하거나 XOR 한 값을 홈 주소로 사용하는 방식
제산법	레코드 키로 해시표의 크기보다 큰 수 중에서 가장 작은 소수로 나눈 나머지를 홈 주소로 삼는 방식
기수변환법	키 숫자의 진수를 다른 진수로 변환시켜 주소 크기를 초과한 높은 자릿수는 절단하고, 이를 다시 주소 범위에 맞게 조정하는 방식
숫자분석법	키 값을 이루는 숫자의 분포를 분석하여 비교적 고른 자리를 필요한 만큼 택해서 홈 주소로 삼는 방식

기타 : 제곱법, 무작위법

■ 서비스 공격 유형

서비스 거부 공격 (DOS: Denial of Service)	대량의 데이터를 한 곳의 서버에 집중적으로 전송하여 표적이 되는 서버의 정상적인 기능을 방해
분산 서비스 거부 공격 (DDoS)	여러 대의 장비에서 한 곳의 서버에 분산 서비스 공격 수행 종류 : Trinoo, Tribe Flood Network, Stacheldraht
Ping of Death	Ping 명령을 전송할 때 허용범위 이상의 ICMP 패킷 을 전송하여 대상 시스템의 네트워크를 마비
Ping Flooding	과도한 ICMP 메시지에 의한 응답 과다로 시스템 에러 유발
SYN Flooding	SYN 패킷 신호만 전송하여 각 서버의 가용 사용자 수를 선점 후 다른 사용자의 서버 접근을 차단
UDP Flooding	다량의 UDP 패킷을 전송하여 네트워크 자원을 고갈
Smurfing (스머핑)	IP / ICMP 특성 을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보내 네트워크를 불능 상태로 만들
TearDrop	Fragment number 값을 변형 하여 수신측에서 패킷 조립 시 오류로 인한 과부하로 시스템 다운을 유도
LAND Attack	송/수신 IP 주소를 모두 타겟 IP주소로 하여 자기 자신에게 무한히 응답하게 하는 공격
Evil twin attack	악의적 사용자가 지인/유명한 사칭하여 로그온한 사용자의 계정 정보 및 신용 정보 탈취
Switching Jamming	위조된 매체 접근 제어(MAC) 주소를 지속 보내 스위치 MAC 주소를 혼란시켜 더미 허브 (Dummy Hub)처럼 작동

■ 정보 보안 솔루션

방화벽 (Firewall)	내부 → 외부로 나가는 패킷은 그대로 통과 외부 → 내부로 유입되는 패킷은 인증된 패킷만 통과
웹 방화벽 (Web Firewall)	일반 방화벽은 탐지 불가한 SQL 삽입 공격, XSS 등 웹 기반 공격을 방어하는 목적으로 웹 서버에 특화된 방화벽
침입 탐지 시스템 (IDS)	Intrusion Detection System 시스템의 비정상적인 사용/오용/남용 등을 실시간 탐지
침입 방지 시스템 (IPS)	Intrusion Prevention System / 방화벽 + 침입 탐지 시스템 비정상적 트래픽/패킷을 능동적으로 차단하고 격리
데이터 유출 방지 (DLP)	Data Loss Prevention / 내부 정보의 외부 유출을 방지 사내 직원이 사용하는 시스템의 모든 정보 탐지/통제
VPN (가상 사설 통신망)	공공 네트워크 암호화 기술을 이용하여 사용자가 마치 자신의 전용 회선 을 사용하는 것처럼 하게 함
NAC (Network Access Control)	네트워크에 접속하는 내부 PC의 MAC 주소를 IP 관리 시스템에 등록된 후 일관된 보안 관리 기능을 제공
ESM (Enterprise Security Management)	상기 보안 솔루션에서 발생한 로그 기록을 통합하여 관리 → 종합적인 보안 관리 체계 수립 가능
SIEM	Security Information & Event Management 보안 경로의 실시간 분석 제공 로그 및 이벤트 통합 관리 / 빅데이터 기반 보안 솔루션
SDP	Software Defined Perimeter / '블랙 클라우드'라 불림 GIG 네트워크 우선권에 따라 DISA에서 수행한 작업에서 발전
Sandbox	외부에서 유입된 프로그램이 보호된 영역에서 동작해 시스템이 부정하게 조작되는 것을 막는 보안 형태
FDS	Fraud Detection System / 금융거래의 이상 거래 탐지 및 차단
템퍼 프루핑	Tamper Proofing / SW의 위·변조 시 SW 오작동시켜 악용 방지 해시함수, 펌거 프린트, 워터마킹 등 보안 요소 생성 후 보호
Trust Zone	독립적 보안 구역을 따로 두어 중요한 정보를 보호하는 하드웨어 기반의 보안 기술



※ 침입 탐지 시스템 (IDS: Intrusion Detection System)

- 시스템의 비정상 사용/오용/남용 등을 실시간 탐지

. **오용 탐지**: 사전에 정립되어 입력해 둔 공격 패턴 감지 시 통보

. **이상 탐지**: 평균적인 상태 기준에서 비정상적 행위 또는 자원의 사용 감지 시 통보

[종류]

HIDS (Host-based)	. 시스템 내부 감시/분석, 내부 변화를 실시간으로 감시하여 외부 침입자 작업 기록 및 추적 → OSSEC, md5deep, AIDE, Samhain
NIDS (Network-based)	. 외부 침입 감시/분석, 네트워크 트래픽 감시 → Snort, Zeek

- IDS 설치 가능 위치 : 패킷 라우터로 들어오기 전 / 라우터 뒤 / 방화벽 뒤 / **DMZ***

※ **DMZ** : 외부 인터넷에 서비스 제공하는 서버가 위치하는 네트워크

■ 정보 보안 프로토콜

SSH (Secure Shell)	. 서로 연결된 컴퓨터 간 원격 명령실행 및 셸 서비스 수행 . 키를 통한 인증은 클라이언트의 공개키를 서버에 등록해야 함 . 전송 데이터는 암호화되며, 기본적으로 22번 포트 사용
SSL (Secure Socket Layer)	. 웹 브라우저 ↔ 서버 간 안전한 데이터 전송 목적 . https로 시작 / 443번 포트 사용
TLS (Transport Layer Security)	. 인터넷 커뮤니케이션을 위한 개인 정보와 데이터 무결성을 제공하는 보안 프로토콜 / SSL의 개선 버전
IPSec	. IP 네트워크 계층을 안전하게 보호하기 위한 보안 프로토콜 → 전송 모드 (트래픽 경로 유출 가능) / 터널 모드 (패킷 전체) . AH(Authentication Header)와 ESP(Encapsulating Security Payload)라는 두 가지 보안 프로토콜을 사용
S-HTTP	. 웹상에서 네트워크 트래픽을 암호화 / 기존 HTTP의 보안 확장
TKIP (Temporal Key Integrity Protocol)	. 임시 키 무결성 프로토콜 / WEP 취약성을 보완하기 위해 암호 알고리즘의 입력 키 길이를 128비트로 늘림

■ 코드 오류

생략 오류 (Omission error)	입력시 한 자리를 빼놓고 기록	ABCD → ABD_
필사 오류 (Transcription error)	임의의 한 자리를 잘못 기록	ABCD → AB FD
전위 오류 (Transposition error)	입력 시 좌우 자리를 바꿔어 기록	ABCD → ACBD
이중 오류 (Double transposition)	전위 오류가 두 가지 이상 발생	ABCD → CABD
추가 오류 (Addition error)	입력 시 한 자리 추가로 기록	ABCD → ABCD E
임의 오류 (Random error)	다른 오류가 두 가지 이상 결합	ABCD → ACBDF

■ ISMS (Information Security Management System)

- 정보보호 관리 체계 : 정보 자산을 안전하게 보호하기 위한 보호 절차와 대책 확보 조직에 맞는 정보보호 정책을 수립하고 위험에 상시 대응가능한 보안 대책 통합 관리

■ 다크 데이터 (Dark Data)

- 특정 목적으로 수집된 데이터가 활용되지 않고 저장만 되어 있는 대량의 데이터

■ 정보 보안 침해 공격

해킹	시스템에 침입해 정보를 수정하거나 빼내는 행위
크래킹	시스템에 침입해 정보를 파괴하거나 변경하는 행위
좀비 PC	악성코드에 감염되어 다른 컴퓨터를 조종하는 행위
C&C 서버	해커가 감염된 좀비 PC에 명령을 내리고 악성코드를 제어하기 위한 용도로 사용하는 서버
웜(Worm)	다른 컴퓨터의 취약점을 이용하여 스스로 전파하거나 메일로 전파되며 스스로를 증식 (독자적으로 실행 가능)
바이러스	파일, 메모리 영역에 자신을 복제(기생)하는 악성 프로그램 (독자적으로 실행 불가)
트로이목마	정당한 프로그램으로 가장하여 숨어 있는 바이러스 (복제 X)
백도어	보안이 제거된 비밀통로로 무단 접근을 위한 통로(뒷문) ★ 탐지 방법 : 무결성 검사, 로그 분석, SetID 파일, 열린 포트 검사
랜섬웨어	내부 문서 파일 등을 암호화해 사용자가 열지 못하게 하고 이를 인질로 금전을 요구하는 데 사용되는 악성 프로그램
APT (Advanced Persistent Threat)	지능형 지속 공격 / 특정 개인 및 조직을 타겟으로 침투 후 내부 서버 제어권 획득하여 무력화된 시스템 상의 데이터 수집
Qshing (큐싱)	QR코드를 통해 악성 앱 다운로드 유도하거나 설치
XSS	Cross Site Scripting / 브라우저 스크립트 취약점을 악용 특정 링크 클릭 시 악성 스크립트 실행되어 개인 정보 탈취
CSRF	Cross-Site Request Forgery / 사용자가 공격자가 의도한 행위(수정, 삭제, 등록 등)를 특정 웹사이트에 요청하게 하는 공격
제로 데이 공격	발견된 취약점의 존재를 공표하기 전에 해당 취약점으로 이용한 보안 공격 (해킹의 신속성)
스니핑	패킷을 엿보면서 계정 정보(ID/PW)를 가로채는 행위
스푸핑 (Spoofing)	검증된 사람이 네트워크를 통해 데이터를 보낸 것처럼 데이터를 변조하여 접속을 시도하는 일종의 속임수 . IP Spoofing : 공격자가 자신의 IP 주소를 다른 주소로 위장 . ARP Spoofing : 공격자의 MAC 주소를 다른 컴퓨터 주소로 위장 . DNS Spoofing : DNS IP 주소를 중간에서 조작하여 위장
세션 하이재킹 (Session Hijacking)	서버에 접속하고 있는 클라이언트들의 세션 정보를 가로채는 공격 Reset 패킷을 통해 강제 종료시킨 후 재연결 시 침입자에게 연결
스피어 피싱	불특정 다수에게 메일 발송 후 가짜 위장 사이트로 유인한 후 금융기관 관련 개인 정보를 빼내는 행위
스미싱	문자 메시지(SMS) 통한 사용자 개인 신용 정보 탈취
파밍	금융기관의 도메인 주소(DNS)를 중간에 가로채 사용자가 금융기관 사이트에 접속한 듯 착각하게 하여 개인정보 탈취
타이포스쿼팅 (Typosquatting)	사용자가 웹 URL 주소를 잘못 입력하는 실수를 악용
웨일링 (Whaling)	CEO, 고위 경영진, 연예인 등 유명인사를 타겟으로 한 스피어 피싱
키로거	키 입력 캐치 프로그램을 이용하여 개인정보를 빼내어 악용하는 행위
버퍼 오버플로우	할당된 메모리의 범위를 넘어선 위치에서 자료를 사용할 때 발생
SQL 삽입 공격 (Injection Attack)	임의의 SQL 문을 주입하고 실행되게 하여 데이터베이스가 비정상적인 동작을 하도록 조작하는 코드 삽입 공격
Brute Force Attack	무작위 대입 공격 / 가능한 모든 문자열 조합으로 암호키 탈취
Watering Hole Attack	타겟이 주로 방문하는 웹 사이트 감염시킨 후 해당 타겟이 감염된 웹 사이트 방문할 때까지 기다리는 공격
Bluebug	블루투스 연결 취약점을 이용한 공격
Credential Stuffing	공격자가 여러 가지의 경로로 수집한 사용자들의 로그인 인증 정보(Credential)를 다른 사이트의 계정 정보에 무작위 대입(Stuffing)
Island Hopping	타겟 기업을 침해하기 위해 보안이 더 취약한 협력사, 파트너 기업의 네트워크를 해킹
사회 공학 (Social Engineering)	컴퓨터 보안에 있어서 인간 상호 작용의 깊은 신뢰를 바탕으로 사람들을 속여 보안 절차를 깨뜨리기 위한 비기술적 침입 수단
Rootkit	. 권한이 없는 사용자가 접근할 수 없는 영역에 접근하여 시스템을 제어하도록 설계된 악성 소프트웨어의 모음 . 해킹에 사용되는 기능들을 제공하는 프로그램들의 모음



■ 웹 관련 용어

- . **Hypertext** : 다른 문서/그림으로 이동할 수 있는 연결을 가지고 있는 텍스트
- . **HTML** : 하이퍼텍스트 및 웹 페이지를 만들 수 있는 마크업 언어
- . **URL** : 인터넷 상에서 특정 웹 페이지의 위치를 나타내는데 사용되는 문자열
- . **MIME** : 이메일에서 텍스트 외 이미지/비디오/오디오 등 여러 유형의 파일을 전송 가능

■ 웹 서비스

SOAP	. Simple Object Access Protocol . 다른 컴퓨터 내 데이터나 서비스를 호출하기 위한 통신규약 프로토콜 . XML 기반 메시지를 다른 컴퓨터 네트워크 상에서 교환
WSDL	. Web Services Description Language (웹 서비스 기술언어) . 웹 서비스에서 제공하는 서비스에 대한 정보를 XML 기반으로 기술 (서비스 제공 장소, 서비스 메시지 포맷, 프로토콜 등)
UDDI	. Universal Description Discovery and Integration . 전역 비즈니스 레지스트리 / 웹 서비스 관련 정보 공개 및 탐색 → 웹 서비스 제공자가 UDDI라는 온라인 저장소에 자신들의 서비스를 등록/저장하고, 소비자들은 그 저장소에 접근 후 원하는 서비스 탐색

■ 인터페이스 구현 기술

XML	. eXtensible Markup Language . HTML 문법의 비호환성과 SGML의 복잡성 해결하기 위해 개발
AJAX	. Asynchronous Javascript and XML . Javascript, XML을 이용한 비동기식으로 웹 페이지의 일부 콘텐츠만 Reload 해오는 방식 → HTML 보다 다양한 작업을 웹 페이지에서 구현 가능
JSON	. JavaScript Object Notation . 속성-값 쌍 (Attribute-Value Pairs) 으로 이뤄진 데이터 오브젝트를 전달하기 위해 사용하는 개방형 표준 포맷 → <u>AJAX</u> 에서 많이 사용 / XML을 대체하는 주요 데이터 포맷

■ 인터페이스 구현 검증 도구

xUnit	Java, C++, Net 등 다양한 언어 지원하는 단위 테스트 프레임워크
JUnit	자바 프로그래밍 언어용 유닛 테스트 프레임워크 같은 테스트 코드를 여러 번 작성하지 않게 함
STAF	서비스 호출, 컴포넌트 재사용 등 다양한 환경을 지원 크로스 플랫폼, 분산 소프트웨어 테스트 환경을 조성
Fitness	웹 기반 테스트케이스 설계/실행/결과 확인 지원
NTAF	STAF의 재사용 및 확장성 + Fitness의 협업 기능 통합 (현재는 폐기됨)
Selenium	다양한 브라우저 및 개발 언어를 지원하는 웹 어플리케이션 테스트
watir	Ruby 언어 기반 어플리케이션 테스트 프레임워크

■ 트리 순회 방법

전위 순회 (Pre-order)	A - B - D - E - H - I - C - F - G - J
중위 순회 (In-order)	D - B - H - E - I - A - F - C - G - J
후위 순회 (Post-order)	D - H - I - E - B - F - J - G - C - A

■ 클라우드 서비스

IaaS	. Infrastructure as a Service . 물리적 자원(서버/네트워크/스토리지)을 가상화
Paas	. Platform as a Service . 응용프로그램 개발 시 필요한 플랫폼(OS/미들웨어/런타임) 제공
SaaS	. Software as a Service . 사용자에게 제공되는 소프트웨어(데이터/앱)를 가상화하여 제공
BaaS	. Blockchain as a Service . 블록체인 기반 서비스 개발/관리의 편의성을 제공하는 클라우드
SecaaS	. Security as a Service . 여러 모델의 클라우드 시스템 기반으로 보안 서비스를 제공
Daas	. Desktop as a Service . 각종 업무용 운영체제/SW 등을 클라우드 방식으로 제공 (재택근무 활용)
FaaS	. Function as a Service . 사용자가 앱 개발 및 런칭에 관련한 하부 구조의 복잡한 빌드, 유지보수 없이 애플리케이션 기능을 개발, 실행, 관리할 수 있도록 플랫폼을 제공

■ RAID (Redundant Array of Inexpensive Disk)

- 여러 개의 디스크를 한 개의 디스크처럼 관리하는 기술 (서버에 주로 사용)
- 데이터 안정성 높고 복구 용이함, 빠른 전송 속도
- **Level값이 클수록 저장장치 높은 신뢰성/효율성**
 - ① **스트라이핑** - 데이터를 여러 개의 디스크에 분할하여 저장
→ 하나라도 손상 시 데이터 복구 불가
 - ② **미러링** - 데이터를 2개의 디스크에 동일하게 저장
→ 한 쪽 손상 시 다른 쪽 이용하여 복구 가능

RAID 0	스트라이핑 방식 / 중복 저장 X, 에러검출 X
RAID 1	미러링 방식 / 중복 저장 O, 높은 신뢰도
RAID 2	스트라이핑 방식 / 해밍코드 활용 에러 검증
RAID 3	스트라이핑 방식 / 패리티(8bit)를 여러 검증 , 바이트 단위로 데이터 저장
RAID 4	RAID 3과 동일하나, 데이터를 블록 단위로 나눠 저장
RAID 5	스트라이핑 방식 / 패리티 블록을 각 디스크마다 분산 저장
RAID 6	스트라이핑 방식 / 패리티 블록을 이중 구조 로 구축

※ EAI (Enterprise Application Integration)

- 기업 내 운영되는 플랫폼 및 애플리케이션 간의 정보전달, 연계, 통합 수행
- Point to Point, Hub&Spoke, Message Bus, Hybrid 형태로 구성

※ FEP (Front-End Processor)

- 입력 데이터를 프로세스가 처리하기 전에 미리 처리하여 프로세스 처리 시간을 줄여주는 프로그램

■ 디지털 저작권 관리 (DRM, Digital Rights Management)

- 콘텐츠 제공자/분배자/소비자, 패키지 (배포가능한 형태로 암호화하는 프로그램)
- 클리어링 하우스 (Clearing House) : 사용 권한, 라이선스 발급, 결제 관리
- DRM 컨트롤러 : 배포된 콘텐츠의 이용 권한을 통제하는 프로그램
- 보안 컨테이너 : 콘텐츠 원본을 안전하게 유통하기 위한 전자적 보안 장치
→ 기술 요소 : **암호화 / 키 관리 / 식별 기술 / 저작권 표현 / 암호화 파일 생성 정책 관리 / 크랙 방지 / 인증**

■ UNIX / LINUX 기본 명령어

cat	파일 내용을 화면에 표시	fsck	파일 시스템 검사/보수
chdir	현재 사용할 디렉터리 위치 변경	getpid	자신의 프로세스 아이디 호출
chmod	파일의 보호 모드 설정	getppid	부모 프로세스 아이디 호출
chown	소유자 변경	ls	디렉터리 내 파일 목록 확인
cp	파일 복사	rm	파일 삭제
exec	새로운 프로세스 수행	wait	상위 프로세스가 하위 프로세스 종료 등의 event 기다림
fork	새로운 프로세스 생성		



■ SPICE (ISO 15504) → 6단계 / 소프트웨어 처리 개선 및 능력 평가 기준 (Software Process Improvement and Capability Determination)

1) 불완전	프로세스가 구현되지 않거나 목적 달성 불가
2) 수행	목적은 달성하지만, 계획/축적 불가
3) 관리	프로세스 수행이 계획되고 관리됨
4) 확립	표준 프로세스를 이용하여 계획/관리
5) 예측 가능	표준 프로세스 능력에 대해 정량적 이해/성능 예측
6) 최적	정의된 프로세스와 표준 프로세스가 지속적으로 개선

■ IT 용어

Stack guard	Stack 상 일정 주소 번지에 프로그래머가 유도하는 Canary를 심고, 스택이 붕괴/변조된 경우에 오버플로우 상태로 가정하여 Canary 체크 후 프로그램 실행을 비정상적으로 중단시키는 방법
Docker	컨테이너 기술을 자동화하여 쉽게 사용하는 오픈소스 프로젝트 SW 컨테이너 안에 응용 프로그램 배치 자동화 역할
Cipher Container	JAVA에서 암호화/복호화 기능을 제공하는 컨테이너
Scytale	암호화 기법으로 문자열의 위치를 바꾸는 방법
Tensor Flow	2015년 공개된 구글 브레인 팀의 기계 학습 오픈소스 SW Library
One Seg	일본/브라질에 상용 중인 디지털 TV 방송 기술
Foursquare	위치 기반 소셜 네트워크 서비스
PaaS-Ta	국내 IT 서비스 경쟁력 강화 목적 개발된 개방형 클라우드 인프라 제어, 관리/실행/개발/서비스/운영 환경으로 구성
VLAN (Virtual LAN)	물리적 배치와 상관없이 논리적으로 LAN 구성하는 기술 접속된 장비들의 성능 향상 / 보안성 증대 효과
SSO (Single Sign On)	한 번의 로그인으로 다른 사이트 로그인도 허용하는 시스템
MQTT	. Message Queuing Telemetry Transport TCP/IP 기반 네트워크에서 발행-구독 기반의 메시징 프로토콜 푸시 기술 기반 경량 메시지 전송 프로토콜 (IBM 개발 주도)
Salt	동일한 패스워드들을 다른 암호 값으로 저장되도록 덧씌우는 무작위의 값 → 같은 패스워드임에도 다른 결과 산출
N-screen	N개의 서로 다른 단말기에서 동일 콘텐츠를 자유롭게 이용
ASLR	. Address Space Layout Randomization 프로그램 실행마다 스택/힙/라이브러리 주소를 랜덤화하여 공격자로 하여금 메모리 상 주소 예측을 어렵게 함
라우터 (Router)	서로 다른 네트워크 대역에 있는 호스트를 상호 간에 통신할 수 있도록 해주는 네트워크 장비
nmap	서버에 열린 포트 정보를 스캐닝하여 보안취약점을 찾는 도구
Tripwire	크래커가 침입하여 백도어를 만들어 놓거나, 설정파일을 변경했을 때 분석하는 도구
Smart Grid	정보 기술을 활용하여 전력망 지능화, 고도화함으로써 고품질의 전력서비스를 제공하고 에너지 이용 효율을 극대화
서비스 지향 아키텍처 (SOA)	구성 계층 : 표현 / 업무 프로세스 / 서비스 중간 / 애플리케이션 / 데이터 저장
Digital twin	물리적 사물을 가상화하여(twin) 실제 자산의 특성 정보를 구현 → 자산 최적화 / 돌발 사고 최소화 / 생산성 증가
Mashup	웹에서 제공하는 정보/서비스를 이용하여 새로운 SW 제작
Mesh Network	차세대 이동통신, 홈네트워킹 등 특수 목적을 위한 새로운 방식의 네트워크 기술 / 대규모 네트워크 생성에 최적화
PICONET	여러 개의 독립된 통신장치가 블루투스 기술이나 UWB 통신 기술을 사용하여 통신망을 형성하는 무선 네트워크 기술

■ URL 구조

<https://ryan:2457@www.dream.com:443/index?id=5&page=1#bottom>

User info
 → 스킴:// 사용자이름:비밀번호@호스트:포트 / 경로?질의#프래그먼트
 Authority

스킴 (Scheme)	사용할 프로토콜 명시 (리소스 요청 및 접근 방법) ex. http, https, ftp, mailto(이메일), rtsp(스트리밍) 등
사용자 정보	User info : 사용자 이름 + 비밀번호
호스트 (Host)	도메인명 또는 IP 주소
포트 (Port)	Socket(소켓) 구분 역할 생략 시 http는 80, https는 443로 기본값 포트번호 설정
경로 (Path)	호스트에서 제공하는 자원의 경로 (계층적 구조)
질의 (Query)	Query parameter 또는 Query string으로 불림 key=value 형태, &로 질의 추가 (ex. ?directoryNo=0¤tPage=1)
프래그먼트 (Fragment)	내부 북마크 기능 (웹문서 해시태그) / 해시태그로 이동을 원하는 요소 의 id를 링크로 연결 시 스크롤 이동없이 해당 위치로 바로 이동 가능



■ SQL (Structured Query Language) 분류

① 데이터 정의어 (DDL : Data Definition Language) → 논리/물리적 데이터 구조 정의

CREATE (생성)	CREATE DOMAIN / SCHEMA / TABLE / VIEW / INDEX → 생성
ALTER (변경)	TABLE 이름 변경 → ALTER TABLE / 컬럼 추가
DROP (삭제)	DROP DOMAIN / SCHEMA / TABLE / VIEW / INDEX → 삭제 ※ CASCADE : 참조하는 모든 개체 함께 제거 ※ RESTRICT : 제거할 요소를 다른 개체가 참조 시 제거 취소

② 데이터 조작어 (DML : Data Manipulation Language)

SELECT (검색)	SELECT FROM 테이블명 [WHERE 조건];
INSERT (삽입)	INSERT INTO 테이블명 VALUES 데이터;
DELETE (삭제)	DELETE FROM 테이블명 [WHERE 조건];
UPDATE (변경)	UPDATE 테이블명 SET 속성명 = 데이터 [WHERE 조건];

※ SELECT 구문 예시

SELECT [DISTINCT] 필드 이름	SELECT 이름 FROM 학생	이름을 산출 / 학생 테이블에서
FROM 테이블명	WHERE 수학>=80;	수학이 80 이상인 자료만
[WHERE 조건식]	GROUP BY 학년	학년 필드를 그룹으로 묶어서
[GROUP BY 필드이름]	HAVING COUNT(*)>=2	레코드 개수가 2이상인 그룹만
[HAVING 그룹 조건식]	ORDER BY 나이 ASC or 생략	나이 오름차순 정렬 (기본)
[ORDER BY 정렬]	ORDER BY 나이 DESC	나이 내림차순 정렬

※ DISTINCT : 중복 레코드 제거한 조회 결과 출력

OR	① 부서='경리' OR 부서='영업' ② IN('경리', '영업')		
AND	① 생일>=#2001-1-1# AND 생일<=#2002-12-31# ② BETWEEN #2001-1-1# AND #2002-12-31#		
만능문자	LIKE '박%' → 박으로 시작되는 모든 글자 LIKE '_은' → 은으로 끝나는 두 글자		
빈 칸	IS NULL	부정문	NOT

③ 데이터 제어어 (DCL : Data Control Language)

→ 데이터 보안/복구/병행수행 제어무결성 유지

COMMIT	수행된 결과를 실제 물리적 디스크로 저장 후 작업의 정상 완료 통보
ROLLBACK	COMMIT 수행되지 않은 작업은 취소하고, 이전 상태로 원복 ※ SAVEPOINT : 트랜잭션 내 ROLLBACK할 저장 위치 지정
GRANT	데이터베이스 사용자에게 사용 권한 부여 → GRANT 권한 리스트 ON 개체 TO 사용자 [WITH GRANT OPTION] 부여받은 권한을 다른 사용자에게 재부여
REVOKE	데이터베이스 사용자의 사용 권한 취소 → REVOKE [GRANT OPTION FOR] 권한 리스트 ON 개체 FROM 사용자 다른 사용자에게 권한 부여 가능한 권한을 취소

[실전 예제 -1] 23년 3회 실기

다음 두 테이블에 대해 SQL문을 실행하였을 때, 나타나는 결과 작성 (속성명 + 값 출력)

A	B	A	B	SELECT A FROM L UNION SELECT A FROM R ORDER BY A DESC;	A
2	X	1	Y		4
1	Y	2	Z		3
3	Z	4	X		2
					1

[L]

[R]

* UNION : 두 집합의 합 (중복은 제거) → 내림차순으로 정렬

[실전 예제 -2] 23년 2회 실기

다음 처리 조건에 맞는 쿼리문 작성

- 학생 테이블에 학번이 1415245, 이름이 '라이언', 학년이 2, 과목이 '수학', 연락처가 '010-1234-5678'인 학생의 정보를 입력
- 인용 부호가 필요한 경우 경우 작은 따옴표(' ') 사용
- 명령문 마지막 세미콜론(;)은 생략 가능

[학생] 테이블

속성명	데이터 타입	비고
학번	INT	PRIMARY KEY
이름	VARCHAR(20)	
학년	INT	
과목	VARCHAR(20)	
연락처	VARCHAR(20)	

INSERT INTO 학생(학번, 이름, 학년, 과목, 연락처)
VALUES(1415245, '라이언', 2, '수학', '010-1234-5678')

[실전 예제 -3] 23년 1회 실기

[학생] 테이블에서 학생 이름이 '지영'인 튜플을 삭제하는 쿼리 작성

[학생] 테이블

순번	학년	이름
1	1	나희
2	2	지영
3	2	소울
4	3	이안



DELETE FROM 학생 WHERE 이름 = '지영';

★ 속성인 [이름]에는 따옴표 붙이지 않기!

[실전 예제 -4] 23년 1회 실기

다음 테이블과 SQL문 결과 데이터를 확인하여 처리한 SQL문 작성
(과목별 그룹을 묶었을 때 과목 평균이 90이상인 과목에 대해서만 출력)

[성적] 테이블

순번	과목	점수
1	데이터베이스	89
2	데이터베이스	94
3	네트워크	45
4	소프트웨어	68
5	네트워크	95
6	소프트웨어	74

결과

과목	최소점수	최대점수
데이터베이스	89	94

- WHERE 사용 금지
- SELECT절에 별칭을 사용하여 작성
- 반드시 GROUP BY와 HAVING을 사용
- 집계함수를 사용
- SQL문 마지막 세미콜론(;) 생략 가능

SELECT 과목, MIN(점수) AS 최소점수, MAX(점수) AS 최대점수 FROM 성적
GROUP BY 과목
HAVING AVG(점수)>=90

[실전 예제 -5] 22년 3회 실기

학생(STUDENT) 테이블에 재료과 학생이 50명, 전기과 학생이 100명, 컴퓨터공학과 학생이 50명 있다고 할 때, 아래 1~3번 SQL문 실행 결과로 표시되는 튜플의 수 구하기
(DEPT 필드는 학과를 의미)

1번. SELECT DEPT FROM STUDENT;

- ▶ 재료과 50명, 전기과 100명, 컴퓨터공학과 50명이 나열된 전체 학생 테이블로 전체 튜플(행)의 개수는 200

2번. SELECT DISTINCT DEPT FROM STUDENT;

- ▶ 앞서 1번에서 실행된 전체 학생 테이블에서 중복된 학과명을 제거 총 3개의 학과명만 한 행씩 출력 → 그때 행의 개수는 3

3번. SELECT COUNT(DISTINCT DEPT) FROM STUDENT WHERE DEPT = '재료과';

- ▶ 학생 테이블에서 학과가 재료과인 튜플만 출력 (이때 학과가 중복된 튜플은 제거) 최종 재료과 한 행만 출력 → 행의 개수는 1



[실전 예제 - 6] 22년 3회 실기

다음과 같이 테이블을 정의하고 튜플을 삽입하였을 때 아래 1, 2번의 SQL문 결과 작성

```
CREATE TABLE 부서 (
  부서코드 INT PRIMARY KEY,
  부서명 VARCHAR(20)
);
CREATE TABLE 직원 (
  직원코드 INT PRIMARY KEY,
  부서코드 INT
  FOREIGN KEY(부서코드) REFERENCES 부서(부서코드)
  ON DELETE CASCADE
);
INSERT INTO 부서 VALUES(10, '영업팀');
INSERT INTO 부서 VALUES(20, '개발팀');
INSERT INTO 부서 VALUES(30, '기획팀');
INSERT INTO 직원 VALUES(1000, 10, '오지은');
INSERT INTO 직원 VALUES(2000, 10, '황나영');
INSERT INTO 직원 VALUES(3000, 20, '김새희');
INSERT INTO 직원 VALUES(4000, 20, '이슬기');
INSERT INTO 직원 VALUES(5000, 20, '성예은');
INSERT INTO 직원 VALUES(6000, 30, '박서희');
INSERT INTO 직원 VALUES(7000, 30, '김지현');
```

[부서] 테이블

부서코드	부서명
10	영업팀
20	개발팀
30	기획팀

[직원] 테이블

직원코드	부서코드	직원명
1000	10	오지은
2000	10	황나영
3000	20	김새희
4000	20	이슬기
5000	20	성예은
6000	30	박서희
7000	30	김지현

1번. SQL문

```
SELECT DISTINCT COUNT(부서코드) FROM 직원
WHERE 부서코드 = 20;
```

▶ 직원 테이블에서 부서코드가 20인 튜플의 개수 = 3 (정답)

★ [부서코드] 자체가 아닌 부서코드의 '개수' (=COUNT(부서코드)) 이므로 DISTINCT는 아무 역할을 하지 않음을 주의! (아래 2번도 동일)

2번. SQL문

```
DELETE FROM 부서 WHERE 부서코드 = 20;
SELECT DISTINCT COUNT(부서코드) FROM 직원
```

▶ 부서 테이블에서 부서코드가 20인 튜플 삭제

→ 직원 테이블 내 부서코드는 부서 테이블 내 부서코드와 외래키로 연결되어 있기에 직원 테이블 내 부서코드가 20인 튜플도 같이 삭제 (DELETE CASCADE)

▶ 이후 직원 테이블에서 부서코드의 개수 = 4 (정답)

직원코드	부서코드	직원명
1000	10	오지은
2000	10	황나영
6000	30	박서희
7000	30	김지현

[실전 예제 - 7] 22년 2회 실기

아래는 <제품>(제품명, 단가, 제조사) 테이블을 대상으로 "C" 제조사에서 생산한 제품들의 '단가' 보다 높은 '단가'를 가진 제품의 정보를 조회하는 SQL문이다.

이때, 괄호 () 안에 알맞은 답을 넣어 SQL문 완성하기

```
SELECT 제품명, 단가, 제조사 FROM 제품
WHERE 단가 > ( ) (SELECT 단가 FROM 제품 WHERE 제조사 = "C");
```

① 제조사가 "C"인 단가들을 출력

② 앞서 1번에서 출력된 C 제조사의 단가들 보다 '모두' 큰 단가들이 출력되어야 함
→ ALL (정답)

[실전 예제 - 8] 22년 2회 실기

다음 <TABLE>을 참조하여 SQL문을 실행했을 때 출력 결과 구하기
(<TABLE> 내 'NULL'은 값이 없음을 의미)

INDEX	COL1	COL2
1	2	NULL
2	5	8
3	3	5
4	7	3
5	NULL	3

```
SELECT COUNT(COL2)
FROM TABLE
WHERE COL1 IN (2, 3) OR COL2 IN (3, 5)
```

▶ COL1의 값이 2 또는 3이거나 COL2의 값이 3 또는 5인 튜플에서 COL2 값만 출력
이때, NULL은 COUNT에 적용되지 않으므로 총 개수는 3

[실전 예제 - 9] 22년 1회 실기

다음 SQL문을 보고 SQL문의 Score 결과가 내림차순으로 정렬되도록 괄호 채우기

[성적] 테이블

	NAME	SCORE
1	PARK	95
2	KIM	90
3	LEE	60

```
SELECT NAME, SCORE FROM 성적
( ① ) BY ( ② ) ( ③ )
```

▶ ①: ORDER / ②: SCORE / ③: DESC

[실전 예제 - 10] 21년 3회 실기

다음은 테이블에서 조건값을 실행한 화면으로 이에 대한 알맞은 결과값 구하기

[A] 테이블

NAME
MARTIN
SCOTT
SMITH

[B] 테이블

RULE
S%
%T%

→ S로 시작하는 모든 문자
→ T가 존재하는 모든 문자

```
SELECT COUNT(*) CNT FROM A CROSS JOIN B
WHERE A.NAME LIKE B.RULE
```

[CNT] 테이블

A.NAME	B.RULE
MARTIN	S%
MARTIN	%T%
SCOTT	S%
SCOTT	%T%
SMITH	S%
SMITH	%T%

CROSS JOIN 후 테이블
(JOIN해서 나올 수 있는 모든 행의 조합)

▶ Cross Join된 우측 테이블에서 B.RULE의 조건을 만족하는 A.NAME 행의 개수는 총 5개 (빨간 박스) = 5 (정답)

[실전 예제 - 11] 21년 2회 실기

'이'씨 성을 가진 사람의 이름을 내림차순으로 출력하는 SQL문의 괄호 채우기

```
SELECT ... FROM ... WHERE 이름 LIKE ( A ) ORDER BY 이름 ( B )
```

▶ A: '이%' / B: DESC (정답)

[실전 예제 - 12] 21년 2회 실기

학생 테이블의 점수가 90점 이상인 학생의 과목평가를 'A'로 수정하는 SQL문 작성

```
( A ) 학생
( B )
과목 평가 = 'A'
WHERE 점수 >= 90
```

▶ A: UPDATE
B: SET

[실전 예제 - 13] 21년 2회 실기

학생 정보와 학과 정보를 조인하여 결과값을 출력하는 SELECT문 작성

```
SELECT * FROM 학생정보 A JOIN 학과정보 B
( ① ) A.학과 = B.( ② )
```

▶ ①: ON
②: 학과

[실전 예제 - 14] 21년 1회 실기

다음 <TABLE>을 참조하여 SQL문을 실행했을 때 출력 결과 구하기

[급여] 테이블

EMPNO	SAL
100	1000
200	3000
300	1500

```
SELECT COUNT(*) FROM 급여
WHERE EMPNO > 100 AND SAL >= 3000 OR EMPNO = 200;
```

▶ EMPNO가 100 초과이고, SAL이 3000 이상이거나 EMPNO가 200인 행의 개수 = 1 (정답)



[실전 예제 - 15] 24년 1회 실기

다음 두 테이블에 대해 SQL문을 실행하였을 때, 나타나는 결과 작성 (속성명 + 값 출력)

```
SELECT B
FROM R1
WHERE C IN (SELECT C FROM R2 WHERE D = 'k');
```

① Sub 쿼리

A	B	C
1	a	x
2	b	x
1	c	w
3	d	w

[R1]

C	D	E
x	k	3
y	k	3
z	s	2

[R2]

① Sub 쿼리: R2 테이블에서 D 속성의 값이 k인 C 속성의 값 산출

② Main 쿼리: R1 테이블에서 C 속성의 값이 x, y인 B 속성의 값 산출

C
x
y

B
a
b

[실전 예제 - 16] 24년 1회 실기

다음 테이블에 대해 SQL문을 실행하였을 때, 나타나는 결과 작성

EMPNO	SAL
100	1000
200	3000
300	1500

[TABLE1]

```
SELECT COUNT(*)
FROM TABLE1
WHERE EMPNO > 100 AND SAL >= 3000 OR EMPNO = 200;
```

WHERE 조건 : EMPNO 100 초과이면서 SAL 3000 이상이거나
또는 EMPNO가 200인 것

EMPNO	SAL
200	3000

[실전 예제 - 17] 24년 2회 실기

다음 SQL 구문의 빈칸 작성

- (1) INSERT INTO 사원 (사원번호, 이름, 주소, 부서) [(1)] (32431, '정실기', '서울', '영업');
- (2) INSERT INTO 부서 (사원번호, 이름, 주소, 부서)
[(2)] 사원번호, 이름, 나이, 23 FROM 사원 WHERE 이름 = '정실기';
- (3) SELECT * [(3)] 사원;
- (4) UPDATE 사원 [(4)] 부서 = '퇴사' WHERE 사원번호 = 32431;

- (1) 신입사원 정보(사원번호:32431, 이름:정실기, 주소:서울, 부서:영업)를 사원 테이블에 추가
- (2) 위 신입사원 정보를 부서 테이블에 추가
- (3) 전체 사원 테이블 조회
- (4) 사원 테이블에서 사원번호 32431인 사람의 부서 값을 '퇴사'로 변경

[정답] ①: VALUES ②: SELECT ③: FROM ④: SET

[실전 예제 - 17] 24년 2회 실기

다음 SQL 구문의 빈칸 작성

```
SELECT count(*)
FROM employee AS e JOIN project AS p ON e.project_id = p.project_id
WHERE p.name IN (
    SELECT name FROM project p WHERE p.project_id IN (
        SELECT project_id FROM employee GROUP BY project_id HAVING count(*) < 2
    )
);
```

② Sub 쿼리

no	first_name	last_name	project_id
1	John	Doe	10
2	Jim	Carry	20
3	Rachel	Redmond	10

[employee]

project_id	name
10	Alpha
20	Beta
10	Gamma

[project]

① Sub 쿼리: employee 테이블에서 project_id 산출!
이때, project_id는 그룹 지정하고,
행 개수가 2개 미만인 것만 산출
(여기서, project_id 값이 10인 행은 2개이므로 제외)

project_id
20

② Sub 쿼리: project 테이블에서 project_id가 20인
name 속성의 값 산출!

name
Beta

③ Main 쿼리: employee 테이블은 p로 하고, project 테이블은 e로 지정 (as 구문)
이때, p (project) 테이블 내 name 값이 Beta인 것의 개수!

[정답] 1

기출풀이집 1 프로그래밍 언어편

정보처리기사 실기

목 차

1. Python	1
2. Java	10
3. C언어	34



안녕하세요! 꿈꾸는라이언입니다.

먼저 정처기 필기 합격을 진심으로 축하드려요! 🎉 🎊

이제 실기를 준비하시는 분들을 위해 **프로그래밍 언어편 요약 노트**를 준비했습니다.

요즘 정처기 실기 시험의 프로그래밍 언어 비중이 매우 높아졌어요. 전체 20문항 중 8~10문제가 프로그래밍 언어 관련 문제라니 절반에 가까운 수준이죠. 😱 따라서 프로그래밍 언어를 제대로 준비하지 않으면 아무리 이론을 잘 알아도 합격이 쉽지 않은게 현실입니다.

실기에서 다루는 프로그래밍 언어는 C언어, 자바, 파이썬인데요, **출제 비중은 C언어(3~4문제) > 자바(2~3문제) > 파이썬(1~2문제) 순**이에요. 그래서 점수의 비중으로는 **C언어가 가장 중요하고, 그 다음으로 자바와 파이썬 순**이에요! 하지만 이 노트에서는 오히려 **정말 유형도 적고 이해하기 쉬운 파이썬부터 시작해서 자바, C언어 순서로 정리**했어요. 처음 프로그래밍 언어를 접하시는 분들도 차근차근 따라오실 수 있게요! 😊

1. **프로그래밍 언어에서는 기출 유형이 반복돼요!** 그래서 20년부터 23년도까지의 모든 프로그래밍 언어 기출문제를 분석하여 자주 출제되는 유형을 요약했어요. 단순 암기식 문제부터 응용력을 요구하는 문제까지 다양한 레벨의 문제가 있지만, 결국 **같은 유형이 반복**되고 있었어요. 이 유형만 확실히 잡으시면 프로그래밍 언어 문제는 충분히 고득점 받으실 수 있을 거예요! 🍀
2. **모든 코드를 이해할 필요는 없어요!** 코드 한줄 한줄 이해를 하고 넘어가면 정말 좋지만, **문제 풀이에 있어서는 출제 유형을 빠르게 파악하는 것이 관건**입니다. 실제로 많은 문제들이 숫자만 바뀌어서 동일한 유형으로 출제되거든요. 예를 들어 완전수를 찾는 문제가 매번 나오는데, 사실 완전수는 100이하에서는 6과 28뿐이에요. 그런데 문제의 코드를 전부 해석하려 하면 손으로 100번까지 계산해야 할 수도 있어요. 😲 하지만 완전수의 특징만 알고 있다면 30초 안에 답을 찾을 수 있죠. 그러니 모든 코드를 이해하는데 집중하기보다는 먼저 어떤 유형의 문제인지를 캐치하는 것이 효과적인 전략이 될 거예요. 이를 위해 제가 정리한 분석을 통해 반복적으로 출제되는 유형을 정확히 파악하시는 걸 추천드려요!
 추가로 각 기출 문제마다 아래와 같이 **난이도**를 표시했어요! **낮은 난이도부터 우선해서 학습해주세요!**
 - ★☆☆ : 반드시 맞춰야 하는 쉬운 문제
 - ★★☆ : 처음에는 다소 어렵지만 합격을 위해서는 도전해야 하는 문제
 - ★★★ : 실제 시험에서는 풀지 말아야할 정도로 어려운 문제

프로그래밍의 실행 순서가 복잡한 문제의 경우, 작은 숫자(①②...)로 코드 진행 순서를 각 코드 앞에 표기해놨어요!

3. **유튜브에 풀이가 있어요!** 모든 문제에 대한 효율적인 풀이 및 코드에 대한 한 줄 한 줄 설명을 적었지만, 이해가 어려운 경우에는 모든 문제에 기출 연도와 회차를 적어두었으니 유튜브에 검색해서 자세한 설명을 보시는 것도 가능해요! 그리고, 프로그래밍 기출 문제는 사람들이 각자 복원한 문제이기 때문에 **정리마다 조금씩 코드가 다를 수는 있어요.** 하지만, 문제의 유형은 동일하니 걱정하지 않으셔도 됩니다.

이 <정처기 실기 요약노트 - 프로그래밍 언어편>으로 공부하시면서 추가 궁금하신 사항이나 의견 있으실 분들을 위해 꿈꾸는라이언 공식 카페에 **‘정처기 실기 질의응답 게시판’**을 별도 만들었어요! 카페 가입하시고 질문 남겨주시면 본 요약노트 제작에 많은 도움을 준 현업 개발자 분께서 바로는 아니더라도 틈틈이 답변해주실거예요! 😊 🍀
 문의주신 모든 분들께 최대한 가능한 선에서 성심껏 빠르게 답변해드린다고 하니 참고해주세요!

★ 꿈꾸는라이언 네이버 카페 : <https://cafe.naver.com/dreamingryan>

그럼 모두들 정처기 합격을 시작으로 각자 계획하셨던 목표와 소중한 꿈을 이루시길 진심으로 응원하겠습니다.
 다들 포기하지 마시고, 끝까지 함께 달려봐요!
 합격 미리 축하드립니다!! ❤️ 🎉



이것만 이해해도 문제 풀이에 정말 큰 도움이 돼요

★ 집합(Set)

집합은 순서가 없고 중복을 허용하지 않는 자료형이에요.

아래와 같이 사용할 수 있어요.

집합 생성

```
fruit_basket = {"apple", "banana", "grape"}
```

집합에 요소 추가

```
fruit_basket.add("orange")
print(fruit_basket) → 출력: {"apple", "banana", "grape", "orange"}
```

집합에서 요소 제거

```
fruit_basket.remove("banana")
print(fruit_basket) → 출력: {"apple", "grape", "orange"}
```

집합 업데이트

```
more_fruits = ["kiwi", "pear", "apple"]
fruit_basket.update(more_fruits)
print(fruit_basket) → 출력: {"apple", "grape", "orange", "kiwi", "pear"}
```

★ 비트 연산자(Bitwise Operator)

2진수 형태로 데이터를 다룰 때 사용해요.

비트 연산자를 이해하기 쉬운 예시로는 "전등 스위치"를 들 수 있어요.

전등 스위치는 켜짐(1)과 꺼짐(0), 두 가지 상태만 있어요.

*여기서 켜짐은 True, 꺼짐은 False를 의미해요.

AND 연산(&)은 두 개의 스위치가 모두 켜져 있을 때만 전등이 켜지는 거예요.

OR 연산(|)은 두 개의 스위치 중 하나라도 켜져 있으면 전등이 켜지는 거예요.

XOR 연산(^)은 두 개의 스위치 중 하나만 켜져 있을 때 전등이 켜지는 거예요.

(서로의 결과가 달라야 전등이 켜진다는 의미예요)

아래와 같이 사용할 수 있어요. 내가 연산자를 보고 계산할 수 있으면 돼요.

AND 연산

```
result = 5 & 3 → 101 & 011 = 001
print(result) → 출력: 1
```

OR 연산

```
result = 5 | 3 → 101 | 011 = 111
print(result) → 출력: 7
```

XOR 연산

```
result = 5 ^ 3 → 101 ^ 011 = 110
print(result) → 출력: 6
```

비트 시프트 연산

왼쪽으로 한칸 이동은 맨 오른쪽에 0하나를 추가한다는 의미예요.

```
result = 5 << 1 → 101 << 1 = 1010
print(result) → 출력: 10
```

오른쪽으로 한칸 이동은 맨 오른쪽 값을 지운다는 의미예요.

여기서 2진수 010과 10은 같은 2를 뜻해요.

```
result = 5 >> 1 → 101 >> 1 = 010
print(result) → 출력: 2
```

★ split()

문자열을 지정된 구분자를 기준으로 분할하여 리스트로 반환합니다.

기본 구분자는 공백입니다.

공백을 기준으로 문자열 분할

```
sentence = "Hello world! How are you?"
words = sentence.split() → sentence.split(" ") 과 동일한 표현
print(words) → 출력: ['Hello', 'world!', 'How', 'are', 'you?']
```

★ 슬라이싱(Slicing)

리스트나 문자열에서 일부분을 잘라내는 것을 말해요.

Index의 시작점은 0부터 시작해요.

끝점의 index는 자를 때 포함하지 않아요.

[0:5]는 첫 번째 문자부터 시작해서 0, 1, 2, 3, 4까지의 문자를 뜻해요.

처음부터 자르고 싶으면 시작점을 생략할 수 있어요.

[:5]

끝까지 자르고 싶으면 끝점을 생략할 수 있어요.

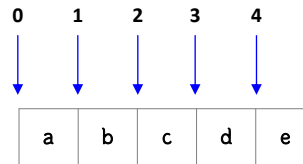
[8:]

문자열 슬라이싱

```
message = "Hello, World!"
print(message[0:5]) → 출력: "Hello"
print(message[:5]) → 출력: "Hello"
print(message[7: ]) → 출력: "World!"
print(message[7:12]) → 출력: "World"
print(message[::2]) → 출력: "Hlo ol!" → 홀수번째 문자열 추출!
(2 page 2번 문제 참고)
```

리스트 슬라이싱

```
numbers = ["a", "b", "c", "d", "e"]
print(numbers[1:4]) → 출력: ["b", "c", "d"]
print(numbers[:3]) → 출력: ["a", "b", "c"]
print(numbers[3: ]) → 출력: ["d", "e"]
print(numbers[::2]) → 출력: ["a", "c", "e"]
→ 홀수번째 문자열 추출!
(2 page 2번 문제 참고)
```



numbers[1:4] 슬라이싱 시,
인덱스 1에서 시작해서
인덱스 4 앞까지 추출
(인덱스 3까지만 포함)

★ 문자열 포매팅 (String Formatting)

문자열 내에 특정 값을 삽입하는 방법이에요.

% 연산자를 사용하여 문자열 포매팅을 할 수 있어요.

% 연산자

```
name = "Alice"
age = 25

print("My name is %s. I'm %d years old." % (name, age))
My name is Alice. I'm 25 years old.
```

★ map()

리스트나 튜플 등의 이터러블 객체의 각 요소에 함수를 적용하여 새로운 이터러블 객체를 반환합니다.

map()은 원본 데이터를 변경하지 않고 새로운 데이터를 생성합니다.

```
numbers = [1, 2, 3, 4, 5]
```

lambda 함수를 사용하여 각 요소에 10을 더하기

```
numbers = [1, 2, 3, 4, 5]
add_ten = map(lambda x: x + 10, numbers)
print(list(add_ten)) → 출력: [11, 12, 13, 14, 15]
```



▶ 정답 split

[기출 예제] 23년 3회 실기 (난이도: ★★☆☆)

다음 Python 프로그램과 그 실행결과를 분석하여 괄호에 들어갈 알맞은 예약어를 쓰시오.
첫 번째 라인의 '5 10'은 입력받은 값에 해당한다.

<실행결과>

x, y의 값을 공백으로 구분하여 입력: 5 10

x의 값: 5

y의 값: 10

정답은 split 이예요

split() 메서드는 문자열을 특정 구분자를 기준으로 나누어 리스트로 반환해요.
기본적으로는 공백을 구분자로 사용해요.

split() 메서드는 사용자 입력을 받아 분리할 때 자주 사용돼요.

```
1 x, y = input("x, y의 값을 공백으로 구분하여 입력: ").( 빈 칸 )(" ")
2 print("x의 값: ", x)
3 print("y의 값: ", y)
```

★ key point

사용자가 5 10 입력한 것을 실행결과에서 알려주고 있어요.

"5 10"이라는 값을 x, y로 공백 기준으로 값을 할당하기 위해서는 공백 기준으로 문자열을 나눠야해요.

문자열을 특정 구분자로 나누는 키워드는 split이예요.

그래서 정답이 split이 돼요.

*구분자를 넘겨주지 않는 split() 은 split(" ")과 동일해요. 즉, 공백 문자열이 기본값이예요.

[기출 예제] 23년 2회 실기 (난이도: ★★☆☆)

다음 파이썬 코드에서 알맞는 출력값을 작성하시오.

```
1 a = "engineer information processing"
2 b = a[:3]
3 c = a[4:6]
4 d = a[28:]
5 e = b+c+d
6 print(e)
```

★ 슬라이싱 (자주 쓰이니 외우면 좋아요)

sequence[start:end:step]의 형태로 사용돼요.

start는 슬라이스가 시작되는 인덱스이고,

end는 슬라이스가 끝나는 인덱스이며,

(해당 인덱스는 문자열에서 제외)

step은 슬라이스가 진행되는 간격이예요.

start와 end는 생략할 수 있고, 생략하면 각각
시퀀스의 시작과 끝이 기본값으로 사용돼요.

step도 생략할 수 있고,
생략하면 기본값 1이 사용돼요.

변수 a에 문자열 "engineer information processing"를 할당해요.

a 문자열의 처음부터 3번째 문자까지 가져와 b에 저장합니다.

여기서 a[:3]은 a의 인덱스 0부터 인덱스 2까지를 의미해요.

따라서 b에는 "eng"가 저장됩니다.

a 문자열에서 5번째부터 6번째 문자까지 가져와 c에 저장합니다.

(인덱스 4부터 인덱스 5까지)

여기서 a[4:6]은 "ne"를 의미해요.

변수 d에 a의 29번째 문자부터 끝까지의 슬라이스를 할당해요.

슬라이싱에서 두 번째 인덱스를 생략하면 문자열의 끝까지 슬라이스 해요.

a의 29번째 문자부터 끝까지는 "ing" 부분 문자열 이예요.

따라서 d에는 "ing"이 할당돼요.

변수 e에 b, c, d를 연결한 문자열을 할당해요.

b는 "eng", c는 "ne", d는 "ing"이므로,

e에는 "engneing"이 할당돼요.

[6행] 변수 e의 값인 "engneing"을 출력해요.

▶ 정답 engneing

★ 집합 (자주 쓰이니 외우면 좋아요)

중복되지 않는 요소들의 모음이예요.

같은 값을 여러 번 추가해도 집합에는 한 번만
저장돼요.

순서가 없어요.

집합의 요소들은 특정한 순서를 가지지 않아요.

[기출 예제] 23년 1회 실기 (난이도: ★★☆☆)

다음 파이썬 코드에서 알맞는 출력값을 작성하시오.

```
1 a = {'한국', '중국', '일본'}
2 a.add('베트남')
3 a.add('중국')
4 a.remove('일본')
5 a.update({'홍콩', '한국', '태국'})
6 print(a)
```

최종적인 a 집합을 출력해요.

정답은 {'한국', '중국', '베트남', '홍콩', '태국'}이예요

a 집합에 다른 집합의 요소들을 추가해요.

update() 메서드는 인자로 주어진 집합의 모든 요소를 a에 추가해요.
'한국'은 이미 a에 존재하므로 중복 추가되지 않아요.

a라는 이름의 집합을 생성하고 초기값으로 '한국', '중국', '일본'을 갖게 해요.

a 집합에 '베트남'을 추가해요.

집합에 새로운 요소를 추가할 때는 add() 메서드를 사용해요.
이 연산 후에 a는 {'한국', '중국', '일본', '베트남'}이 돼요.

a 집합에 '중국'을 추가하려고 해요.

하지만 '중국'은 이미 a에 존재하는 요소예요.

집합은 중복된 요소를 허용하지 않기 때문에

이 연산은 a를 변화시키지 않아요.

이 연산 후에도 a는 여전히 {'한국', '중국', '일본', '베트남'}이예요.

a 집합에서 '일본'을 제거해요.

집합에서 특정 요소를 제거할 때는 remove() 메서드를 사용해요.
이 연산 후에 a는 {'한국', '중국', '베트남'}이 돼요.

▶ 정답 {'한국', '중국', '베트남', '홍콩', '태국'}



[기출 예제] 22년 3회 실기 (난이도: ★★★)

▶ 정답 [101,102,103,104,105]

다음 파이썬 코드에 대한 출력값을 작성하시오.

```
1 TestList = [1,2,3,4,5]
2 TestList = list(map(lambda num : num + 100, TestList))
3
4 print(TestList)
```

TestList라는 이름의 리스트를 생성하고 초기값으로 [1,2,3,4,5]를 할당해요.

결과적으로, TestList의 각 요소에 100을 더한 새로운 리스트를 생성해요.

map() 함수는 첫 번째 인자로 함수를, 두 번째 인자로 이터러블(리스트 등)을 받아요.
map() 함수는 이터러블의 각 요소에 함수를 적용한 결과를 반환해요.

여기 코드에서는 람다 표현식 `lambda num : num + 100`이 map() 함수의 첫 번째 인자로 사용되었어요.
이 람다 표현식은 인자 `num`을 받아 `num + 100`을 반환하는 함수예요.
map()의 두 번째 인자로는 TestList가 전달되었어요.

따라서, map()은 TestList의 각 요소에 람다 함수를 적용해요.

```
1 + 100 = 101
2 + 100 = 102
3 + 100 = 103
4 + 100 = 104
5 + 100 = 105
```

map()의 결과는 맵 객체(map object)로 리스트로 변환하기 위해 list() 함수를 사용했어요.
최종적으로 TestList에는 [101, 102, 103, 104, 105]가 할당돼요.

정답은 [101, 102, 103, 104, 105] 이 출력돼요.

[기출 예제] 22년 2회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

▶ 정답 REMEMBER AND STR

```
1 a = "REMEMBER NOVEMBER"
2 b = a[:3] + a[12:16];
3 c = "R AND %s" % "STR";
4 print(b+c);
```

a라는 변수에 문자열 "REMEMBER NOVEMBER"를 할당해요.

문자열 슬라이싱을 사용하여 새로운 문자열을 만들어 b에 할당하는 부분이에요.

a[:3]은 a의 처음부터 3번째 문자까지의 슬라이스로, "REM"이에요.
(인덱스 0부터 인덱스 2까지)

a[12:16]은 a의 13번째 문자부터 16번째 문자까지의 슬라이스로, "EMBE"예요.
(인덱스 12부터 인덱스 15까지)

두 슬라이스를 + 연산자로 연결하면 "REMEMBE"가 돼요.
따라서 b에는 "REMEMBE"가 할당돼요.

문자열 포매팅을 사용하여 새로운 문자열을 만들어 c에 할당하는 부분이에요.
"R AND %s"는 포맷 문자열로, %s 부분에 다른 문자열이 들어갈 자리예요.
% 연산자 뒤에 오는 "STR"이 %s 자리에 들어가요.
따라서 c에는 "R AND STR"이 할당돼요.

b와 c를 연결한 문자열을 출력하는 부분이에요.
b는 "REMEMBE", c는 "R AND STR"이에요.
두 문자열을 + 연산자로 연결하면 "REMEMBER AND STR"이 돼요.

정답은 REMEMBER AND STR 이예요.

[기출 예제] 22년 1회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

▶ 정답 a= 20 b= 2

```
1 def exam(num1, num2=2):
2     print('a=', num1, 'b=', num2)
3     exam(20)
```

이 코드의 특징은 함수의 매개변수에 기본값을 지정하는 방법을 보여줘요.

함수 정의에서 num2=2와 같이 매개변수에 =로 값을 할당하면, 그 값이 해당 매개변수의 기본값이 돼요.
함수를 호출할 때 해당 매개변수에 값을 전달하지 않으면, 기본값이 사용돼요.

여기서는, exam(20)은 num1에 20을 전달하고 num2에는 아무것도 전달하지 않았어요
따라서, num1은 20, num2는 기본값 2가 사용돼요.

그러므로 이 코드의 출력 결과는 a= 20 b= 2 예요.



[가출예제] 21년 3회 실기 (난이도: ★★★)

▶ 정답 False

다음 파이썬 코드에 대한 출력값을 작성하시오.

```
1 a, b = 100, 200
2 print(a==b)
```

a에 100을, b에 200을 할당하는 거예요.
이렇게 하면 a와 b에 각각 다른 값이 저장돼요.
a와 b가 같은지 확인하는 거예요.

==은 두 값이 같은지 비교하는 연산자예요.
만약 a와 b가 같다면 True를,
다르다면 False를 결과로 돌려줘요.

a는 100, b는 200이니가
당연히 둘은 같지 않아요.
그래서 a==b의 결과는 False가 되는 거죠.

[가출예제] 21년 2회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

```
1 a = 100
2 result = 0
3 for i in range(1,3):
4     result = a >> i
5     result = result + 1
6 print(result)
```

a에 100을 할당하고,
result에 0을 할당했어요.
이렇게 하면 a는 100, result는 0으로 초기화되는 거예요.
for 루프를 사용해서 i를 1부터 2까지 반복해요.
range(시작 숫자, 끝 숫자) 형태를 사용하는데, 이때 끝 숫자는 포함되지 않습니다.
슬라이스와 비슷하게 인덱스 시작 숫자 이상 인덱스 끝 숫자 - 1 이하 까지를 뜻해요.
루프 안에서는 아래 로직을 수행해요.

▶ 정답 26

★ key point

비트 연산자가 나오면, 2진수로 변환 후
왼쪽 혹은 오른쪽으로 이동 후 다시
10진수로 변환한다고 생각하면 돼요.

result = a >> i는 a를 오른쪽으로 i번 시프트한 결과를 result에 할당해요.
>>는 비트 오른쪽 시프트 연산자예요.
이 연산자는 a의 비트를 오른쪽으로 i번 이동시켜요.

조금 어려울 수 있어요.
첫 번째 반복(i=1)에서 a >> 1은 a의 비트를 오른쪽으로 1번 이동시키는 거예요.
2진수로 표현하면 1100100(기존 100)에서 0110010(= 110010)(비트 이동)이 되죠.
이것은 10진수로 50이에요.
여기에 1을 더하면 51이 돼요.

두 번째 반복(i=2)에서는 a >> 2로 a의 비트를 오른쪽으로 2번 이동시켜요.
2진수로 1100100(100)에서 0011001(= 11001)이 되는데, 이것은 10진수로 25예요.
여기에 1을 더하면 26이 돼요.

루프가 끝나면 print(result)로 최종 result의 값을 출력해요.
정답은 26이에요.

[가출예제] 21년 1회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

▶ 정답 skiddp

```
1 class good : good이라는 클래스를 정의했어요.
2     li = ["seoul", "kyeonggi", "incheon", "daejeon", "daegu", "pusan"]
3
4 g = good()
5 str01 = ''
6 for i in g.li:
7     str01 = str01 + i[0]
8
9 print(str01)
```

good 클래스의 인스턴스를 생성해서 g라는 변수에 할당하는 거예요.
이렇게 하면 g를 통해 good 클래스의 속성과 메서드에 접근할 수 있게 돼요.
빈 문자열을 str01이라는 변수에 할당하는 거예요.
이 변수는 앞으로 도시 이름의 첫 글자를 모으는 데 사용될 거예요.
for 루프를 사용해서 g.li의 각 요소를 i라는 변수에 하나씩 할당하면서 반복해요.
여기서는 순서대로, seoul, kyeonggi ... 이 값들이 i에 할당돼요.
str01 = str01 + i[0]은 현재 str01의 값에 i의 첫 번째 문자(i[0])를 더하는 거예요.
문자열에서 배열처럼 index로 문자 하나씩 가져올 수 있어요.

첫 번째 반복에서는 "seoul"의 첫 글자 "s"가 str01에 더해져요.
두 번째 반복에서는 "kyeonggi"의 첫 글자 "k"가 str01에 더해져요.
이런 식으로 계속 반복되면서 str01에는 각 도시 이름의 첫 글자가 모이게 돼요.

루프가 끝나면 print(str01)로 최종 str01의 값을 출력해요.
정답은 skiddp 이예요.



▶ 정답
[1, 2, 3]
7
123
45
6789

[기출예제] 20년 4회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

```

1 lol = [[1,2,3],[4,5],[6,7,8,9]]
2 print(lol[0])
3 print(lol[2][1])
4 for sub in lol:
5     for item in sub:
6         print(item, end = '')
7     print()

```

리스트들을 요소로 가지고 있는 2차원 리스트예요.
lol의 첫 번째 요소를 출력하는 거예요. lol[0]은 [1,2,3]이므로, 이 코드는 [1,2,3]을 출력해요.
세 번째 요소의 두 번째 요소를 출력하는 거예요.
lol[2]는 [6,7,8,9]이고, lol[2][1]은 7이므로, 이 코드는 7을 출력해요.
중첩 for 루프를 사용하고 있어요.
바깥쪽 루프에서는 lol의 각 요소(리스트)를 sub라는 변수에 하나씩 할당하면서 반복해요.
안쪽 루프에서는 각 sub의 요소를 item이라는 변수에 하나씩 할당하면서 반복해요.
현재 item의 값을 출력하는데,
end = ""는 출력 후 줄바꿈을 하지 않고 바로 다음 출력을 이어서 하라는 의미예요.
이렇게 하면 각 리스트의 요소들이 한 줄에 모두 출력돼요.
첫번째 배열인 [1,2,3]는 123
두번째 배열인 [4,5]는 45
세번째 배열인 [6,7,8,9]는 6789
이렇게 출력돼요.

안쪽 루프가 끝나면 print()를 사용해서 줄바꿈을 해요.
이렇게 하면 각 리스트의 요소들이 한 줄씩 출력되는 거죠.
item 기준으로는 한줄로 출력하고,
Sub 기준으로는 한줄씩 내려서 출력돼요.

item에 이라는 파이썬 키워드가 있는 건 아니예요.
lol, sub과 동일한 변수명이예요!

[기출예제] 20년 2회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

▶ 정답 ['중국', '한국', '베트남', '홍콩', '태국']

```

1 a={'일본','중국','한국'}
2 a.add('베트남')
3 a.add('중국')
4 a.remove('일본')
5 a.update(['홍콩','한국','태국'])
6 print(a)

```

집합(set) 자료형을 다루고 있어요. 집합은 순서가 없고, 중복된 요소를 허용하지 않는 자료형이에요.
자주 나오고 있어요. 중복되지 않는다는 사실 알고 있으면 좋아요
중괄호 {}를 사용하여 생성할 수 있고, 빈 집합을 생성할 때는 set()을 사용해요.
a 집합에 '베트남'을 추가하는 거예요. 이 연산 후에 a는 {'일본', '중국', '한국', '베트남'}이 돼요.
a 집합에 '중국'을 추가하려고 하는 거예요. 하지만 '중국'은 이미 a에 존재하는 요소예요.
집합은 중복된 요소를 허용하지 않기 때문에, 이 연산은 a를 변화시키지 않아요.
a 집합에서 '일본'을 제거하는 거예요. 이 연산 후에 a는 {'중국', '한국', '베트남'}이 돼요.
a 집합에 '홍콩', '한국', '태국'을 추가하는 거예요.
update() 메서드는 다른 집합이나 리스트 등의 이터러블 객체를 인자로 받아서,
그 안의 모든 요소를 집합에 추가해요. 이 때, '한국'은 이미 a에 존재하므로 중복 추가되지 않아요.
이 연산 후에 a는 {'중국', '한국', '베트남', '홍콩', '태국'}이 돼요.
최종적인 a 집합을 출력하는 거예요.
{'중국', '한국', '베트남', '홍콩', '태국'}이 출력돼요.

▶ 정답
Seynaau

[기출예제] 24년 1회 실기 (난이도: ★★★)

다음 파이썬 코드에 대한 출력값을 작성하시오.

```

1 a = ["Seoul", "Kyeonggi", "Incheon", "Daejun", "Daegu", "Pusan"]
2 str = "S"
3
4 for i in a:
5     str = str + i[1]
6
7 print(str)

```

문자열을 요소로 가지고 있는 리스트예요.
str 변수를 생성 후 S를 할당했어요.
1번 라인에서 선언한 배열의 문자열들을 한번씩 i에 할당해요.
변수 str에 str 과 문자열 i의 index 1 위치의 문자를 더해요.
(index는 0부터 시작해요. index 1은 2번째 문자를 뜻해요.)

n[0]	n[1]	n[2]	n[3]	n[4]
S	e	o	u	l

a = ["Seoul", "Kyeonggi", "Incheon", "Daejun", "Daegu", "Pusan"]
즉, S + e + y + n + a + a + u = Seynaau 이 출력돼요.



▶ 정답 ab3ca3

[기출 예제] 24년 2회 실기 (난이도: ★★★)

다음은 Python에 대한 문제이다. 아래 코드를 읽고 알맞는 출력 값을 작성하시오.

```

1 def fnCalculation(x,y):
2     result = 0; → result라는 변수를 만들고 0으로 초기화해요. 이 변수는 나중에 결과를 저장할 거예요.
3     for i in range(len(x)): → x의 길이만큼 반복하는 루프를 시작해요. i는 0부터 x의 길이-1까지 변해요.
4         temp = x[i:i+len(y)] → x의 일부분을 잘라서 temp에 저장해요. i부터 시작해서 y의 길이만큼의 부분 문자열을 만들어요. ab, bd, dc...ca까지 y와 같은지 체크해요.
5         if temp == y: → 만약 temp가 y와 같다면, result를 1 증가시켜요. 이렇게 해서 x 안에 y가 몇 번 나타나는지 세고 있어요.
6             result += 1;
7     return result → 최종적으로 계산된 result 값을 반환해요.
8
9 a = "abdcabcbca" → 세 개의 문자열 변수 a, p1, p2를 정의하고 값을 할당해요.
10 p1 = "ab";
11 p2 = "ca";
12
13 out = f"ab{fnCalculation(a,p1)}ca{fnCalculation(a,p2)}" → fnCalculation 함수를 두 번 호출하고, 그 결과를 문자열 안에 넣어 새로운 문자열을 만들어요.
14 print(out) → f-string을 사용해서 결과를 문자열에 직접 삽입하고 있어요.

```

※ 파이썬의 포매팅 문자열로, 문자열 안에 직접 변수나 표현식을 넣을 수 있어요.

즉, abdcabcbca에 ab가 몇 번 나타나는지 체크한 뒤 3을 반환해요.

abdcabcbca에 ca가 몇 번 나타나는지 체크한 뒤 3을 반환해요.

그래서 ab3ca3이 출력돼요.

▶ 정답 3

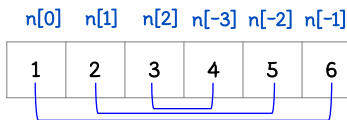
[기출 예제] 24년 3회 실기 (난이도: ★★★)

다음은 파이썬에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1 def func(lst):
2     for i in range(len(lst) // 2): → 리스트 길이의 절반인 3(len(lst) // 2)만큼 반복해요. (// 은 몫만 계산하는 연산자)
3         lst[i], lst[-i - 1] = lst[-i - 1], lst[i] → 리스트의 앞쪽 값(lst[i])과 뒤쪽 값(lst[-i-1])을 서로 바꿔요.
4         i = 0이면 첫 번째 값(lst[0])과 마지막 값(lst[-1])을 바꿉니다.
5 lst = [1, 2, 3, 4, 5, 6]
6 func(lst) → 리스트 lst를 입력받아, 리스트를 뒤집어요. (결과: [6, 5, 4, 3, 2, 1])
7 print(sum(lst[::-2]) - sum(lst[1::2])) → 리스트에서 짝수 인덱스(lst[::-2])와 홀수 인덱스(lst[1::2]) 값을 각각 더하고, 그 차이를 계산해요.

```



i[0]과 i[-1]의 값을 바꿔요.
i[1]과 i[-2]의 값을 바꿔요.
i[2]와 i[-3]의 값을 바꿔요.

lst[::-2]: 리스트의 짝수 인덱스 값 [6, 4, 2] (0, 2, 4번째 값) 0부터 2씩 증가하는 인덱스

lst[1::2]: 리스트의 홀수 인덱스 값 [5, 3, 1] (1, 3, 5번째 값) 1부터 2씩 증가하는 인덱스

sum(lst[::-2]): 6 + 4 + 2 = 12

sum(lst[1::2]): 5 + 3 + 1 = 9

차이: 12 - 9 = 3 이 정답이에요.

★ key point

배열의 뒤집기와 리스트 슬라이싱에 대해서만 알고 있다면 어렵지 않게 풀 수 있어요.

★ key point

type 함수의 정확한 이해와 조건문의 논리적인 흐름을 아는지 확인하는 문제예요.

▶ 정답 45

[기출 예제] 24년 3회 실기 (난이도: ★★★)

다음은 파이썬에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1 def func(value):
2     if type(value) == type(100): → 먼저 value의 type과 100의 type 즉, 정수(int) 타입인지 확인해요. 맞다면, 100을 반환해요.
3         return 100
4     elif type(value) == type(""): → 정수 타입이 아니면 문자열(str) 타입인지 확인해요. 맞다면 그 문자열 길이를 반환해요.
5         return len(value)
6     else:
7         return 20 → 문자열 타입이 아니면 20을 반환해요.
8
9
10 a = '100.0'
11 b = 100.0
12 c = (100, 200)
13
14 print(func(a) + func(b) + func(c))

```

a는 문자열 "100.0" 이예요. 따라서, type(value) == type("")의 조건이 참이예요.
문자열 길이 5를 반환해요.

b는 실수형 숫자 100.0 이예요. 즉, 정수 타입과 문자열 타입 모두 아니기 때문에 20을 반환해요.

c는 튜플 (100, 200) 이예요. b와 동일하게 정수 타입과 문자열 타입 모두 아니기 때문에 20을 반환해요.

- func(a) → 5
- func(b) → 20
- func(c) → 20

으로 5 + 20 + 20 = 45가 출력돼요.



[기출 예제] 25년 1회 실기 (난이도: ★★☆☆)

다음은 파이썬에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1 class Node:
2     def __init__(self, value):
3         self.value = value
4         self.children = []
5
6 def tree(li):
7     nodes = [Node(i) for i in li]
8     for i in range(1, len(li)):
9         nodes[(i - 1) // 2].children.append(nodes[i])
10    return nodes[0]
11
12 def calc(node, level=0):
13     if node is None:
14         return 0
15     return (node.value if level % 2 == 1 else 0) + sum(calc(n, level + 1) for n in node.children)
16
17 li = [3, 5, 8, 12, 15, 18, 21]
18
19 root = tree(li)
20
21 print(calc(root))

```

노드를 만들 때 실행되는 초기화 함수예요.
value라는 값을 받아서, **노드 안에 저장**하고 **자식 노드들을 담은 빈 리스트**도 함께 가지고 있어요.

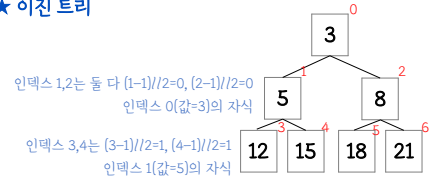
리스트 li의 각 값을 이용해 Node 객체들을 순서대로 만들어요. ex) li = [3, 5, 8, 12, 15, 18, 21]라면, [Node(3), Node(5), ..., Node(21)]

인덱스 i의 노드를, 인덱스 (i-1)//2의 노드의 자식으로 연결해요. 이 규칙 덕분에 리스트가 완전 이진 트리로 변경돼요.
(왼쪽부터 빈 자리 없이 채워지는 형태)
※ //는 나눗셈한 몫을 반환해요 ex) 7 // 2 = 3

재귀 종료
현재 레벨이 홀수면 node.value를, 짝수면 0을 더해요.
이 노드 자식들에 대해, (level+1) 하여 calc를 호출한 결과를 전부 더해요.

li로 트리를 만들어요.

★ 이진 트리



★ key point

파이썬 문제임을 감안할 때, **별 2.5개** 정도로 꽤 어려운 문제입니다.

- tree() 함수로 **완전 이진 트리**를 만든다는 점
- calc() 함수에서 **홀수 레벨의 값만 더한다**는 로직
- 그리고 **클래스 구조 · 리스트 인덱스 연산 · 재귀 호출**의 동작 원리

이 세 가지를 이해하고 나면, 작은 리스트에서는 일일이 값을 대입해 확인하지 않아도 논리만으로 쉽게 풀 수 있습니다.

물론 아직 익숙하지 않다면, **실제 시험에서는 전체 리스트를 전부 써 보면서 동작 방식을 확인**해 보는 것도 좋은 방법입니다.

(리스트가 작을 때는 오래 걸리지 않기 때문입니다. 만약 대입할 값이 많다면 다른 문제부터 푸는 것을 추천해 드립니다.)



[문제 유형] equals

★ == 연산자

==는 기본 데이터 타입(예: int, float, char 등)의 값을 비교할 때 자주 사용돼요. 두 값이 같은지 확인하려면 ==를 사용하면 되죠.

참조 데이터 타입(예: 객체)에 대해서 ==를 사용하면, 그것들이 같은 메모리 주소를 가리키고 있는지 확인합니다. 즉, 두 변수가 같은 객체를 참조하고 있는지를 봐요.

```
String text1 = new String("hello");
String text2 = new String("hello");
System.out.println(text1 == text2); // false, 왜냐하면 메모리 주소가 다르니까요.
```

여기서 text1과 text2는 같은 내용의 문자열을 가지고 있지만, 각기 다른 객체이므로 == 연산자는 false를 반환합니다.

★ equals 메서드

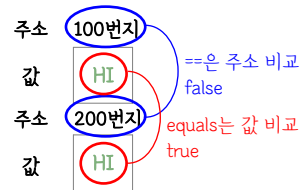
equals 메서드는 객체의 내용이 같은지를 확인하는 데 사용돼요.

예를 들어, String 클래스에서 equals 메서드는 문자열의 내용이 같은지를 비교합니다.

```
String text1 = new String("hello");
String text2 = new String("hello");
System.out.println(text1.equals(text2)); // true, 내용이 같기 때문이에요.
```

여기서 text1과 text2의 내용은 같기 때문에 equals 메서드는 true를 반환해요.

간단히 말하면, ==는 두 객체가 실제로 같은 객체인지, 즉 메모리 상에서 같은 위치를 참조하는지를 확인하는 반면, equals 메서드는 두 객체가 '논리적으로' 동등한지를 비교하는 데 사용돼요.



▶ 정답
true
false
true
true

[기출 예제] 23년 2회 실기 (난이도: ★☆☆)

다음은 자바에 대한 문제이다. 알맞은 출력 값을 작성 하시오.

```
1 public class Main{
2     public static void main(String[] args) {
3
4         String str1 = 'Programming';
5         String str2 = 'Programming';
6         String str3 = new String('Programming');
7
8         println(str1==str2)
9         println(str1==str3)
10        println(str1.equals(str3))
11        print(str2.equals(str3))
12    }
13 }
14 }
```

[4행] [5행] 이 둘은 하나의 객체가 생성되고 자바에서 같은 객체를 할당해줍니다.
이렇게 명시적으로 new 를 통해 객체를 생성하는 경우,
같은 값("Programming") 이더라도 별도의 공간에 객체를 생성해 str3에 할당합니다.
→ str1과 str2의 객체 주소는 동일해요
→ str1과 str3의 객체 주소는 달라요. new 를 통해 새롭게 생성했기 때문이에요.
→ str1과 str3의 값을 비교해요. 둘의 문자열은 동일해요
→ str2과 str3의 값을 비교해요. 둘의 문자열은 동일해요



[문제 유형] 단순 for문

★ for문

for 문은 일정한 규칙에 따라 반복해서 명령을 실행할 때 사용해요.

for 문은 보통 세 부분으로 구성되어 있어요

초기화: 반복을 시작하기 전에 처음에 설정해야 하는 부분이에요. 여기서 반복할 때 사용할 변수를 설정합니다.

조건: 반복을 계속할지 말지를 결정하는 조건이에요. 이 조건이 참이면 반복을 계속하고, 거짓이면 반복을 멈춰요.

증감: 반복할 때마다 실행되는 부분으로, 보통 반복하는 변수를 증가시키거나 감소시키는데 사용돼요.

```
for(int i = 0; i < 5; i++) {
    System.out.println("반복 중: " + i);
}
```

```
// 반복 중: 0
// 반복 중: 1
// 반복 중: 2
// 반복 중: 3
// 반복 중: 4
```

int i = 0; - 반복을 시작하기 전에 변수 i를 0으로 초기화해요.

i < 5; - i가 5보다 작은 동안에는 반복을 계속해요. i가 5에 도달하면 반복을 멈춰요.

i++ - 매 반복마다 i의 값을 1씩 증가시켜요.

결과적으로 이 for 문은 "반복 중: 0"부터 "반복 중: 4"까지 총 5번의 메시지를 출력하게 됩니다.

[기출예제] 20년1회 실기 (난이도: ★★★)

▶ 정답 0123

다음은 자바 소스코드이다. 출력 결과를 쓰시오.

실제 실행코드	1	public class Main {	
	2	public static void main(String[] args) {	
	3	int i;	
	4	int a[] = {0,1,2,3};	4 크기인 배열 a를 생성했어요.
	5	for(i=0; i<4; i++) {	i가 0,1,2,3 까지 반복해요. (4보다 작을 때까지)
	6	System.out.print(a[i] + " ");	a 배열의 0번째, 1번째, 2번째, 3번째 값을 출력해요.
	7	}	차례대로 0123이 출력돼요
	8	}	
	9	}	

[기출예제] 22년3회 실기 (난이도: ★★★)

▶ 정답 0123

아래 자바 코드에서 출력되는 값을 작성하시오.

실제 실행코드	1	class Test {	
	2	③ static int[] mkarr() {	
	3	int[] tmpArr = new int[4];	크기가 4인 정수형 배열 tmpArr를 선언하고 생성해요.
	4		
	5	for (int i = 0; i < tmpArr.length; i++) {	0부터 크기가 4인 정수형 배열의 길이보다 작을 때(4미만)까지 반복
	6	tmpArr[i] = i;	0번째 배열에는 0
	7	}	1번째 배열에는 1
	8		2번째 배열에는 2
	9	return tmpArr;	3번째 배열에는 3을 넣고 있어요.
	10	}	0 1 2 3 배열 반환
	11		
	12	public static void main(String[] args) {	
	13	② int[] arr;	배열 객체를 선언만 하고, 할당은 [14행]에서 해요
	14	arr = mkarr();	위 {0, 1, 2, 3} 배열을 할당
15	for (int i = 0; i < arr.length; i++) {		
16	System.out.print(arr[i]);	[15행]을 보면, 0부터 배열을 출력하는데	
17	}	arr의 길이만큼 (전체)를 출력하도록 반복하고 있어요.	
18	}		
19	}	정답은 0123 이에요	



▶ 정답 61

[기출예제] 22년 2회 실기 (난이도: ★☆☆)

아래 자바 코드에서 출력되는 값을 작성하십시오.

실제 실행코드

```

1 class Main {
2     public static void main(String[] args) {
3         ① Cond obj = new Cond(3);
4         ④ obj.a=5;
5         ⑤ int b = obj.func();
6         ⑦ System.out.print(obj.a + b);
7     }
8 }
9
10 class Cond {
11     int a;
12     ② public Cond(int a) {
13         ③ this.a = a;
14     }
15
16     ⑥ public int func() {
17         int b = 1;
18         for (int i = 1; i < a; i++) {
19             b += a * i;
20         }
21         return a + b;
22     }
23 }

```

Cond 객체 생성 및 생성자에 3을 넘겨주고 있어요. Cond 안의 멤버변수 a에 3이 할당돼요. 하지만, [4행]에서 바로 a의 값을 5로 덮어쓰고 있어요. [16행]이 실행돼요. 56이 반환돼요. (로직은 아래에서 설명할게요) obj.a + b 는 5 + 56이므로, 정답은 61이에요.

매개변수 a를 받아서 Cond 멤버변수 a에 할당해요. 여기서는 [3행]에 의해 3이 할당되었다가 [4행]에 의해 5가 되었어요.

b의 시작값 1이에요. 1부터 a 미만까지 +1 증가하면서 반복해요. 여기서 a는 5여서, 1~4까지 반복해요. a * i 는 (5 * 1) 한 값을 b에 더해줘요. 순서대로 를 반복해서 b에 더해줘요. 즉 b의 시작값 1 + (5 * 1) + (5 * 2) + (5 * 3) + (5 * 4) 이 b에 할당돼요. b는 51이에요. func() 메소드의 반환은 a + b이니, 5 + 51을 반환하면 56가 반환돼요.

Cond 클래스

▶ 정답 996

[기출예제] 22년 3회 실기 (난이도: ★☆☆)

아래 자바 코드에서 출력되는 값을 작성하십시오.

실제 실행코드

```

1 class Exam{
2     public static void main(String[] args){
3         int a = 0;
4
5         for(int i = 1; i < 999; i++) {
6             if(i % 3 == 0 && i % 2 == 0)
7                 a = i;
8         }
9
10        System.out.print(a);
11    }
12 }

```

정수형 변수 a를 선언하고 0을 할당합니다. 1부터 998까지 (999미만) +1씩 증가하면서 반복해요. %는 나머지를 뜻해요. 즉, i를 3으로 나누고 남은 나머지가 0이면서, i를 2로 나누고 남은 나머지가 0인 경우에 a에 i를 할당해라 라는 의미예요. 이 말만 보면 3의 배수이면서 2의 배수인 6의 배수를 찾으라는 말인데요! 6의 배수마다 a에 계속해서 덮어쓰는 코드이니, 사실상 출력되는 값은 998에서 가장 가까운 6의 배수가 되겠네요! 998은 3의 배수가 아니고, 997은 2의 배수가 아닙니다. 996은 3의 배수면서 2의 배수입니다! 그러므로, 정답은 996이 됩니다!

[기출예제] 24년 3회 실기 (난이도: ★☆☆)

아래 자바 코드에서 출력되는 값을 작성 하시오.

▶ 정답 00AAA

실제 실행코드

```

1 public class Main {
2     static String[] s = new String[3];
3
4     static void func(String[] s, int size) {
5         for (int i = 1; i < size; i++) {
6             if (s[i - 1].equals(s[i])) {
7                 System.out.print("O");
8             } else {
9                 System.out.print("N");
10            }
11        }
12        for (String m : s) {
13            System.out.print(m);
14        }
15    }
16
17    public static void main(String[] args) {
18        s[0] = "A";
19        s[1] = "A";
20        s[2] = new String("A");
21        func(s, 3);
22    }
23 }
24
25

```

길이가 3인 문자열(String) 배열 s를 선언해요. 배열(s)과 정수(size)를 입력으로 받아 실행돼요. 1부터 size보다 작을 때까지 반복해요. (여기서는 size 3 [23행]이 들어가요.) i의 이전 값과 현재 값을 비교해요. (배열 주소가 아닌 값의 비교) 값이 같다면 O를 출력 (개행 없음) 값이 다르다면 N를 출력 (개행 없음) 배열(s)의 모든 값을 출력해요. (개행 없이 쭉 나열해서 출력해요)

배열(s)의 0 번째 위치에 문자열 A를 넣어요. 배열(s)의 1 번째 위치에 문자열 A를 넣어요. 배열(s)의 2 번째 위치에 문자열 A를 넣어요. func 함수를 호출합니다. 위에서 설명했듯이 1 인덱스 부터 시작해서 2(3보다 작은)까지 수행해요. s[0]와 s[1]을 비교, s[1]과 s[2]를 비교해요. 전부 A 문자열이 들어가 있기에 OO를 출력해요. 이후, s 배열 전체를 출력해요. AAA가 출력돼요. 즉, 정답은 00AAA가 돼요.

비교

비교



▶ 정답 24513

[기출예제] 22년 3회 실기 (난이도: ★★★)

아래 자바 코드에서 출력되는 값을 작성하시오.

```

1 public class Test {
2     public static void main(String[] args) {
3         int result[] = new int[5];
4
5         int arr[] = { 77, 32, 10, 99, 50 };
6
7         for (int i = 0; i < 5; i++) {
8             result[i] = 1;
9
10            for (int j = 0; j < 5; j++) {
11                if(arr[i] < arr[j]) {
12                    result[i]++;
13                }
14            }
15        }
16
17        for (int k = 0; k < 5; k++) {
18            System.out.print(result[k]);
19        }
20    }
21 }
```

크기가 5인 정수형 배열 result를 선언하고 생성해요.
배열은 같은 타입의 데이터를 연속적으로 저장하는 자료구조입니다.
배열의 크기는 생성 시에 결정되며, 이 경우 result 배열은 5개의 정수를 저장할 수 있어요.

크기가 5인 정수형 배열 arr을 선언하고, 초기값으로 77, 32, 10, 99, 50을 저장합니다.

배열 선언과 동시에 할당해요. 이걸 초기화 라고 불러요.

i가 0부터 4까지 (5보다 미만) +1씩 증가하면서 반복해요.

이중 반복문으로
i가 0일 때, j가 0 ~ 4까지 반복하고,
i가 1일 때, j가 0 ~ 4까지 반복하고,
...
i가 4일 때까지 반복하는 거예요.

그 안에 조건문 if는 다음과 같아요.
arr의 i번째의 값이 arr의 j번째의 값보다 작으면, result[i]의 값을 +1 증가시켜요.
(result[i]의 값은 [8행]에서 넣어준 1이 시작 값이에요.)

arr[0] 인 77보다 큰 값은 99 하나예요. 즉, 1을 더해줘요. $1 + 1 = 2$
arr[1] 인 32보다 큰 값은 77, 99, 50 세개 예요. 즉, 3을 더해줘요. $1 + 3 = 4$
arr[2] 인 10보다 큰 값은 77, 32, 99, 50 네개 예요. 즉, 4를 더해줘요. $1 + 4 = 5$
arr[3] 인 99보다 큰 값은 없어요. 즉, 그냥 시작값 1이에요.
arr[4] 인 50보다 큰 값은 77, 99 두개 예요. 즉, 2를 더해줘요. $1 + 2 = 3$
정답은 24513이에요.
(한 줄 내리지 않아야 해요. print는 줄 내림 없이 출력하고, println은 출력 후 한 줄 내려요)

★ key point

이중으로 for문이 동작하는 것뿐, 어렵지 않아요.
코드를 해석하는 게 어렵다면, **i=0을 한번 손으로 직접 계산**해보세요. 규칙이 보일거예요.

▶ 정답 0+1+2+3+4+5=15
그냥 15만 적으면 틀려요!

[기출예제] 21년 1회 실기 (난이도: ★★★)

클래스 내에서 객체 생성 없이 사용할 수 있는 메소드로써 출력 결과를 작성하시오.

```

1 public class Main {
2     public static void main(String[] args) {
3         int i, j;
4         for(j=0, i=0; i<=5; i++) {
5             j+=i;
6             System.out.print(i);
7             if(i==5) {
8                 System.out.print("=");
9                 System.out.print(j);
10            } else {
11                System.out.print("+");
12            }
13        }
14    }
15 }
```

문제가 크게 어렵지는 않았지만, **조심해야할 포인트가 몇가지** 있어요!

- println이 아닌 print라 줄내림없이 출력할 것!
- 더하기 결과값인 15만 출력하는 것이 아닌, $0+1+\dots+5$ 까지 전부 출력할 것!

실수하지 않도록 조심해야해요!

j와 i에 0을 할당하고 (시작값), i가 5보다 작거나 같을 때까지 +1 증가하면서 반복해요.
+= 연산자는 $j = j + i$ 로 표현할 수 있어요. 즉, 자기 자신과 i를 계속 더해줘요.
i가 0, 1, 2, 3, 4, 5까지 순서대로 출력돼요. println이 아닌 print이므로 줄내림하지 않아요.
i가 5일 때는 + 대신에 =를 출력하기 위한 조건문이에요.
i가 5일 때, =를 출력하고 여태 i를 계속 더했던 j를 출력해요.
i가 5가 아닌 경우에는 +를 출력하는 코드예요.



[문제 유형] 단순 while문

★ while

특정 조건이 참인 동안 계속해서 코드 블록을 실행하게 해요. 마치 "아직 배가 고프다면 계속 음식을 더 먹어"라고 말하는 것과 비슷해요. 조건이 거짓이 되는 순간, 반복이 멈춰요.

```
int i = 0; // 'i'라는 변수를 0으로 시작해요.
while (i < 5) { // 'i'가 5보다 작은 동안 계속 반복해요.
    System.out.println("i의 값은 " + i + "입니다."); // 현재 'i'의 값을 출력해요.
    i++; // 그리고 'i'를 1씩 증가시켜요.
}
```

for 문과의 차이

while 문은 조건이 참인 동안 계속 반복되는데, 주로 반복 횟수가 명확하지 않을 때 사용해요.

for 문은 반복 횟수가 정해져 있거나, 반복을 위한 변수의 시작과 끝이 명확할 때 주로 사용돼요.

★ continue

continue를 만나면, 반복문의 현재 반복을 즉시 종료하고 다음 반복으로 넘어가요.

마치 "이번 단계는 여기까지 하고, 다음 단계로 바로 넘어가자"라는 뜻이에요.

반복문의 나머지 부분을 건너뛰고, 반복문의 시작 부분으로 돌아가 새로운 반복을 시작해요.

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        continue; // i가 5일 때, 여기서 나머지 반복문을 건너뛰고 다음 반복으로 넘어가요.
    }
    if (i == 8) {
        break; // i가 8이 되면, for 문을 완전히 끝내요.
    }
}
```

★ break

break는 반복문을 완전히 끝내는 역할을 해요.

마치 "여기까지만 하고, 반복문을 완전히 벗어나자"라고 할 때 쓰는 것과 같아요.

break를 만나면, 그 즉시 반복문이 종료되고 반복문 바깥의 다음 코드로 넘어가요.

[기출 예제] 20년 3회 실기 (난이도: ★★★)

▶ 정답 30

다음은 자바 코드이다. 출력 결과를 쓰시오.

실제 실행코드

```
1 public class Main {
2     public static void main(String[] args) {
3         int i=0; → i가 0으로 선언 및 할당 되었어요.
4         int sum=0; → sum도 0으로 선언 및 할당 되었어요.
5         while (i<10) { → i가 10 미만일 때까지 반복해요. 여기서는 0,1,2,3,4,5,6,7,8,9 까지 반복해요
6             i++; → i의 값을 바로 +1 증가시켜요. (while문 들어가면서 i가 아닌 i+1로 시작하니 주의해주세요!)
7             if(i%2 ==1) → i를 2로 나눈 나머지가 1일 때, (의미상 홀수를 뜻해요)
8                 continue; → continue해요. 이 키워드는 다음 반복문으로 넘어간다는 의미예요.
9                 sum += i; → 여기서는 [5행]으로 올라가요. (즉, [9행]을 수행하지 않기 위함이에요)
10        }
11        System.out.println(sum);
12    }
13 }
```

[7행] 조건문에 걸리지 않았을 경우, (짝수일 때)
sum 변수에 i를 더해요.
즉, 1,2,3,4,5,6,7,8,9,10 중에서 짝수만 더한 결과값을 출력하겠다는 코드예요

정답은 2+4+6+8+10=30이에요.



[문제 유형] 상속

★ 상속 (Inheritance)

상속은 부모 클래스가 가진 특성(변수)과 행동(메서드)을 자식 클래스가 물려받는 것을 말해요. 마치 부모님의 유전자를 물려받아 닮은 것처럼, 자바에서는 클래스도 다른 클래스의 특성을 물려받을 수 있어요.

```
class Parent {
    void sayHello() {
        System.out.println("Hello, I'm a parent!");
    }
}

class Child extends Parent {
    // Child 클래스는 Parent 클래스의 sayHello 메서드를 상속받았어요.
}
```

Child 클래스는 Parent 클래스로부터 sayHello 메서드를 상속받으니, Child 객체를 만들어 sayHello를 호출하면 "Hello, I'm a parent!"라는 메시지가 출력될 거예요.

★ 오버라이딩 (Overriding)

오버라이딩은 상속받은 메서드를 자식 클래스에서 재정의하는 것을 말해요. 자식 클래스가 상속받은 메서드를 마음에 들게 바꾸고 싶을 때 사용해요.

```
class Child extends Parent {
    void sayHello() {
        System.out.println("Hello, I'm a child!");
    }
}
```

여기서 Child 클래스는 Parent의 sayHello 메서드를 오버라이딩해서 "Hello, I'm a child!"라는 새 메시지를 출력하도록 변경했어요.

★ 오버로딩 (Overloading)

오버로딩은 같은 이름의 메서드를 여러 개 가지되, 매개변수의 유형이나 개수를 다르게 하는 것을 말해요. 이렇게 하면 같은 일을 하는 메서드라도 조금씩 다른 상황에서 사용할 수 있어요.

```
class MathOperations {
    int sum(int a, int b) {
        return a + b;
    }

    // 오버로딩된 sum 메서드, 매개변수의 개수가 다름.
    int sum(int a, int b, int c) {
        return a + b + c;
    }
}
```

MathOperations 클래스에는 sum이라는 메서드가 두 개 있어요. 하나는 두 숫자를 더하고, 다른 하나는 세 숫자를 더해요. 매개변수가 다르기 때문에 두 메서드는 서로 다른 버전으로 존재할 수 있어요.

위 세개의 개념은 자바에서 정말 중요해요! 먼저, 이해하시면 좋아요

[기출 예제] 23년 3회 실기 (난이도: ★★★)

다음은 Java 코드이다. 올바른 출력 결과를 작성하시오.

▶ 정답 "BDCDD"

```

1 public class main{
2     public static void main(String[] args) {
3         ① A b = new B();
4         ② b.paint();
5         ④ b.draw();
6     }
7 }
8
9 class A {
10     public void paint() {
11         System.out.print("A");
12         draw();
13     }
14     ⑤ public void draw() {
15         ⑥ System.out.print("B");
16         ⑦ draw();
17     }
18 }
19
20 class B extends A {
21     ③ public void paint() {
22         ④ super.draw();
23         ⑩ System.out.print("C");
24         ⑪ this.draw();
25     }
26     ⑧ ⑫ public void draw() {
27         ⑨ ⑬ System.out.print("D");
28     }
29 }
30
```

실제 실행코드

자식 클래스 B의 paint() 실행 : [22행] → [14행] 실행 → [15행] "B"출력 → [16행] → [26행] 실행 → [27행] "D"출력

자식 클래스 B의 draw() 실행 : [26행] 실행 → [27행] "D"출력 (끝)

부모 클래스 (A)

클래스 A의 메소드와 클래스 B의 메소드 이름이 동일 (= 자식 클래스인 B 메소드 실행)

자식 클래스 B의 draw() 실행 : [26행]

자식 클래스 B는 부모 클래스 A로부터 상속!

부모 클래스 A의 draw() 호출 = [14행]

자식 클래스 B의 draw() 호출 = [26행]

★ super.~: 부모의 메소드를 호출

★ this.~: 현재 실행 중인 나의 메소드 호출 / 생략도 가능 → draw() = this.draw()

super와 this에 대한 자세한 설명은 다음장에 있어요.



★ this 키워드

this는 현재 객체, 즉 메서드나 생성자를 호출하는 객체를 가리키는 데 사용돼요.
같은 이름의 클래스 변수와 메서드 매개변수가 충돌할 때 이를 구별하기 위해 사용되기도 해요.
클래스 내의 다른 생성자를 호출할 때도 this()를 사용할 수 있어요.

```
class Flower {
    private String name;

    Flower(String name) {
        this.name = name; // 여기서 'this.name'은 클래스 변수를, 'name'은 생성자의 매개변수를 가리켜요.
    }

    void printName() {
        System.out.println(this.name); // 'this'로 클래스 변수 'name'에 접근해요.
    }
}
```

여기서 this.name은 클래스의 name 변수를 참조하고, 단순히 name은 생성자로 전달된 매개변수를 참조해요.

★ super 키워드

super는 부모 클래스를 가리키는 데 사용되며, 부모 클래스의 변수나 메서드, 생성자에 접근할 때 사용해요.
자식 클래스에서 오버라이드한 메서드가 부모 클래스의 메서드를 호출하고 싶을 때 super를 사용해요.

```
class Plant {
    String type = "Plant";

    void printType() {
        System.out.println(this.type); // 'this'로 자신의 type 변수에 접근해요.
    }
}

class Flower extends Plant {
    String type = "Flower";

    void printType() {
        super.printType(); // 'super'로 부모 클래스의 printType 메서드를 호출해요.
        System.out.println(this.type); // 'this'로 자신의 type 변수에 접근해요.
    }
}
```

Flower 클래스에서 super.printType()은 Plant 클래스의 printType 메서드를 호출하고, this.type은 Flower 클래스 자신의 type 변수를 참조해요.

▶ 정답 a=10

[기출 예제] 20년 2회 실기 (난이도: ★★☆☆)

다음은 자바 코드이다. 출력 결과를 쓰시오.

A 부모 클래스	1 2 3 4 5 6 7 8 9 10	class A { private int a; ④ public A(int a){ this.a = a; } ⑥ public void display(){ System.out.println("a=" + a); } }	a의 멤버 변수를 10을 할당해요.
B 자식 클래스	11 12 13 14 15 16 17 18	class B extends A { ② public B(int a){ ③ super(a); ⑤ super.display(); } }	super()는 부모 클래스의 생성자를 호출하는 키워드예요. [3행]이 수행돼요. 부모의 [6행] display()를 호출해요. 즉, a에 10을 할당한 뒤, a를 출력하는 코드예요. 정답은 a=10이에요
실제 실행코드	19 20 21 22 23	public class Main { public static void main(String[] args){ ① B obj = new B(10); } }	B의 생성자에 10을 넣어 B 객체를 생성합니다. [12행]이 수행돼요.



▶ 정답 11

[기출예제] 21년 2회 실기 (난이도: ★★★)

다음은 JAVA 관한 문제이다. 알맞는 출력값을 작성하시오.

```

1 public class Ovr1 { Ovr1 부모 클래스
2     public static void main(String[] args) {
3         ①Ovr1 a1 = new Ovr1(); Ovr1 타입의 객체 a1을 생성하고 있어요.
4         ②Ovr2 a2 = new Ovr2(); Ovr2 타입의 객체 a2를 생성하고 있어요. Ovr2는 Ovr1를 상속받았기 때문에 Ovr1의 메서드를 가지고 있어요.
5         ③System.out.println(a1.sun(3,2) + a2.sun(3,2)); → Ovr1의 sun은 3+2 = 5이고,
6     }                                     Ovr2의 sun은 3-2+5 = 6이기 때문에 정답은 11이 출력돼요.
7
8     ④int sun(int x, int y) { → Ovr1 클래스에 sun이라는 메서드를 정의하고 있어요.
9         return x + y;         두 정수를 받아서 그 합을 반환합니다.
10    }
11 }
12
13 class Ovr2 extends Ovr1 {
14     ⑤int sun(int x, int y) { → sun 메서드를 재정의(오버라이드)하고 있어요.
15         return x - y + ⑥super.sun(x,y); 이 메서드는 인자로 받은 두 정수의 차에 부모 클래스의 sun 메서드의 결과를 더해 반환합니다.
16     } super.sun(x,y)는 부모 클래스인 Ovr1의 sun 메서드를 호출하는 방법이에요.
17 }
18 }

```

★extends

class 자식 extends 부모의 형태로 사용해요.

부모를 상속 받은 자식은 부모의 변수 및 메소드를 사용할 수 있고, 오버라이드를 통해 내 코드로 수정해서 사용할 수 있어요.

여기서는 [15행]이 부모의 sun을 오버라이드해서 사용하고 있어요. 이럴 경우, 자식 코드에서 부모의 메소드를 호출 하고 싶을 때는 super.메소드 명 으로 사용할 수 있어요.

▶ 정답 5000

[기출예제] 23년 1회 실기 (난이도: ★★★)

아래 자바 코드에서 출력되는 값을 작성 하시오.

```

1 class Parent {
2     int x = 1000;
3
4     Parent() {
5         this(3000);
6     }
7
8     Parent(int x) {
9         this.x = x;
10    }
11 }
12
13 class Child extends Parent {
14     int x = 4000;
15
16     ②Child() {
17         ③this(5000);
18     }
19
20     ④Child(int x) {
21         ⑤this.x = x;
22     }
23
24     ⑦int getX() {
25         ⑧return x;
26     }
27 }
28
29 public class Main {
30     public static void main(String[] args) {
31         ①Child obj = new Child();
32         ⑥System.out.println(obj.getX());
33     }
34 }

```

부모 클래스

자식 클래스

실제 실행코드

기본 생성자 [4행]은 [8행] 생성자를 호출해서, X에 3000을 할당해요.

기본 생성자 [16행]은 [20행] 생성자를 호출해서, X에 5000을 할당해요.

만약, getX()가 (Child에 없고) Parent에만 구현되어 있었다면, 정답은 3000이 되었을거예요. 코드를 보면, getX()는 자기 클래스의 x를 반환하니깐요.

Child를 기본 생성자로 생성했어요. [16행] -> [17행] -> [20행] -> [21행]의 순서에 의해 Child의 x는 5000이 되었어요. Child의 getX() 메소드는 [24행] Child의 x를 반환하기 때문에 정답은 5000이 돼요.



[기출 예제] 23년 3회 실기 (난이도: ★★★)

▶ 정답 2

다음은 Java 코드이다. 올바른 출력 결과를 작성하시오.

부모 클래스 (Parent)

```

1 class Parent {
2     int compute(int num) {
3         if(num <= 1)
4             return num;
5         return compute(num-1) + compute(num-2);
6     }
7 }
8

```

자식 클래스 (Child)

```

9 class Child extends Parent {
10    int compute(int num) {
11        if(num <= 1)
12            return num;
13        return compute(num-1) + compute(num-3);
14    }
15 }
16

```

실제 실행코드

```

17 public class main {
18     public static void main(String args[]) {
19         Parent obj = new Child();
20         System.out.print(obj.compute(7));
21     }
22 }
23

```

이 문제에서는 호출되지 않아요.
[19행]에서 Parent obj = new Parent();로 할당 되었다면 해당 메소드가 호출되었을 것.

실제 호출되는 메소드.

n이 1 이하일 때, n을 반환한다 (종료조건)

자식 클래스 Child 객체를 부모 클래스 Parent에 할당.

겉대기는 부모 클래스이지만, 자식 객체가 할당되었으므로, compute 메소드는 자식클래스인 Child 객체의 메소드로 동작해요.

로직의 수행은 왼쪽 선언된 타입이 아닌 오른쪽에 할당되는 객체의 메소드로 동작한다는 사실을 기억하면 좋아요!

자기 자신을 계속 호출하고 있어요.

[20행]에서 7을 넣어줬으니, compute(6) + compute(4)가 호출돼요.

* compute(n)에서 n이 1 이하일 때, n을 반환해요.

즉, compute(6)의 값과 compute(4)의 값을 구해서 더해주면 정답이에요.

compute(6)은 compute(5) + compute(3)의 합
compute(5)는 compute(4) + compute(2)의 합
compute(4)는 compute(3) + compute(1)의 합
compute(3)은 compute(2) + compute(0)의 합
compute(2)는 compute(1) + compute(-1)의 합

compute(1) + compute(-1)부터 거꾸로 계산하면서 올라가면서 풀어요.

즉, compute(6)는 1, compute(4)는 1이므로, 정답은 2

[기출 예제] 20년 4회 실기 (난이도: ★★★)

다음은 자바 소스 코드이다. 출력 결과를 쓰시오.

Parent 부모 클래스

```

1 class Parent{
2     public int compute(int num){
3         if(num <=1) return num;
4         return compute(num-1) + compute(num-2);
5     }
6 }
7

```

Child 자식 클래스

```

8 class Child extends Parent {
9     public int compute(int num){
10        if(num<=1) return num;
11        return compute(num-1) + compute(num-3);
12    }
13 }
14

```

실제 실행코드

```

15 class Main{
16     public static void main (String[] args){
17         Parent obj = new Child();
18         System.out.print(obj.compute(4));
19     }
20 }

```

매개변수 num이 1과 같거나 작을 때, num을 반환해요. (종료조건)

Parent 부모 클래스를 선언 해 실제 객체는 자식인 Child를 생성해 할당 했어요.

[9행]을 실행시켜요.

동일한 메소드가 부모, 자식에 존재할 경우, 자식 클래스의 메소드를 호출되고,
- 위의 경우를 오버라이드 라고 표현해요. (부모의 메소드를 자식이 새롭게 구현하는 경우)
- 단, 메소드 이름, 매개변수들의 개수와 타입, 반환 타입이 전부 같아야 해요

부모에만 존재하는 메소드일 경우에만 부모 메소드로 호출돼요.

[18행] compute메소드의 경우, 자식에도 동일한 메소드가 존재하므로, 자식의 compute가 호출되고, 부모 compute는 무시됩니다.

자기 자신을 계속 호출하고 있어요.

[18행]에서 4를 넣어줬으니, compute(3) + compute(1)이 호출돼요.

* compute(n)에서 n이 1 이하일 때, n을 반환해요. (종료조건)

즉, compute(3)의 값과 compute(1)의 값을 구해서 더해주면 정답이에요.

compute(3)은 compute(2) + compute(0)의 합
compute(2)는 compute(1) + compute(-1)의 합
compute(1)은 1과 같으니 ([10행] 조건문), 1을 반환해요. = 1

즉, compute(3)는 0, compute(1)는 1이므로, 정답은 1이에요

23년도 3회 실기 문제와 숫자만 다른 동일한 문제예요.



▶ 정답 new

[기출예제] 20년 2회 실기 (난이도: ★★★)

다음은 자바 코드이다. 다음 밑줄에 들어갈 키워드를 쓰시오.

```

1  class Parent {
2      public void show() {
3          system.out.println("Parent");
4      }
5  }
6
7  class Child extends Parent {
8      ③public void show() {
9          ④system.out.println("Child");
10     }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         ①Parent pa = _____ Child();
16         ②pa.show();
17     }
18 }

```

Parent 부모 클래스

Child 자식 클래스

실제 실행코드

Parent 부모 클래스의 show()가 Child 자식 클래스 show로 오버라이드 되었어요.

즉, [16행] 호출 시, [8행]의 show가 호출돼요. 물론, 이 문제와는 상관없어요!

★ key point

이제는 아마, 나을 일 없는 가장 기본적인 **자바의 객체 생성에 대한 문법 문제**이예요.

자바에서 생성자를 만드는 기본 문법은 A a = new A()입니다. A는 클래스명이에요. 여기서서는 Parent pa = new Child(); 이 부분이에요. 그래서 정답은 **new**가 돼요.

가장 기본적인 문법이에요.

[기출예제] 24년 1회 실기 (난이도: ★★★)

아래 JAVA언어 코드의 실행 순서를 중복 번호없이 작성하시오.
실행 순서 : 5 → () → () → () → () → ()

```

1  class Parent {
2      int x, y;
3
4      Parent(int x, int y) { ①
5          this.x=x;
6          this.y=y;
7      }
8
9      int getT() { ②
10         return x*y;
11     }
12 }
13
14 class Child extend Parent {
15     int x;
16
17     Child (int x) { ③
18         super(x+1, x);
19         this.x=x;
20     }
21
22     int getT(int n){ ④
23         return super.getT()+n;
24     }
25 }
26
27 class Main {
28     public static void main(String[] args) { ⑤
29         Parent parent = new Child(3); ⑥
30         System.out.println(parent.getT()); ⑦
31     }
32 }

```

해당 문제는 **실행 순서만을 작성**하는 것이기 때문에 절대 어렵지 않아요. 코드 파라미터 등을 신경 쓰지 말고 순서대로 따라가면 돼요.

▶ 정답
6 → 3 → 1 → 7 → 2

우리는 main 부터 로직이 수행되는 걸 알고 있습니다. (모르고 있더라도 보기에서 5번이라고 적어줬기 때문에 상관없습니다) 5번 부터 수행합니다.

로직은 한 줄 씩 수행하면서 내려가기 때문에 **6번**이 수행됩니다. new Child(3);이 수행되면서 Child 클래스의 생성자 **3번**을 호출합니다. (1번이 아니라 3번이 호출되는 이유는 new Child(); 이기 때문이에요. 해당 라인에서 1번이 호출되려면 new Parent(); 가 되어야 해요.)

한 줄 내려가면 **super** 메소드를 호출합니다. super는 부모 this 는 자기 자신인데, super() 는 **super의 생성자**를 뜻해요. **1번**으로 이동해 super() 생성자를 수행하고, 돌아오면 [29행]이 종료돼요.

한 줄 내려가 **7번**이 수행되고, parent.getT()가 수행되어야 출력될 수 있으니 **2번**이 수행되며 값을 반환하고 (4번이 아니라 2번일까? 하고 헷갈릴 수 있어요. Child는 Parent를 상속받고 있고 부모의 메소드를 전부 사용할 수 있어요. 2번과 4번은 메소드명만 getT로 같고 파라미터가 다르기 때문에 오버로딩으로 아예 다른 메소드예요. parent.getT();는 파라미터가 없는 메소드 호출했기 때문에 Parent의 getT()가 호출된거예요.)

그 값이 출력되며 프로그램이 종료됩니다.



▶ 정답 9

[기출 예제] 24년 1회 실기 (난이도: ★★☆☆)

다음 Java 코드를 보고 알맞는 출력 값을 작성하시오.

```

1  class classOne {
2      int a, b;
3
4      public classOne(int a, int b) {
5          this.a = a;
6          this.b = b;
7      }
8
9      public void print() {
10         System.out.println(a + b);
11     }
12 }
13
14
15 class classTwo extends classOne {
16     int po = 3;
17
18     public classTwo(int i) {
19         super(i, i+1);
20     }
21
22     public void print() {
23         System.out.println(po*po);
24     }
25 }
26
27 public class Main {
28     public static void main(String[] args) {
29         classOne one = new classTwo(10);
30         one.print();
31     }
32 }

```

Parent
부모 클래스

Child
자식 클래스

실제
실행코드

★key point

a와 b에 값을 할당하는 로직을 수행해서,
Print 메소드도 부모 메소드로 호출되나? 하고 헛갈릴 수 있어요.

하지만, 부모 자식 간의 호출 방식을 이해하고 있었다면
정말 쉽게 풀 수 있는 문제예요

`new classTwo(10);`를 통해 객체를 생성합니다.
[18행]이 호출되고, [19행]에서 `super(10, 11);` 이 호출됩니다.
[4행]이 호출되고, a는 10, b는 11이 할당됩니다.

`one.print();`이 호출될 때 코드는 먼저 자식에 `print()`가 존재하는지 확인하고,
없으면 부모 `print()`를 수행해요.

자식에 `print()`가 존재하고 그 로직은 `po` 변수와 `po`변수를 곱한 값을 출력하는 거예요
정답은 $3 * 3$ 인 9가 돼요.

★key point

자바의 상속과 그에 따른 다형성과 멤버 변수 참조에 대해서 잘 아는지
체크하는 문제예요.

여기서 중요한 점은

- 멤버 변수는 참조 변수의 타입을 따르고,
- 함수는 실제 객체의 타입을 따라요.

▶ 정답 52

[기출 예제] 24년 3회 실기 (난이도: ★★☆☆)

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1  public class Main {
2      public static void main(String[] args) {
3          Base a = new Derivate();
4          Derivate b = new Derivate();
5
6          System.out.print(a.getX() + a.x + b.getX() + b.x);
7      }
8  }
9
10
11 class Base {
12     int x = 3;
13
14     int getX() {
15         return x * 2;
16     }
17 }
18
19 class Derivate extends Base {
20     int x = 7;
21
22     int getX() {
23         return x * 3;
24     }
25 }

```

실제 실행
함수

부모 클래스

자식 클래스

a라는 변수를 선언해 부모 타입 Base로 객체를 생성하지만,
실제로 자식 클래스 Derivate의 객체를 할당해요.

즉, `a.x`는 3이지만, `a.getX()`는 Derivate의 `getX()`를 호출해 $7 * 3 = 21$ 이 돼요.

b는 자식 클래스 Derivate로 객체 생성 및 할당을 해요.

즉, `b.x`는 7이고, `b.getX()`도 $7 * 3 = 21$ 이 돼요.

그래서 $21(a.getX()) + 3(a.x) + 21(b.getX()) + 7(b.x) = 52$ 가 출력돼요.



▶ 정답 54

[기출 예제] 25년1회 실기 (난이도: ★★★)

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1  public class Main {
2      public static void main(String[] args) {
3          new Child();
4          System.out.println(Parent.total);
5      }
6  }
7
8
9  class Parent {
10     static int total = 0;
11     int v = 1;
12
13     public Parent() {
14         total += (++v);
15         show();
16     }
17
18     public void show() {
19         total += total;
20     }
21 }
22
23
24 class Child extends Parent {
25     int v = 10;
26
27     public Child() {
28         v += 2;
29         total += v++;
30         show();
31     }
32
33     @Override
34     public void show() {
35         total += total * 2;
36     }
37 }

```

실제 실행코드

Parent 부모 클래스

Child 자식 클래스

★key point

아래 포인트를 알면 너무 쉬운 문제예요.

1. 생성자는 항상 부모→자식 순으로 실행돼요.
2. show()는 오버라이딩된 자식 메서드가 호출돼요.
3. static 변수 total은 클래스 전체에서 하나만 공유돼요.

- [3행] new Child()를 호출하면,

1. Parent() 생성자가 호출

- ① ++v = v (Parent) 1 → 2, total = 0 + 2(++v) = 2
- ② show() 호출 → 오버라이딩된 child.show() 호출
 - total = 2(total) + (2(total) * 2) = 6

2. Child() 생성자 실행

- ① v = 10(v) + 2 = 12
- ② total = 6 + 12(v) = 18 (그 뒤에 v는 13이 돼요)
- ③ show() 호출 → 오버라이딩된 child.show() 호출
 - total = 18 + (18*2) = 54

- [4행] System.out.println(Parent.total)

- 최종 total 값인 54가 출력돼요.



[문제 유형] 추상 클래스

★ abstract

자바에서 추상 클래스란, 완전하지 않은 '미완성 설계도' 같은 것인데요. 직접적으로 객체를 만들 수는 없지만, 다른 클래스들이 상속받아 완성해야 하는 기본 틀을 제공해요. 이 추상 클래스는 추상 메서드를 포함할 수 있어요. 추상 메서드는 실행할 구체적인 코드가 없고, 그저 메서드의 이름, 입력값, 반환 타입만 정의돼 있죠. 상속받는 클래스는 이 추상 메서드들을 반드시 구현(완성)해야 해요.

예를 들어, 모든 차에는 주행 기능이 있지만, 각기 다른 차의 주행 방식은 조금씩 다를 수 있어요. 이 때 '차'를 추상 클래스로 생각하고 '주행' 기능을 추상 메서드로 두면, 각 차는 자신의 방식으로 '주행' 메서드를 구현하게 되죠.

```
abstract class Car {
    abstract void drive(); // 추상 메서드: '어떻게 주행할지'는 여기서 정의하지 않아요.

    void startEngine() {
        System.out.println("엔진이 켜졌어요!"); // 일반 메서드: 모든 차가 공통적으로 사용해요.
    }
}

class Sedan extends Car {
    void drive() {
        System.out.println("매끄럽게 도로를 주행해요!"); // '주행'에 대한 구체적인 구현이 여기에 와요.
    }
}

class Truck extends Car {
    void drive() {
        System.out.println("강력하게 짐을 싣고 달려요!"); // 트럭에 맞게 '주행'을 다르게 구현해요.
    }
}
```

여기서 Car는 추상 클래스로서, drive 메서드를 추상 메서드로 가지고 있어요. Sedan과 Truck은 Car 클래스를 상속받아 drive 메서드를 각자의 방식으로 구현하죠. Car 클래스는 직접 new Car()로 객체를 만들 수 없지만, Sedan과 Truck은 실제 사용 가능한 차들이 되어요.

[기출예제] 23년 1회 실기 (난이도: ★★★)

▶ 정답 Vehicle name: Spark

다음 JAVA 코드에서 알맞는 출력 값을 작성하시오.

부모 클래스	1	abstract class Vehicle {	class 앞에 abstract 키워드가 붙으면 추상 클래스가 돼요. 이는 단독으로 구현(생성)될 수 없음을 의미해요. 여기서의 Car 클래스처럼 상속으로만 생성 가능해요.	
	2	⑤ private String name;		
	3	abstract public String getName(String val);		abstract 키워드가 붙여 getName(val) 메소드의 구현을
	4	⑦ public String getName() {		명시적으로 자식에게 넘겼어요. (추상 클래스에서는 구현을 자식에게 미룰 수 있는 효과가 있어요)
	5	return "Vehicle name: " + name;		getName() 메소드의 구현은 Vehicle에 되어있어요.
자식 클래스	6	}	추상 클래스라고 모든 메소드를 구현하지 못하는 건 아니에요.	
	7	}		
	8	}		
	9			
	10			
	11			
	12	class Car extends Vehicle {		Car의 name 변수와 부모인 Vehicle의 name 변수를 동시에 val 매개변수로 할당하는 코드예요.
	13	④ private String name;		
자식 클래스	14	② public Car(String val) {	Vehicle의 추상 메소드 getName(String val)을 오버라이드해서 구현한 메소드예요. 여기서는 사용되지 않아요.	
	15	③ name=super.name=val;		
	16	}		
	17			
	18	public String getName(String val) {		
실제 실행코드	19	return "Car name: " + val;	Vehicle과 상관없는 Car의 메소드예요. 여기서는 사용되지 않아요.	
	20	}		
	21			
	22	public String getName(byte val[]) {		
	23	return "Car name: " + val;		
	24	}		
	25	}		
	26			
	27	public class Main {		
	28			
29	public static void main(String[] args) {	Vehicle 클래스에 Car 객체를 생성 후 할당했어요. 여기서는 [5행] Vehicle의 getName() 메소드가 호출돼요. 자식인 Car 객체에는 getName() 메소드가 없기 때문이에요		
30				
31	① Vehicle obj = new Car("Spark");			
32	⑥ System.out.println(obj.getName());			
33	}			
34	}			
35	}			
36	}			

★ abstract

'완성(구현)되지 않은' 클래스나 메서드를 지정할 때 사용되며, 이를 상속받은 클래스에서 완성해야 합니다.

★ 오버라이드

상속받은 클래스에서 부모 클래스의 메서드를 재정의하여 새로운 기능으로 바꾸는 것이예요.

★ 오버로딩

같은 이름의 메서드를 매개변수만 다르게 해서 여러 개 만드는 것이예요. 이름이 같더라도, 매개변수의 타입이나 매개변수의 개수 가 다르면 다른 메소드처럼 사용할 수 있어요.



▶ 정답 Vehicle name: Spark

[기출 예제] 20년 3회 실기 (난이도: ★★★)

다음은 자바 코드이다. 출력 결과를 쓰시오.

class 앞에 abstract 키워드가 붙으면 **추상 클래스**가 돼요.
이는 **단독으로 구현(생성)**될 수 없음을 의미해요.
여기서의 Car 클래스처럼 **상속으로만 생성** 가능해요.

```

1  abstract class Vehicle{
2      private String name;
3      abstract public String getName(String val);
4
5      public String getName(){
6          return "Vehicle name:" + name;
7      }
8      public void setName(String val){
9          name = val;
10     }
11 }
12
13 class Car extends Vehicle{
14     public Car(String val){
15         setName(val);
16     }
17     public String getName(String val){
18         return "Car name : " + val;
19     }
20     public String getName(byte val[]){
21         return "Car name : " + val;
22     }
23 }
24
25 public class Main {
26     public static void main(String[] args){
27         Vehicle obj = new Car("Spark");
28         System.out.print(obj.getName());
29     }
30 }

```

Vehicle
부모 클래스

Car
자식 클래스

실제
실행코드

abstract 키워드를 붙여 getName(val) 메소드의 구현을

명시적으로 자식에게 넘겼어요.

(추상 클래스에서는 구현을 자식에게 미룰 수 있는 효과가 있어요)

abstract 클래스도 메소드를 구현할 수 있어요.

구현하지 않고, 자식에게 구현을 미룰 수도 있구요.

여기서는 [28행] getName()은 Car(자식 클래스)에서

오버라이드 하지 않고 있어서, 부모의 [5행] getName()이 호출돼요.

즉, 정답은 Vehicle name: Spark 이예요.

사실상, Car 자식 클래스가 이 코드에서는 사용되지 않아요.

코드 설명만 해보자면, Vehicle의 [3행] 추상 메소드는 자식에서
무조건 구현해줘야 해요.

그게 [17행]에서 구현되었어요.

[20행]은 메소드명은 같지만, 매개변수 타입이 다르므로 전혀
다른 메소드예요. 그냥 Car에서만 구현된 Car만의 메소드예요.

[14행]은 Car의 생성자로, 매개변수 val를 부모인 Vehicle의 setName
메소드를 이용해서 name 변수에 값을 할당해주고 있어요.

(여기서는 Spark로 할당)

부모의 메소드는 super.setName(val)의 형태로 호출해야하지만,
따로 자식에서 동일한 메소드가 존재하지 않으면 **super 키워드**
없이도 호출 가능해요.

super를 붙여야 할 때는, 자식에도 setName 메소드를

오버라이딩 했을 경우에 부모의 setName을 호출하고 싶을 때 예요.

23년도 1회 실기 문제와 동일한 문제예요.



[문제 유형] 인터페이스

★ interface

추상클래스와 정말 유사하게 ‘구현되지 않은’ 메서드를 미리 선언하고, 이를 상속받은 클래스에서 완성해야 합니다.

사실, 여기서는 interface의 개념을 모르더라도 크게 문제 풀이 지장이 없어요.

그냥 추상 클래스와 비슷하게 인터페이스를 상속받은 클래스는 인터페이스가 가진 메소드를 구현해야 한다는 개념만 알고 넘어가도 돼요.

추상 클래스와 차이점을 가볍게 설명하자면, (이해 하지 않아도 돼요)

추상 클래스는 멤버 변수를 가질 수 있고, 인터페이스는 멤버 변수를 가질 수 없어요.

▶ 정답
25, 20

[가출 예제] 24년 2회 실기 (난이도: ★★☆☆)

다음은 Java 언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

실제 실행코드 <pre> 1 class Main { 2 public static void main(String[] args) { 3 int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9}; 4 ODDNumber OE = new ODDNumber(); 5 System.out.print(OE.sum(a, true) + ", " + OE.sum(a, false)); 6 } 7 } </pre>	→ 배열 a에 [1, 2, 3, 4, 5, 6, 7, 8, 9] 를 할당합니다. → ODDNumber 클래스를 생성 후 OE 변수에 할당해요. → OE.sum(a, true)와 OE.sum(a, false)의 값을 받아서 출력합니다.
인터페이스 <pre> 9 interface Number { 10 int sum(int[] a, boolean odd); 11 } </pre>	Number를 상속 받았기 때문에 메소드 sum을 구현해야 해요. sum의 인자로 true일 때, 배열 a의 홀수들의 합 false일 때, 짝수들의 합을 반환해요. 모든 홀수의 합 (1 + 3 + 5 + 7 + 9 = 25) 모든 짝수의 합 (2 + 4 + 6 + 8 = 20) 그러므로 정답은 25, 20가 출력이 돼요
구현체 <pre> 13 class ODDNumber implements Number { 14 public int sum(int[] a, boolean odd) { 15 int result = 0; 16 for(int i=0; i < a.length; i++) { 17 if((odd && a[i] % 2 != 0) (!odd && a[i] % 2 == 0)) { 18 result += a[i]; 19 } 20 } 21 return result; 22 } 23 } </pre>	→ 배열 a의 0부터 끝까지 루프를 돌면서 아래의 조건에 해당 하는지 체크해요. → <code>odd && a[i] % 2 != 0</code> → result에 i번째 값을 더해줘요. → 배열을 다 돌면 result를 반환해요. 즉, odd가 true이면 배열의 홀수 값의 합 odd가 false이면 배열의 짝수 값의 합을 반환하는 메소드예요. 이거나 → <code>!odd && a[i] % 2 == 0</code> odd가 false이면서 i 번째 배열의 값이 짝수 일 때 (a[i]를 2로 나눈 나머지가 0일 때)



[문제 유형] JAVA 문법

[기출 예제] 21년 3회 실기 (난이도: ★★★)

▶ 정답 3

다음 Java 코드에 대한 알맞는 출력값을 쓰시오.

Connection 클래스

```

1  class Connection {
2      private static Connection _inst = null;
3      private int count = 0;
4
5      static public Connection get() {
6          if (_inst == null) {
7              _inst = new Connection();
8              return _inst;
9          }
10         return _inst;
11     }
12
13     public void count() {
14         count++;
15     }
16
17     public int getCount() {
18         return count;
19     }
20 }

```

실제 실행코드

```

21
22 public class testcon {
23     public static void main(String[] args) {
24         Connection conn1 = Connection.get();
25         conn1.count();
26         Connection conn2 = Connection.get();
27         conn2.count();
28         Connection conn3 = Connection.get();
29         conn3.count();
30
31         System.out.print(conn1.getCount());
32     }
33 }

```

→ _inst라는 정적 변수를 사용해서 인스턴스를 저장하고 있어요. 처음에는 _inst가 null이에요.

→ get() 메서드를 호출하면 _inst가 null인지 확인해요. 만약 null이면 새로운 Connection 인스턴스를 생성하고 _inst에 할당해요. 이미 _inst에 인스턴스가 있다면 그 인스턴스를 그대로 반환해요.

→ 호출 시, count 멤버 변수의 값을 +1씩 증가 시켜요.

→ 호출 시, count 를 반환해요.

→ 즉, [24행] [26행] [28행] 모두 같은 Connection 객체를 반환해요. **최초에 null일 경우에만 생성**하고, 그 이후에는 이미 생성된 객체를 반환하니까요.

→ 그러므로, Connection 객체 안의 멤버변수 count도 동일한 변수이고, 어디서 count() 메소드를 호출하더라도 같은 변수의 값이 +1씩 증가해요. count()가 총 3번 호출되었으니, 정답은 3이에요.

★ 싱글톤 디자인 패턴

클래스의 인스턴스를 오직 하나만 생성하고, 그 인스턴스를 어디서든 접근할 수 있게 해주는 디자인 패턴이에요.

클래스의 인스턴스를 하나만 생성하고 싶을 때 사용해요.

인스턴스는 오직 하나만 생성되고, 프로그램 어디에서든 Connection.get()을 호출하면 같은 인스턴스를 받을 수 있어요.

[기출 예제] 24년 1회 실기 (난이도: ★★★)

다음 Java 코드에 대한 알맞는 출력값을 쓰시오.

21년도 3회 실기에서 나왔던 문제가 24년도에 동일하게 출제되었어요. 싱글톤 패턴과 static에 대해서 알고 있었다면 쉽게 맞출 수 있었어요.

▶ 정답 4

Connection 클래스

```

1  class Connection {
2
3      private static Connection _inst = null;
4      private int count = 0;
5
6      static public Connection get() {
7          if(_inst == null) {
8              _inst = new Connection();
9              return _inst;
10         }
11         return _inst;
12     }
13
14     public void count() {
15         count++;
16     }
17
18     public int getCount() {
19         return count;
20     }
21 }

```

실제 실행코드

```

22
23 public class main {
24     public static void main(String[] args) {
25
26         Connection conn1 = Connection.get();
27         conn1.count();
28
29         Connection conn2 = Connection.get();
30         conn2.count();
31
32         Connection conn3 = Connection.get();
33         conn3.count();
34
35         conn1.count();
36         System.out.print(conn1.getCount());
37     }
38 }

```

→ _inst라는 정적 변수를 사용해서 인스턴스를 저장하고 있어요. 처음에는 _inst가 null이에요. count는 멤버 변수로 정적 변수는 아니지만 (= Connection 객체마다 생성되는 변수지만) Connection 객체가 _inst 변수로 싱글톤 패턴을 통해 한번만 생성되어 사용되기 때문에 사실상, count 변수는 하나의 객체에 하나의 변수로만 사용돼요.

→ get() 메서드를 호출하면 _inst가 null인지 확인해요. 만약 null이면 새로운 Connection 인스턴스를 생성하고 _inst에 할당해요. 이미 _inst에 인스턴스가 있다면 그 인스턴스를 그대로 반환해요.

→ 호출 시, count 멤버 변수의 값을 +1씩 증가 시켜요.

→ 호출 시, count 를 반환해요.

→ [28행] [31행] [34행] 모두 같은 Connection 객체를 반환해요.

→ 그러므로, Connection 객체 안의 멤버변수 count도 동일한 변수이고, 어디서 count() 메소드를 호출하더라도 같은 변수의 값이 +1씩 증가해요. count()가 [29행] [32행] [35행] [37행] 총 4번 호출되었으니, 정답은 4이에요.



[문제 유형] JAVA 문법

▶ 정답 -8

[기출예제] 22년 2회 실기 (난이도: ★★☆☆)

아래 자바 코드에서 출력되는 값을 작성하시오.

실제
실행코드

```

1  class Test{
2      public static void main(String args[]){
3
4          int i = 3;
5          int k = 1;
6
7          switch(i) {
8              case 1: k += 1;
9              case 2: k++;
10             case 3: k = 0;
11             case 4: k += 3;
12             case 5: k -= 10;
13             default: k--;
14         }
15
16         System.out.print(k);
17     }
18 }

```

정수형 변수 i를 선언하고 3을 할당합니다.
정수형 변수 k를 선언하고 1을 할당합니다.

i가 3, case 1은 통과해요.
i가 3, case 2은 통과해요.
i가 3, case도 3이기에, k에 0을 할당해요.
break: 가 없기 때문에 빠져나오지 못하고
case 4, case 5, default까지 로직을 수행해요.

즉, +3 (case 4) - 10 (case 5) -1 (default)를
전부 수행해서 k는 -8이 됩니다.

정답은 -8입니다.

★ switch

여러 개의 선택 사항 중 하나를 선택하여 실행하는 제어문이에요.

기본 구조는 다음과 같아요.

```

switch (변수) {
    case 값1:
        // 변수가 값1과 같을 때
        break;
    case 값2:
        // 변수가 값2와 같을 때
        break;
    ...
    default:
        // 어떤 case에도 해당하지 않을 때
        break;
}

```

break는 현재 case를 종료하고 switch 구문을 빠져나가는 키워드예요.

일치하는 case까지만 실행하고, switch 구문이 종료돼요.

여기서는 break 구문이 없으니, 빠져나오지 못해요.

그래서 [10행] case 3에 걸린 이후로 아래의 case 로직들도 전부 수행돼요

[기출예제] 24년 2회 실기 (난이도: ★★☆☆)

다음은 Java 코드에 대한 문제이다.

아래 코드를 확인하여 알맞는 출력값을 작성하시오.

실제
실행코드

```

1  class Main {
2      public static void main(String[] args) {
3          int[] a = new int[] {1, 2, 3, 4};
4          int[] b = new int[] {1, 2, 3, 4};
5          int[] c = new int[] {1, 2, 3};
6
7          check(a, b);
8          check(a, c);
9          check(b, c);
10     }
11
12     public static void check(int[] a, int[] b) {
13         if (a == b) {
14             System.out.print("O");
15         } else {
16             System.out.print("N");
17         }
18     }
19 }

```

a: [1, 2, 3, 4]
b: [1, 2, 3, 4]
c: [1, 2, 3]

아래의 check 메소드에 위에서 생성한 배열들을 조합해 호출합니다.
인자로 넘겨진 두 배열 같은 객체이면 O, 다른 객체이면 N을 출력합니다.
(정확하게 표현하면, 객체의 주소를 비교합니다)
배열 안에 들어간 값과 전혀 상관없이 a, b, c 배열 모두 새롭게 별도로 생성된 객체이기에 별도의 주소를 가지고 있습니다.
그러므로 정답은 NNN이 됩니다.

▶ 정답
NNN

★ 비교 연산자

개념만 알고 있다면 정말 쉽게 풀 수 있는 문제예요.

기본형 타입 (int, Boolean 등)의 경우, 그 값이 같은 지 비교합니다.

ex) int a = 1; int b = 1 일 때, a와 b는 같습니다.

객체 타입 (배열, String, 사용자 정의 클래스 등)의 경우, == 연산자로 비교할 때 **메모리 주소를 비교**합니다.

홍길동이라는 사람이 2명을 생성한다고 했을 때 그 둘이 같지 않은 것과 동일합니다.

ex) Person p1 = new Person("홍길동"); Person p2 = new Person("홍길동");

p1 == p2는 false가 됩니다.

메모리 주소 안에서 두 개의 객체가 생성되었고, 그 두 객체 주소는 다르니까요.



▶ 정답 static

[기출예제] 21년 2회 실기 (난이도: ★★★)

클래스 내에서 객체 생성 없이 사용할 수 있는 메소드로서 빈칸에 들어갈 내용을 작성하시오.

[출력결과]

positive

실제
실행코드

```
1 public class Test {
2     public static void main(String[] args) {
3         System.out.print(Test.check(1));
4     }
5
6     ( ) String check (int num) {
7         return (num >= 0) ? "positive" : "negative";
8     }
9 }
```

[3행]을 보면 객체 생성 없이 메소드를 호출하고 있죠.
이런 경우는 **static** 키워드가 붙어있어야 가능해요.
물론, `System.out.println(check(1));`도 호출 가능해요.

★만약, static을 안 쓰면서 호출하려면 어떻게 코드가 작성되어야 할까요?

[3행]이 아래와 같이 대체되면 동일하게 동작해요!

```
Test test = new Test();
System.out.print(test.check(1));
```

★static

자바 프로그래밍에서 'static'을 사용하면, **그 요소는 특정 객체에 속하지 않고, 해당 클래스의 모든 객체가 공유**합니다.

서로 다른 객체라도 static으로 선언된 변수는 그 클래스의 모든 인스턴스에서 공유되어, 어떤 객체가 그 값을 변경하면 다른 모든 객체에서도 그 변경된 값을 볼 수 있어요.

'static' 메소드도 클래스의 **특정 인스턴스가 아니라 클래스 자체에 속해 있기 때문에, 객체를 생성하지 않고도 호출**할 수 있습니다. 이런 특성 때문에 'static'은 클래스 수준에서 공유되는 변수나 메소드를 만들 때 유용하게 사용됩니다.

[기출예제] 21년 1회 실기 (난이도: ★★★)

다음은 JAVA 관한 문제이다. 알맞는 출력값을 작성하시오.

	[0][0]	[0][1]	[0][2]
[0][0]	45	50	75
[1][0]	89		

```
1 public class Good {
2     public static void main(String[] args) {
3         int[][]arr = new int[][]{{45,50,75},{89}};
4         System.out.println(arr[0].length);
5         System.out.println(arr[1].length);
6         System.out.println(arr[0][0]);
7         System.out.println(arr[0][1]);
8         System.out.println(arr[1][0]);
9     }
10 }
```

→ 이차원 배열을 생성합니다.

배열의 0번째에는 크기 3의 배열이 들어가 길이가 3이예요.

배열의 0번째에는 크기 1의 배열이 들어가 길이가 1이예요.

배열의 0번째의 0번째 값은 45예요.

배열의 0번째의 1번째 값은 50예요.

배열의 1번째의 0번째 값은 89예요.

첫 번째 배열에는 **괄호의 가장 밖 값**들이 들어가고, {45, 50, 75} | {89}
두 번째 배열부터는 그 값안에서의 배열이라고 생각하면 쉬워요.

그러므로 정답은 3, 1, 45, 50, 89의 값이 줄내림으로 출력돼요.

▶ 정답

[4행] 3

[5행] 1

[6행] 45

[7행] 50

[8행] 89

[기출예제] 22년 1회 실기 (난이도: ★★★)

다음 Java 코드 중에서 밑줄에 들어갈 알맞는 코드를 작성하시오.

▶ 정답 Car

Car 클래스

```
1 class Car implements Runnable {
2     int a;
3
4     public void run() {
5         system.out.println("message")
6     }
7 }
8
9 public class Main{
10     public static void main(String args[]) {
11         Thread t1 = new Thread(new ( ));
12         t1.start();
13     }
14 }
```

Thread 클래스에서 Runnable 인터페이스를 구현받은 객체를 받아서 run() 메소드를 비동기로 수행할 수 있게 돼요.
여기서는 Car 클래스의 [4행] run() 메소드가 [12행] start() 호출 시, 비동기로 수행돼요.
(비동기는 백그라운드에서 돈다고 생각하면 편해요. 기존 로직 흐름과는 별도로 동작해요.)

new Tread의 생성자 안에는 Runnable의 구현체가 들어가야해요.
그러므로, 정답은 Car예요!
(물론, 아무것도 없이 Thread를 생성하는 것도 가능은 해요! 지금은 그건 고려하지 않아도 돼요)

[11행] 위에서 별도로 동작시키고 싶은 thread를 생성했고, start() 메소드를 호출해야 Runnable의 run() 메소드가 새롭게 생긴 thread에서 동작해요.
(여기 코드와는 상관 없지만 저 t1.start() 메소드가 종료되길 기다리고 싶으면 t1.join()을 호출하면 돼요! 그럼 Runnable의 run()이 다 수행될 때까지 기다려요!)

★ key point

이 문제는 Thread 클래스를 써보지 않은 사람은 사실 알기 어려운 정보예요.

Thread 라는 클래스의 생성자 매개변수가 어떤 타입인지를 알고 있어야 하기 때문이죠. 문제에서는 Car 클래스 하나만 정의 되어 있으니,

이런 문제가 나온다면 유연하게 Car를 정답으로 쓰는 것도 방법이에요!



★ 어려울 수 있어요.

알고 보면, 쉬운데 컴파일 오류를 찾는 문제를 시험장에서 만나면 어려울 수 있어요.

컴파일 오류를 찾는 문제가 나오면, **먼저 static 메소드에 일반적인 클래스 변수를 사용하고 있는지 확인해주세요. Static 메소드에서는 static 변수만 사용가능해요. static이 없는 클래스 변수를 사용하고 있다면, 그 행이 오류 원인이에요.**

▶ 7번 라인

[기출 예제] 23년 3회 실기 (난이도: ★★★)

다음 자바 코드를 실행할 경우 예러가 발생이 된다. 예러가 발생하는 라인명을 작성하시오.

```

1  class Person {
2      private String name;
3      public Person(String val) {
4          name = val;
5      }
6      public static String get() {
7          return name;
8      }
9      public void print() {
10         System.out.println(name);
11     }
12 }
13
14 public class main {
15     public static void main(String[] args) {
16         Person obj = new Person("Kim");
17         obj.print();
18     }
19 }
  
```

Person 클래스

실제 실행코드

Person 클래스의 멤버 변수

Person 클래스의 static 메소드

name을 출력해요.

코드가 실행되기 전에 컴파일 오류가 발생합니다.

★ key point

static 메소드에서는 **static(정적) 변수만** 사용 가능해요.

(멤버 변수는 사용 불가능)

★ static

자바 프로그래밍에서 'static'을 사용하면, **그 요소는 특정 객체에 속하지 않고, 해당 클래스의 모든 객체가 공유**합니다.

서로 다른 객체라도 static으로 선언된 변수는 그 클래스의 모든 인스턴스에서 공유되어, 어떤 객체가 그 값을 변경하면 다른 모든 객체에서도 그 변경된 값을 볼 수 있어요.

'static' 메소드도 클래스의 특정 인스턴스가 아니라 클래스 자체에 속해 있기 때문에, **객체를 생성하지 않고도 호출**할 수 있습니다. 이런 특성 때문에 'static'은 클래스 수준에서 공유되는 변수나 메소드를 만들 때 유용하게 사용됩니다.

틀린 표현이지만, 이해가 쉽게 표현하자면 C언어의 포인터 처럼 모두 하나의 주소를 바라보는 것과 비슷해요.

Ex)

GV80에서 GV70으로
값이 변경돼요

```

class Car {
    static String name;
    Car(String n) { name = n; }
}
Car c1 = new Car("GV80");
Car c2 = new Car("GV70");
Car.name == c1.name == c2.name == GV70
  
```

★ 왜? static 메소드에서 멤버 변수는 사용 불가능 할까요?

암기가 아닌, 이해를 하고 싶으신 분들은 보세요!

오른쪽 설명을 먼저 읽어 보시고 읽으면 좋아요

static은 객체를 생성하지 않고도 호출할 수 있어요.

예를 들어, Person.get() 으로 호출하는 게 가능해요.

먼저, Person의 name은 언제 할당될까요?

Person a = Person("kim"); 이렇게 Person 객체를 생성될 때, name의 값이 할당 돼요.

- (1) static 메소드는 객체 생성 없이도 사용할 수 있다 했는데,
- (2) 그런 static 메소드 안에서 객체가 생성되어야만 값이 할당되는 멤버 변수(name과 같은)를 사용하려고 하면 당연히 오류가 날 수 밖에요!

위의 이유 말고도, Person a = Person("Kim"); 와 Person b = Person("Lee"); 이렇게 두개의 객체가 선언되고

static 메소드인 Person.get()을 호출하려고 했다고 가정해볼게요.

클래스에 속해 있는 get()은 어떤 객체의 name을 반환해줘야할까요?

(선택할 수 없죠)

이러한 이유들로 **static** 메소드에서는 멤버변수를 사용할 수 없어요

★ 멤버 변수

클래스 안에 선언된 변수를 의미해요.

이 변수들은 클래스의 **객체가 생성될 때마다 해당 객체와 연결**되어 있습니다.

▶ 정답

S

[기출 예제] 24년 2회 실기 (난이도: ★★★)

다음은 Java에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

```

1  class Main {
2      public static void main(String[] args) {
3          String str = "ITISTESTSTRING";
4          String[] result = str.split("T");
5          System.out.print(result[3]);
6      }
7  }
  
```

→ 'str'이라는 변수를 만들고, 그 안에 "ITISTESTSTRING"이라는 텍스트를 저장해요.

→ 'str'에 저장된 문자열을 'T'를 기준으로 나누고, 그 결과를 'result'라는 배열에 저장해요.

split 메서드는 문자열을 특정 문자를 기준으로 나누는 기능을 해요.

이 경우, "ITISTESTSTRING"을 'T'로 나누면 다음과 같이 돼요.

["I", "IS", "ES", "S", "RING"]

index 3은 4번째 위치한 S가 출력되게 됩니다.



[기출예제] 24년 3회 실기 (난이도: ★★★)

▶ 정답
101

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

실제 실행코드

```

1 public class Main {
2     public static void main(String[] args) {
3         int sum = 0;
4         try {
5             func();
6         } catch (NullPointerException e) {
7             sum = sum + 1;
8         } catch (Exception e) {
9             sum = sum + 10;
10        } finally {
11            sum = sum + 100;
12        }
13        System.out.print(sum);
14    }
15
16    static void func() throws Exception {
17        throw new NullPointerException();
18    }
19 }

```

sum 변수를 선언하고 0을 할당해요.
 NullPointerException을 던져요.
 NullPointerException이 던져지면, 해당 블록을 실행해요.
 func()에서 NullPointerException을 던졌기 때문에 sum + 1을 실행해요. (sum = 1)
 만약, NullPointerException에 대한 catch 문이 없었다면 해당 블록에 들어가겠지만,
 위에서 NullPointerException에 대한 catch 문이 존재하기 때문에 해당 블록은 무시돼요.
 finally는 try의 성공 실패 여부에 상관없이 무조건 거치는 블록이에요.
 그렇기 때문에 sum에 100을 추가로 더해줘요.
 101이 출력됩니다.

★ key point

예외 처리(Exception Handling)에 대한 이해를 확인하려고 해요.

특히, 예외가 발생했을 때 try, catch, finally 블록이 어떻게 동작하는지와, 예외 종류에 따라 적절한 처리가 이루어지는지를 알아보는 문제예요.

특정 예외 처리를 올바르게 했는지:

예외의 종류(NullPointerException, Exception)에 따라 적절한 catch 블록이 실행되는지.

finally의 동작:

예외 발생 여부와 상관없이 finally 블록이 항상 실행된다는 것을 아는지.

[기출예제] 24년 3회 실기 (난이도: ★★★)

▶ 정답
B0

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

실제 실행코드

```

1 class Main {
2     public static class Collection<T> {
3         T value;
4         public Collection(T t) {
5             value = t;
6         }
7         public void print() {
8             new Printer().print(value);
9         }
10        class Printer {
11            void print(Integer a) {
12                System.out.print("A" + a);
13            }
14            void print(Object a) {
15                System.out.print("B" + a);
16            }
17            void print(Number a) {
18                System.out.print("C" + a);
19            }
20        }
21    }
22    public static void main(String[] args) {
23        new Collection<>().print();
24    }
25 }

```

value라는 이름의 제네릭 타입(T)의 변수를 선언해요.
 객체를 만들 때 입력받은 값 t를 value에 저장해요.
 새로운 Printer 객체를 만들고, 그 객체의 print 메서드를 호출하면서 value를 넘겨요.
 value의 실제 타입(T)에 따라 Printer 클래스의 어떤 print 메서드가 호출될지가 결정돼요.
 0이 전달되었으므로 T는 Integer로 추론돼요.
 하지만, 중요한 점은 제네릭 타입 소거(type erasure)라는 개념이에요.
 컴파일 시점에 제네릭 타입 정보가 소거되면서, T는 실제로 Object로 변환돼요.
 그래서 value의 실제 런타임 타입은 Object가 되고,
 print 메소드를 호출할 때 Object 버전의 오버로드된 메소드가 선택되는 거예요.
 그래서, B0이 출력돼요.

★ key point

메서드 오버로딩(Method Overloading)과 동적 바인딩(dynamic binding)의 작동 방식을 이해하고 있는지 확인하려고 해요.

특히, 컴파일 시점과 실행 시점에서의 메서드 호출 결정이 어떻게 이루어지는지 알아보는 문제예요.

사실상, 틀리기 쉬운 꼬아서 낸 어려운 문제예요. 제네릭에 대한 깊은 이해도를 필요로 하는 문제예요.

이해하지 못하더라도, 비슷하게 나온다면 침착하게 풀면 맞출 수 있어요.

메서드 오버로딩의 동작 방식:

컴파일러는 메서드의 입력 타입에 따라 어떤 메서드를 호출할지 결정해요.

제네릭 타입의 한계:

제네릭 타입은 컴파일 시점에 타입 추론만 하고, 실행 시점에는 구체적인 타입 정보를 유지하지 않아요.

오버로딩된 메서드에서 우선순위:

컴파일러는 입력 타입에 가장 적합한 메서드를 선택하지만, 더 구체적인 타입이 아니라면 Object 메서드가 선택될 수 있다는 점을 이해해야 해요.



[기출예제] 25년 1회 실기 (난이도: ★★★)

▶ 정답
출력1출력5

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1 public class Main {
2
3     public static void main(String[] args) {
4
5         int a=5,b=0;
6
7         try{
8             System.out.print(a/b); → b가 0이므로 "0으로 나눌 수 없다"는 산술 예외(ArithmeticException)가 발생해요.
9         }catch(ArithmeticException e){ → 산술 예외가 일어났을 때 이 줄이 실행되어 "출력1"을 화면에 찍어요.
10             System.out.print("출력1");
11         }catch(ArrayIndexOutOfBoundsException e) { → 배열 인덱스를 잘못 사용했을 때 잡는 예외 블록이므로, 해당 블록에는 도착하지 않아요.
12             System.out.print("출력2");
13         }catch(NumberFormatException e) { → 문자열을 숫자로 바꿀 때 형식이 맞지 않으면 잡는 예외 예외 블록이므로, 해당 블록에는 도착하지 않아요.
14             System.out.print("출력3");
15         }catch(Exception e){ → 위 세 가지 예외 외에 모든 예외를 포괄적으로 잡아내는 마지막 블록이에요. [9행] 예외에 걸렸기에 해당 블록에 도착하지 않아요.
16             System.out.print("출력4");
17         }finally{ → 예외 발생 여부와 상관없이 반드시 실행되는 블록이에요.
18             System.out.print("출력5"); → "출력5"를 찍습니다.
19         }
20     }
21 }
```

try가 성공하거나, catch가 실행된 뒤에도 항상 실행돼요.
두 출력을 이어서 출력되어 출력1출력5 가 출력돼요.

★ key point

예외(Exception) 문제는 자주 출제되지 않는 만큼 개념적인 부분이 출제되었습니다.

예외의 흐름과 개념 정도만 알아두시면 좋습니다.

- 0으로 나누기: ArithmeticException 발생
- catch 블록은 위에서부터 순서대로 실행돼요.
- 마지막 catch(Exception e)로 나머지 예외를 한꺼번에 처리할 수 있어요.
- finally 블록은 예외 발생 여부와 관계없이 항상 실행돼요.



[문제 유형] 연산자

[기출 예제] 23년 2회 실기 (난이도: ★★★)

다음은 JAVA 코드 문제이다.

가지고 있는 돈이 총 4620원일 경우

1000원, 500원, 100원, 10원의 지폐 및 동전을 이용하여 보기의 조건에 맞춰 최소한의 코드를 통해 팔호안을 작성하시오.

[보기]

아래 주어진 항목들을 갖고 팔호안의 코드를 작성

변수: m

연산자: /, %

팔호: [,], (,)

정수: 1000, 500, 100, 10

실제 실행코드

```

1 public class Problem{
2     public static void main(String[] args){
3         m = 4620;
4
5         a = (          );
6         b = (          );
7         c = (          );
8         d = (          );
9
10        System.out.println(a); // 천원짜리 4장 출력
11        System.out.println(b); // 오백원짜리 1개 출력
12        System.out.println(c); // 백원짜리 1개 출력
13        System.out.println(d); // 십원짜리 2개 출력
14    }
15 }
16

```

★ 연산자

/ 연산자 (나눗셈 연산자): 이 연산자는 두 숫자를 나누는 데 사용됩니다. 예를 들어, 10 / 3은 10을 3으로 나눈 값, 즉 3.3333... (소수점 이하의 숫자가 있을 경우) 또는 정수 나눗셈인 경우 3 (소수점 이하를 버림)의 결과를 줍니다.

% 연산자 (나머지 연산자): 이 연산자는 두 숫자를 나눈 후 남는 '나머지'를 구하는 데 사용됩니다. 예를 들어, 10 % 3은 10을 3으로 나눈 후 남는 나머지인 1을 결과로 줍니다.

4620원을 1000원으로 가질 수 있는 **최대한의 수**를 구해야 해요.

4620원이 1000원으로 나누면 **4** 몫과 **620**의 나머지가 나와요.

몫만 필요하고, 나머지는 필요 없으니,

4620 / 1000으로 표현할 수 있어요.

즉, a에 들어갈 값은 **m / 1000** 이예요

위의 계산으로 1000원짜리로 4장 가지고 있고,

나머지 금액(620원) 중 500원 동전으로 최대 몇 개 가지고 있을 수 있는지 계산하는 식을 구하면 돼요.

(4620원을 1000원으로 나누고 남은 나머지)를 500원으로 나눈 몫을 구해야해요.

(m % 1000) / 500 으로 표현할 수 있어요.

위의 계산으로 1000원 4장, 500원 하나를 가지고 있고,

남은 나머지(120원) 중 100원으로 최대 몇 개 가지고 있을 수 있는지 계산하는 식을 구하면 돼요.

최소한의 코드로 작성하러했으니,

(4620원을 500원으로 나누고 남은 나머지)를 100원으로 나눈 몫으로 표현할 수 있어요.

(m % 500) / 100 으로 표현할 수 있어요.

위의 계산으로 1000원 4장, 500원 하나, 100원 하나를 가지고

있고, 나머지 금액(20원) 중 10원으로 최대 몇 개 가지고 있을 수 있는지 계산하는 식을 구하면 돼요.

최소한의 코드로 작성하러했으니,

(4620원을 100원으로 나누고 남은 나머지)를 10원으로 나눈 몫으로 표현할 수 있어요.

(m % 100) / 10 으로 표현됩니다.

[기출 예제] 22년 1회 실기 (난이도: ★★★)

아래 자바 코드에서 출력되는 값을 작성하시오.

실제 실행코드

```

1 class A {
2     int a;
3     int b;
4 }
5
6 public class Main {
7
8     static void func1(A m){
9         m.a *= 10;
10    }
11
12    static void func2(A m){
13        m.a += m.b;
14    }
15
16    public static void main(String args[]){
17        A m = new A();
18
19        m.a = 100;
20        func1(m);
21        m.b = m.a;
22        func2(m);
23
24        System.out.printf("%d", m.a);
25    }
26 }

```

클래스 A

클래스 A의 a 멤버 변수에 10을 곱해줍니다.
[20행]에서 호출하여, m.a에 m.a * 10을 곱하여 1000이 됩니다.

클래스 A의 a 멤버 변수에 m.b를 더해줍니다.
[22행]에서 호출하여, m.a에 m.a + m.b를 하여 2000이 됩니다.

변수 m에 A 클래스를 선언 및 할당합니다.

m.a에 100을 할당해요.

[9행]을 수행해요. m.a가 1000이 되었어요.

m.b에 1000을 할당해요.

[13행]을 수행해요. m.a가 2000이 되었어요.

%d는 숫자를 치환해줘요.
m.a가 출력되서 정답은 2000이예요.

★ 할당 연산자

- 더하기 할당 연산자

변수의 현재 값에 어떤 값을 더한 후, 그 결과를 다시 변수에 할당해요.
변수 = 변수 + 값 (ex. a = a + 1) 이라면,
변수 += 값 (ex. a += 1)로 표현할 수 있어요.

- 곱하기 할당 연산자

변수의 현재 값에 어떤 값을 곱한 후, 그 결과를 다시 변수에 할당해요.
변수 = 변수 * 값 (ex. a = a * 1) 이라면,
변수 *= 값 (ex. a *= 1)로 표현할 수 있어요.

▶ 정답 2000



▶ 정답

1. $n > 0, n \geq 1, i < 8, i \leq 7$ (넷중 아무거나 써도 정답이에요)
2. $n \% 2$

[기출예제] 20년 4회 실기 (난이도: ★★☆☆)

다음은 n 이 10일 때, 10을 2진수로 변환하는 자바 소스 코드이다. 1,2에 알맞는 값을 적으시오.

[출력결과]: 00001010

```

1 public class Main {
2     public static void main(String[] args) {
3         int[] a = new int[8];
4         int i=0; int n=10;
5         while ( 1 ) {
6             a[i++] = ( 2. );
7             n /= 2;
8         }
9
10        for(i=7; i>=0; i--) {
11            System.out.print(a[i]);
12        }
13    }
14 }
```

실제 실행코드

8 크기의 빈 배열을 생성했어요.

코드의 흐름대로 천천히 따라가면 어렵지 않아요.

[6행]에서 i 가 0부터 +1씩 증가하면서 값을 채우고 있는데,
[5행]에서 while문의 조건식을 어떻게 해야 [3행]에서 선언한 8번만 반복할 수 있을까요?

여러가지 방법이 있지만, 간단하게 1번의 정답은 $i < 8$ 이나 $i \leq 7$ 을 적을 수 있어요.
 i 가 8 미만 (7보다 같거나 작음) 까지만 동작하도록 1번 정답을 적으면 돼요.

그 다음으로는 먼저, [10행]을 살펴봐요.
[10행]의 출력이 거꾸로 찍히고 있어요.
[4행]의 i 가 0부터 배열을 채워넣고 있으니
우리는 거꾸로해서 01010000의 순서로 [6행] 2번 정답 코드에서 할당되어야 해요.

[7행]의 $n /= 2$ 는 $n = n / 2$ 라고 표현할 수 있어요.
이는 n 을 계속 2로 나눈 몫을 덮어쓰는 거예요.
여기서는 $10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 0 \rightarrow \dots$ 이렇게 진행 될거예요.

근데, n 의 값을 살펴보니 2번째와 4번째만 홀수고 나머지는 전부 짝수네요?
만약에 짝수는 0 홀수는 1로 값을 a 에 할당 할 수만 있다면
01010000으로 배열이 채워 질거예요.

짝수는 0 홀수는 1로 표현되는 2번의 정답은 $n \% 2$ 이 들어가야해요.
 $\%$ 는 나누고 남은 나머지를 반환하거든요
예를 들어,
 $10 \% 2$ 의 나머지는 0
 $5 \% 2$ 의 나머지는 1
 $2 \% 2$ 의 나머지는 0
이런식으로요.

예를 들어, while (count < 5)는 count가 5보다 작은 동안에는
루프 안의 코드를 계속 실행한다는 뜻이에요.

★ /= 연산자

나눗셈 후 대입 연산자예요.
변수의 값을 어떤 수로 나눈 후,
그 결과를 다시 같은 변수에 대입해요.

예를 들어, $a /= 3$ 은 a 의 값을 3으로 나눈 후, 그 결과를 다시 a 에 저장한다는 뜻이에요.
만약 a 가 10이었다면, $a /= 3$ 실행 후에는 a 의 값이 3이 돼요.
($10 / 3$ 의 몫은 3)

★ %= 연산자

$\%$ 는 나머지 연산 후 대입 연산자예요.
변수의 값을 어떤 수로 나눈 후,
그 나머지를 다시 같은 변수에 대입해요.

예를 들어, $a \% 3$ 은 a 를 3으로 나눈 나머지를 구한 후, 그 결과를 다시 a 에 저장한다는 뜻이에요.
만약 a 가 10이었다면, $a \% 3$ 실행 후에는 a 의 값이 1이 돼요. (10을 3으로 나눈 나머지는 1)

★ i++ (후위 증가 연산자)

현재 i 의 값을 먼저 사용한 후에, i 의 값을 1 증가시켜요.
예를 들어, $int j = i++$;라고 하면, 먼저 j 에 i 의 현재 값이 할당되고, 그 다음에 i 의 값이 1 증가해요.

따라서 i 가 초기에 5였다면,
 j 에는 5가 할당되고, i 는 6이 되는 거예요.



[기출 예제] 21년 3회 실기 (난이도: ★★★)

▶ 정답 7

다음 Java 코드에 대한 알맞는 출력값을 쓰시오.

```

1 public class testco {
2     public static void main(String[] args) {
3         int a = 3, b = 4, c = 3, d = 5;
4         if ((a == 2 | a == c) & !(c > d) & (1 == b ^ c != d)) {
5             a = b + c;
6             if (7 == b ^ c != a) {
7                 System.out.println(a);
8             } else {
9                 System.out.println(b);
10            }
11        } else {
12            a = c + d;
13            if (7 == c ^ d != a) {
14                System.out.println(a);
15            } else {
16                System.out.println(d);
17            }
18        }
19    }
20 }

```

변수 a, b, c, d에 각각 3, 4, 3, 5라는 초기값을 할당했어요.

이 if 문은 여러 개의 조건을 **비트 연산자**로 연결하고 있어요. 어려우니 하나씩 하나씩 넘어가 볼게요.

(a == 2 | a == c)
a가 2이거나 a가 c와 같은지 확인해요. 여기서는 **| (OR)** 연산자를 사용했어요. a와 c가 같으니 참이에요.

!(c > d)
c가 d보다 크기 않은지 확인해요. **!(NOT)** 연산자를 사용했어요. c가 d보다 크기 않아 참이에요.

(1 == b ^ c != d)
1이 b와 같은지, 그리고 c가 d와 다른지 확인해요. 여기서는 **^ (XOR)** 연산자를 사용했어요. **1==b의 결과와 c!=d의 결과가 달라야 참**이에요. false ^ true로 두 결과가 다르니 참이에요.

모든 조건을 만족해 안으로 들어가요.

a에 b와 c를 더한 값을 할당해요. a는 **7**이 되었어요.

그리고 또 다른 if 문이 나와요. 이번에는 **7 == b ^ c != a**라는 조건을 확인해요. 만약 이 조건이 참이면 a 값을 출력하고, 거짓이면 b 값을 출력해요. **7 == b(5) 이므로 거짓, c(3) != a(7) 이므로 참**이에요. 둘의 결과가 달라 참을 반환해요.

[4행] [6행]이 참이므로 **정답은 7**이 돼요.

만약 처음 if 문의 조건을 만족하지 않으면 else 문으로 들어가요. a에 c와 d를 더한 값을 할당해요.

또 if 문이 나와요. 이번에는 **7 == c ^ d != a**라는 조건을 확인해요. **7 == c와 d != a의 결과값이 달라야 참**같은 거짓이에요. 여기서는 해당 조건문을 타지 않아요.

★ 비트 연산자

이 코드에서 비트 연산자가 조금 어려울 수 있어요.

| (OR)는 두 조건 중 하나라도 참이면 참이 되는 연산자예요.

^ (XOR)는 두 조건이 서로 다를 때만 참이 되는 연산자예요.

!(NOT)은 조건의 반대를 나타내는 연산자예요.

이해하기 어려울 수 있지만, 비트 연산자를 사용하면 여러 조건을 효율적으로 확인할 수 있어요.

[기출 예제] 20년 4회 실기 (난이도: ★★★)

다음은 자바 소스 코드이다. 출력 결과를 보고, 1,2에 알맞는 값을 적으시오.

▶ 정답

1) 3
2) 5

[출력 결과]

1 4 7 10 13

2 5 8 11 14

3 6 9 12 15

```

1 public class Main {
2     public static void main(String[] args) {
3         int[][] a = new int[(1.)][(2.)];
4         for(int i = 0; i < 3; i++) {
5             for(int j = 0; j < 5; j++) {
6                 a[i][j] = i*3 + (i+1);
7                 System.out.print(a[i][j] + " ");
8             }
9             System.out.println();
10        }
11    }
12 }

```

실제 실행코드

사실 코드를 보면 어려워 보이지만, 바로 정답을 찾을 수 있는 요소들이 있어요.

[6행]을 계산하려고 보면 어렵겠지만, **[6행]**을 볼 필요가 없어요.

아래의 생각의 흐름을 따라 정답을 유추해볼게요.

출력 결과를 보면 **2차원 배열**처럼 보이네요.

행이 3개 열이 5개 인 것 만으로 사실 **1번 정답은 3, 2번 정답은 5**가 바로 나와요.

근데, 3, 5 인지 5, 3인지 헷갈릴 수 있어요.

그럴 때는 **for문**을 살펴보면 돼요.

[4행] for문의 i는 3보다 작을 때까지 반복해요.

근데, **그가 a[i][j] 왼쪽에 들어가 있는 걸 확인** 할 수 있어요.

[5행] for문의 j는 5보다 작을 때까지 반복하는데 두 번째 배열에 j가 들어가 있죠.

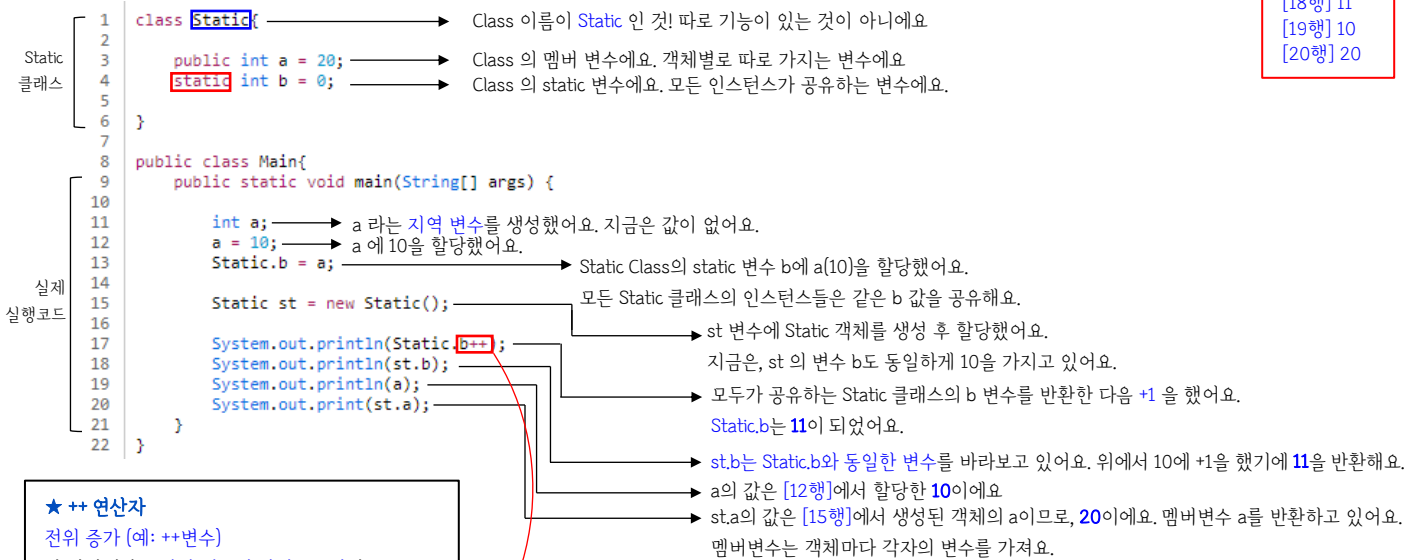
이런 두가지 조건들만 가지고도 빠르게 정답을 유추하여 정답을 적을 수 있어요.



[가출예제] 23년1회 실기 (난이도: ★★★)

아래 자바 코드에서 출력되는 값을 작성하십시오.

▶ 정답
[17행] 10
[18행] 11
[19행] 10
[20행] 20



★ ++ 연산자

전위 증가 (예: ++변수)

이 방식에서는 먼저 변수의 값이 1 증가하고, 그 후에 이 값이 사용됩니다.

예를 들어, int a = 5; int b = ++a;라고 하면, a는 6이 되고, b도 6이 됩니다.

후위 증가 (예: 변수++)

이 방식에서는 변수의 현재 값이 먼저 사용되고, 그 후에 변수의 값이 1 증가합니다.

예를 들어, int a = 5; int b = a++;라고 하면, b에는 a의 원래 값인 5가 저장되고, 그 후에 a는 6이 됩니다.

★ 지역변수

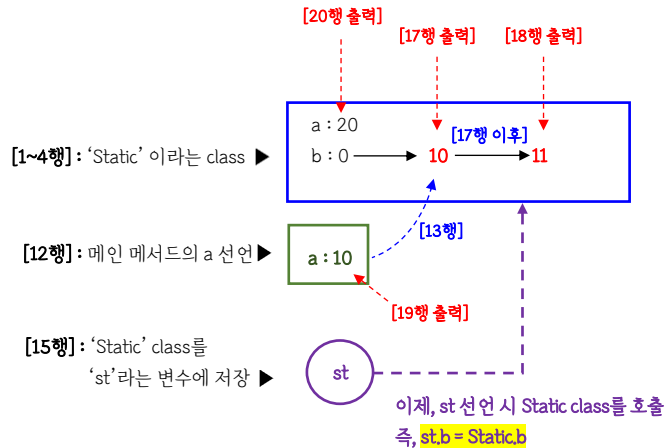
지역변수는 현재 진행되는 메소드 안에서만 생성된 변수를 말해요.

지역변수는 메소드 안에서만 살아있고, 메소드가 종료되면 사라져요.

지역변수는 해당 메소드 외부에서는 접근(읽고, 쓰기) 할 수 없어요.

동일한 이름의 변수가 멤버 변수도 선언되어 있을 때, 지역 변수를 사용해요.
(이때, 멤버 변수는 shadowed 되었다고 해요.)

★ 아래 도식표에서 행 순서대로 따라가보세요!





[문제 유형] 재귀 함수

[기출예제] 24년 2회 실기 (난이도: ★★★)

다음은 Java에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

잠깐! calculFn 의 [11행]을 보고 재귀 함수인 것을 먼저 캐치해야 해요!

▶ 정답
dcba

```

1 class Main {
2     public static void main(String[] args) {
3         String str = "abacabcd"; → "abacabcd"라는 문자열을 'str'이라는 변수에 저장합니다.
4         boolean[] seen = new boolean[256]; → 256개의 false 값으로 초기화된 배열을 만들어요. 각 문자가 이미 처리되었는지 여부를 추적하는 데 사용해요.
5         System.out.print(calculFn(str, str.length()-1, seen)); → 'calculFn' 함수를 호출하고 그 결과를 화면에 출력해요.
6     }
7
8     public static String calculFn(String str, int index, boolean[] seen) {
9         if(index < 0) return ""; → 인덱스가 0보다 작으면 빈 문자열을 반환해요.
10        char c = str.charAt(index); → 인덱스의 문자를 가져와요.
11        String result = calculFn(str, index-1, seen); → 인덱스를 하나 줄여서 다시 호출해요. (재귀 호출)
12        if(!seen[c]) { → 현재 문자가 아직 처리되지 않았다면,
13            seen[c] = true; → 처리된 것으로 표시
14            return c + result; → 현재 문자를 결과 문자열의 앞에 추가해요.
15        }
16        return result; → 현재 문자가 이미 처리되었다면, 결과를 그대로 반환해요.
17    }
18 }

```

이 함수에 문자열, 문자열의 마지막 인덱스, 그리고 'seen' 배열을 전달해요.

즉, 이 메소드는 주어진 문자열에서 중복된 문자를 제거하고, 마지막으로 등장한 순서대로 문자들을 나열해요.
abacabcd 문자열을 뒤에서부터 중복 제거하면 dcba가 출력돼요.

★ 재귀 함수

재귀 함수란 자기 자신을 호출하는 함수예요.

종료 조건을 만나기 전까지 계속 자기 자신을 호출하면서 종료 조건을 만나면 값을 반환해요.

큰 문제를 같은 형태의 작은 문제로 나누어 해결하는 방식이에요.

▶ 정답 20

[기출예제] 25년 1회 실기 (난이도: ★★★)

다음은 Java에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

```

1 public class Main {
2
3     public static void main(String[] args) {
4         int[] nums = {3, 5, 8, 12, 17};
5         System.out.println(maxSum(nums, 0, nums.length - 1));
6     }
7
8     static int maxSum(int[] nums, int start, int end) {
9         if (start >= end) return 0; → 종료 조건 (더 이상 쪼갤 수 없음)
10        int mid = (start + end) / 2; → 시작과 끝의 중간을 구함
11        return nums[mid] + Math.max(maxSum(nums, start, mid), maxSum(nums, mid + 1, end));
12    }
13 }

```

[11행]에서 이 함수는 중간 값 + max(왼쪽 배열, 오른쪽 배열) 값을 구하고자 함을 우리는 알았어요.

이렇게 배열이 작은 경우 값을 대입해보는 게 쉽게 문제 풀 수 있어요.

함수의 의미 자체는 배열을 반으로 나눌 때마다 그 구간의 중간(피벗) 값을 더해 가면서,
왼쪽과 오른쪽 중 '더 큰 쪽'을 선택해 내려갔을 때 얻을 수 있는 합의 최댓값을 구하자하는 문제예요.
(대입해서 푸는 게 맞기 때문에 함수의 목적을 알 필요는 없어요)

n[0]	n[1]	n[2]	n[3]	n[4]
3	5	8	12	17

종료 조건인 start와 end의 값이 같을 때까지 식을 정리해보면 돼요.

$\text{maxSum}(0,4) = n[2] + \max(\text{maxSum}(0,2), \text{maxSum}(3,4))$
 $\text{maxSum}(0,2) = n[1] + \max(\text{maxSum}(0,1), \text{maxSum}(2,2))$
 $\text{maxSum}(0,1) = n[0] + \max(\text{maxSum}(0,0), \text{maxSum}(1,1))$

좌측 식이 모두 정리되었어요. start, end가 같은 걸 0으로 치환해서 계산해보면 다음과 같아요.

$\text{maxSum}(0,1) = 3 + \max(0,0)$
 $\text{maxSum}(0,2) = 5 + \max(3,0)$
 $\text{maxSum}(0,4) = 8 + \max(5, \text{maxSum}(3,4))$

이제 maxSum(3,4)만 구해서 계산하면 돼요.

$\text{maxSum}(3,4) = n[3] + \max(\text{maxSum}(3,3), \text{maxSum}(4,4))$
 $\text{maxSum}(3,4) = 12 + \max(0,0) = 12$

즉, $\text{maxSum}(0,4) = 8 + \max(5, 12) = 8 + 12 = 20$ 이예요.



★ key point

오버로딩과 재귀에 관련된 문제입니다.

오버로딩은 메소드의 매개변수의 타입이나 개수가 다르면 서로 다른 메소드로 취급합니다.

▶ 정답

4

[기출예제] 25년 1회 실기 (난이도: ★★★)

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(calc("5"));
4     }
5
6     static int calc(int value) {
7         if (value <= 1) return value; → 1 이하가 되면 그 값을 반환합니다. 재귀 함수의 종료 조건입니다.
8         return calc(value - 1) + calc(value - 2);
9     }
10
11    static int calc(String str) {
12        int value = Integer.valueOf(str); → 문자열을 정수로 변환합니다. 문자열 5가 숫자 5가 됩니다.
13        if (value <= 1) return value; → 1 이하가 되면 그 값을 반환합니다. 재귀 함수의 종료 조건입니다.
14        return calc(value - 1) + calc(value - 3); → 이제 정수 매개변수인 calc이 호출됩니다.
15    }
16 }

```

$\text{calc}("5") = \text{calc}(4) + \text{calc}(2)$
 $\text{calc}(4) = \text{calc}(3) + \text{calc}(2)$
 $\text{calc}(3) = \text{calc}(2) + \text{calc}(1)$
 $\text{calc}(2) = \text{calc}(1) + \text{calc}(0)$
 $\text{calc}(1) = 1$
 $\text{calc}(0) = 0$
 $\text{calc}(2) = 1 + 0 = 1$
 $\text{calc}(3) = 1 + 1 = 2$
 $\text{calc}(4) = 2 + 1 = 3$
 $\text{calc}("5") = 3 + 1 = 4$

먼저, 하나씩 분해하고
 분해된 값들을 종료조건에 도달하여 값이 나오면
 그걸 역순으로 계산하면 정답이 나와요!

정답은 4가 출력돼요!



[문제 유형] 함수 호출

▶ 정답 234

[기출 예제] 20년 3회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

★ 가장 기본이 되는 함수 호출 문제예요.

20년도에 딱 한번만 출제되고, 더 이상 출제되지 않고 있는 문제예요.

```

1  #include <stdio.h>
2
3  int r1() {
4      return 4;
5  }
6
7  int r10() {
8      return (30+r1()); 30 + 4 = 34
9  }
10
11 int r100() {
12     return (200+r10());
13 }
14
15 int main() {
16     printf("%d\n", r100());
17     return 0;
18 }

```

[16행]을 호출하면, 200 + [7행] → [8행] 30 + [3행] → [4행] 4 반환
호출된 순서대로 반환하다보면 4 + 30 + 200 으로 234가 반환되어
정답은 234 예요.



[문제 유형] 완전수

★ 완전수

완전수는 자기 자신을 제외한 모든 양의 약수를 더하면 자기 자신이 되는 수를 말해요.
예를 들어, 6은 완전수인데, 1, 2, 3은 6의 약수이고 이들의 합 ($1 + 2 + 3$)은 6과 같거든요.
완전수는 100이하에서 6과 28만 존재해요! 암기하면 정말 좋아요.

[기출예제] 23년 3회 실기 (난이도: ★★★)

▶ 정답 34

다음 C 프로그램 출력 결과 구하기

```

1  #include <stdio.h>
2
3  int test(int n) {
4      int i, sum = 0;
5
6      for (i = 1; i <= n / 2; i++){
7          if (n % i == 0)
8              sum += i;
9      }
10
11     if (n == sum)
12         return 1;
13     return 0;
14 }
15
16 int main(){
17     int i, sum=0;
18
19     for (i = 2; i <= 100; i++){
20         if (test(i))
21             sum += i;
22     }
23
24     printf("%d ", sum);
25     return 0;
26 }

```

정수 n을 받아서, n이 완전수인지 아닌지를 판별해요.
완전수는 자기 자신을 제외한 약수의 합이 자기 자신과 같은 수를 말해요.

for 루프를 사용해서 1부터 n/2까지의 수 i에 대해 반복해요.
만약 n을 i로 나눈 나머지가 0이면 (즉, i가 n의 약수이면), sum에 i를 더해요.

루프가 끝나고 나서, n과 sum이 같으면 1(true)을 반환하고,
그렇지 않으면 0 (false)을 반환해요.

for 루프를 사용해서 2부터 100까지의 수 i에 대해 반복해요.
각 i에 대해 test(i)를 호출해서, i가 완전수인지 확인해요.
만약 test(i)가 참 (즉, 1)을 반환하면, sum에 i를 더해요.
루프가 끝나고 나서, printf 함수를 사용해서 sum을 출력해요.

2~100사이의 완전수는
6($1+2+3$)과 28($1+2+4+7+14$) 두개예요.
그래서 정답은 6과 28의 합인 34예요.

★ for구문

for 문은 일종의 반복적인 작업을 수행하기 위한 지시문이에요. for 문은 크게 세 부분으로 나누어 생각할 수 있습니다. 초기화(선언 및 할당), 조건, 그리고 반복 후 작업입니다. 예를 들어, 우리가 1부터 10까지 숫자를 차례대로 출력하고 싶다면, for 문을 사용하여 이 작업을 할 수 있습니다. for 문은 다음과 같은 형식이에요.

```

for (초기화; 조건; 반복 후 작업) {
    // 반복해서 실행할 명령들
}

```

예를 들어, 1부터 10까지 출력하는 for 문은 다음과 같이 작성돼요.

```

for (int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}

```

이 코드는 i가 1부터 시작해서 10이 될 때까지 i의 값을 출력하며, 각 반복마다 i를 1씩 증가시켜요.

★ [6행] for 문 n=6 일 때

이 부분에서 i는 1에서 시작하여 $n / 2$ ($6 / 2 = 3$) 이하일 때까지 반복합니다. 즉, i는 1, 2, 3의 값을 갖게 돼요.

왜 n/2까지인가? 숫자의 약수는 그 숫자의 절반을 넘을 수 없기 때문입니다. 예를 들어, 6의 경우 3보다 큰 수는 6의 약수가 될 수 없습니다.

반복문 내부에 if ($n \% i == 0$) 조건문이 있습니다. 이는 n을 i로 나눈 나머지가 0인지 확인하여 i가 n의 약수인지를 판단해요.

N이 6일 때, i가 1, 2, 3일 경우에 이 조건은 참이 돼요. 즉, 이들은 모두 6의 약수입니다. 참일 경우, $sum += i$;를 통해 약수 i의 값을 sum에 더해해요. 이렇게 해서 1, 2, 3을 sum에 차례로 더해해요.

For 문이 종료된 후, if ($n == sum$) 조건문을 통해 n이 sum과 같은지 확인합니다. 이 경우, $sum = 1 + 2 + 3 = 6$ 이 되므로, n이 6이라면 이 조건은 참이 돼요.

조건이 참이면, 함수는 return 1;을 실행하여 1을 반환해요. 이는 n이 완전수임을 의미해요.

★ key point

손으로 하나 하나 계산하는 건 비효율적이에요.

코드에서 원하는게 완전수의 합인 걸 캐치했다면, 완전수를 찾는 최적화된 과정을 진행해야해요.

[사실, 제일 쉬운 방법은 완전수를 외우는 거예요. 문제에서는 6, 28 그리고 다음 완전수는 496 이거든요. 완전수 문제에서는 100 이하로만 나올 확률이 높아요]

완전수는 짝수예요.

왜냐하면 1을 제외한 모든 완전수의 약수는 짝수-홀수 쌍으로 이루어져 있거든요.

따라서 2부터 시작해서 짝수만 체크하면 돼요.

어떤 수의 약수를 찾을 때, 그 수의 제곱근까지만 확인하면 돼요.

왜냐하면 어떤 수의 약수들은 그 수의 제곱근을 기준으로 대칭을 이루기 때문이에요.

예를 들어, 36의 약수는 1, 2, 3, 4, 6, 9, 12, 18, 36이에요. 36의 제곱근은 6이에요.

1, 2, 3은 6보다 작고, 36, 18, 12는 6보다 커요. 6은 6과 같고요.

따라서 6보다 작거나 같은 약수들만 확인하고, 나머지 약수들은 36을 그 약수들로 나누면 구할 수 있어요.



[기출예제] 22년 3회 실기 (난이도: ★★☆☆)

▶ 정답 2

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```

1  #include <stdio.h>
2
3  void main() {
4      int n;
5      int k;
6      int s;
7      int e1 = 0;
8
9      for(n=6; n<=30; n++){
10         s=0;
11         k=n/2;
12         for(int j=1; j<=k; j++){
13             if(n%j==0){
14                 s=s+j;
15             }
16         }
17
18         if(s==n){
19             e1++;
20         }
21     }
22
23     printf("%d", e1);
24 }
```

이 문제는 6부터 30까지의 정수 중에서 완전수(perfect number)의 개수를 찾는 코드예요.
(우리는 완전수가 6, 28인걸 알고있으니 정답은 2인걸 바로 알 수 있어요)

앞에서 이야기 했듯, 완전수는 100 이하 수에는 6, 28 밖에 없고
굳이 더 적어보자면 6, 28, 496, 8128, 33550336 ... 이렇게 있어요.

완전수 문제가 나왔을 때, 6과 28 정도는 외워두면 정말 크게 시간을 단축할 수 있으니
외워두는 걸 추천해요!!

그럼, 코드로 이게 완전수 문제인지는 어떻게 알 수 있을까요?

[13행] [14행] [18행]이 중요해요.

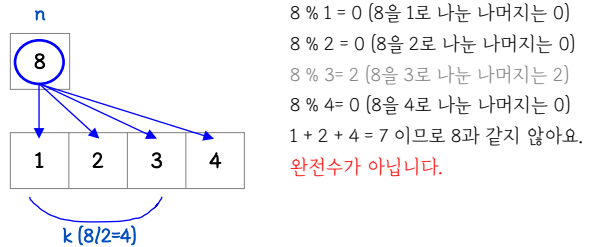
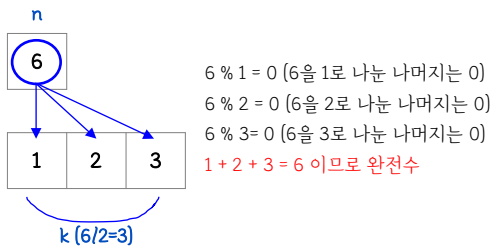
[13행]은 n을 나눴을 때 나머지가 0인 값들에 대해서, 즉 n의 약수에 대해서

[14행]은 s에 전부 더하고,

[18행]은 그 값이 n과 같을 때 를 의미해요.

위 조건들만 체크하더라도, 손으로 계산을 하나 하나 하지 않아도 정답을 알 수 있어요.
여기서는 6~30 사이의 완전수의 개수를 묻는 문제예요.
6과 28인 2가 출력돼요.

★ 완전수를 손으로 계산하고 싶으면 다음과 같이 계산해볼 수 있어요.





[문제 유형] 단순 for문

▶ 정답 BCD

[기출 예제] 23년 2회 실기 (난이도: ★★★)

다음 C 프로그램 출력 결과 구하기

```

1  #include <stdio.h>
2
3  void main() {
4      int n[3] = {73, 95, 82};
5      int i, sum = 0;
6
7      for (i = 0; i < 3; i++) {
8          sum += n[i];
9      }
10
11     switch (sum / 30) {
12         case 10:
13         case 9:
14             printf("A");
15         case 8:
16             printf("B");
17         case 7:
18         case 6:
19             printf("C");
20         default:
21             printf("D");
22     }
23 }

```

n[0] n[1] n[2]
 73 95 82

배열 선언

i가 3보다 작으면 아래 Sum += n[i] 실행
 이후 i++ 실행 (i를 1씩 증가)

i	0	1	2	3
sum	sum+n[0] = 0 + 73 = 73	73 + n[1] = 73 + 95 = 168	168 + n[2] = 168 + 82 = 250	for문 종료

Sum/30 = 250을 30으로 나눴을 때 몫 = 8
 'Case 8'에 해당되는 코드 실행 = "B" 출력
 이후 Break 없기 때문에 마지막 default까지 실행
 최종 'BCD' 출력 (정답)

★ Switch case 구문

switch(조건식) {
 case 변수1: 실행 코드 1
 break;
 case 변수2: 실행 코드 2
 break;
 ...
 case 변수n: 실행 코드 n
 break;
 default:
 }

조건식 값과 case 변수값이 일치 시 해당 case 코드 실행
 (ex. 조건식 = 변수2 → 실행코드 2 실행)

break : 해당 Switch 구문 종료!
 break 없을 시 아래 코드로 넘어가 계속 실행

default : 모든 case 변수에 해당 조건이 없을 시
 실행하는 코드

[기출 예제] 21년 2회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 주어진 밑(base)과 지수(exp)를 사용하여 거듭제곱을 계산하는 코드예요
 (단, 출력문의 출력 서식을 준수하시오.)

```

1  #include <stdio.h>
2
3  int mp(int base, int exp) {
4      int res = 1;
5      for(int i=0; i < exp; i++) {
6          res *= base;
7      }
8
9      return res;
10 }
11
12 int main() {
13     int res;
14     res = mp(2,10);
15     printf("%d",res);
16     return 0;
17 }

```

두 개의 정수 인자, base와 exp를 받아요. 여기서 base는 밑이고, exp는 지수예요.

res라는 변수를 1로 할당하고,

for 반복문을 사용해 base를 exp번 만큼 곱해요. res = res * base와 같은 표현이에요.

거듭제곱 값인 res를 반환해요.

2의 10제곱을 계산해요.

2를 10번 곱하는 거예요.

즉, $2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 1024$ 를 계산하는 거죠.
 정답은 1024가 돼요.

▶ 정답 1024



▶ 정답 505

[기출 예제] 23년 2회 실기 (난이도: ★★☆☆)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```

1  #include <stdio.h>
2
3  int main(){
4      int c = 0; 'c'라는 이름의 변수를 만들고, 0으로 할당하고 있어요.
5
6      for(int i = 1; i <= 2023; i++) { 'i'라는 변수를 1부터 시작해서 2023까지 1씩 증가시키면서 반복하고 있어요.
7          if(i%4 == 0) c++; 모듈러 연산자를 이용해, 'i'를 4로 나눈 나머지를 구해요.
8              만약 그 나머지가 0이면, 'c'의 값을 1 증가시키는 거예요.
9          printf("%d", c); 즉, 'i'가 4의 배수일 때마다 'c'의 값을 1씩 증가시키는 것이죠.
10     } 즉, 코드의 핵심 목적은 1부터 2023까지의 수 중에서 4의 배수가 몇 개인지 세는 것입니다.

```

1~2023 중, 4의 배수는 2023를 4로 나누면 나와요.
 $2023/4 = 505$ 이 나와요.
 정답은 505를 출력해요.

★ 모듈로(modulo) 연산자

% 연산자는 모듈로(modulo) 연산자라고 불러요.

이 연산자는 두 수를 나눈 후 나머지를 반환해요.

예를 들어, $7 \% 3$ 은 1이에요. 7을 3으로 나누면 몫은 2이고 나머지는 1이거든요.

수학적으로 표현하면, $a \% b = a - (a / b) * b$ 와 같아요.

여기서 /는 몫을 나타내요.

% 연산자는 주로 다음과 같은 상황에서 사용돼요

어떤 수가 특정 수의 배수인지 확인할 때

예를 들어, $x \% 2 == 0$ 이면 x는 2의 배수예요. (x는 짝수)

$x \% 3 == 0$ 이면 x는 3의 배수예요.

어떤 수를 특정 범위 내로 제한할 때

예를 들어, $x \% 10$ 은 항상 0에서 9 사이의 값이에요.

이런 특성을 이용해서 큰 수를 일정 범위 내의 수로 "축소"할 수 있어요.

순환적인 패턴을 만들 때

예를 들어, 0부터 6까지의 값을 반복하고 싶다면,

$x \% 7$ 을 사용할 수 있어요.

x가 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10일 때,

$x \% 7$ 은 0, 1, 2, 3, 4, 5, 6, 0, 1, 2, 3이 돼요.



▶ 정답 29

[기출 예제] 22년1회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

```

1  #include <stdio.h>
2
3  int isPrime(int number) { → 주어진 숫자가 소수인지 아닌지 판별해요.
4      int i;
5      for (i=2; i<number; i++) { →
6          if (number % i == 0) return 0; →
7      } → 2부터 (주어진 숫자 - 1)까지
8      return 1; → 각 숫자로 주어진 숫자를 나눠봐요.
9  } → 만약 나누어 떨어지면 (즉, % 연산 결과가 0이면 = 약수가 존재) 그 숫자는 소수가 아니라고 판단하고
10     → 0을 반환해요. 그렇지 않으면 1을 반환해요, 즉 그 숫자는 소수예요.
11
12 int main(void) { → 여기서 13195예요.
13     int number = 13195, max_div=0, i; → max_div는 가장 큰 소수 약수를 저장할 변수예요, 0으로 할당되어 있어요.
14     for (i=2; i<number; i++) → 2부터 number까지 반복하면서
15         if (isPrime(i) == 1 && number % i == 0) max_div = i; → 각 숫자가 소수인지(isPrime(i) == 1) 이면서,
16         printf("%d", max_div); → 그 숫자가 소수이면서 number의 약수라면, number의 약수인지(number % i == 0) 확인해요.
17         return 0; → max_div에 그 숫자를 저장하고, 출력해요.
18     }

```

특정 숫자의 가장 큰 소수인 약수를 찾는 예제예요.

여기서 사용된 개념은 '소수'와 '약수'인데, 개념은 다음과 같아요.

★ 소수(Prime Number)

1과 자기 자신 외에는 어떤 수로도 나눌 수 없는 숫자예요.

예를 들어, 2, 3, 5, 7 등이 소수예요.

★ 약수(Divisor)

한 수를 나누어 떨어지게 하는 숫자를 말해요.

예를 들어 6의 약수는 1, 2, 3, 6이에요.

★ 이런 식으로 답을 유추할 수 있어요.

단순 for문 문제지만, 답을 찾는 과정을 유추하기 까다로운 문제예요.

13195의 약수 중 가장 큰 소수를 찾아야 하는 걸 알았다고 하더라도, 이걸 손으로 풀기에는 까다로울 수 있어요.

이때, 조금은 쉽게 찾을 수 있는 방법을 알려드릴게요.

방법은 소수를 이용해서 나눠지지 않을 때까지 나누는 거예요.

13195 / 소수1 => (나온 값) / 소수2 => (나온 값) / 소수3 를 진행하다보면, 더 이상 나눠지지 않는 시점이 오는데
여태 나누는데 사용했던 모든 소수(소수1, 소수2, 소수3...) 중 가장 큰 값이 가장 큰 소수이면서 약수인 값이에요.

13195 / [소수] = 나온 값은 일단, 13195의 약수이면서 그 값이 더 이상 나눠지지 않는다면 가장 큰 소수일 테니까요

소수는 2, 3, 5, 7, 11 ... 이렇게 있어요.

2로도 나눠지지 않아요. 넘어가요. (ex. 13195 % 2 = 1의 나머지가 있으니 나눠지지 않아요.)

3로도 나눠지지 않아요. 넘어가요. (ex. 13195 % 3 = 1의 나머지가 있으니 나눠지지 않아요.)

5으로 정확하게 나눠지는데, 2,639가 나와요. (지금까지 13195의 약수면서 가장 큰 소수는 5예요)

다음으로는 7로 나눠지는데, 377가 나와요. (지금까지 13195의 약수면서 가장 큰 소수는 7예요)

그 다음 11로는 나눠지지 않아요. 넘어가요.

13으로 나눠지는데, 29가 나와요. (13, 29 모두 소수이기 때문에 정답은 29예요)

29는 더 이상 나눠지지 않는 소수이기 때문에 29가 정답이에요.

★ 소수

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100



★ Key Point

1. 문자의 ASCII 코드 값을 이용한 값 차이
2. for 문을 이용한 정렬 문제예요.

▶ 정답

4

BACDE

[기출 예제] 25년1회 실기 (난이도: ★★★)

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1  #include <stdio.h>
2  char Data[5] = {'B', 'A', 'D', 'E'};
3  char c;
4
5  int main(){
6      int i, temp, temp2;
7
8      c = 'C';
9      printf("%d\n", Data[3]-Data[1]);
10
11     for(i=0;i<5;++i){
12         if(Data[i]>c)
13             break;
14     }
15
16     temp = Data[i];
17     Data[i] = c;
18     i++;
19
20     for(;i<5;++i){
21         temp2 = Data[i];
22         Data[i] = temp;
23         temp = temp2;
24     }
25
26     for(i=0;i<5;i++){
27         printf("%c", Data[i]);
28     }
29 }
```

Data[3]은 E 이고, Data[1]은 A예요.

사실, 각각의 ASCII 코드는 69, 65 이기 때문에 69-65=4가 나와요.

하지만, ASCII 코드 값을 몰라도 풀 수 있어요.

A<B<C...<Z 순으로 +1씩 커진다는 걸 알고 있으면 돼요.

A,B,C,D,E 총 4 단계 차이 때문에 4가 출력됨을 알 수 있어요.

B, A, D, E 순으로 순회를 하는데,

C 보다 큰 (뒤에 위치한) 문자가 나오면 빠져나와요.

C 보다 뒤에 위치한 문자의 인덱스를 구하기 위한 for 문이에요.

스왑하기 위해 temp 변수에 기존 Data[i] = Data[3] = E 를 할당해요.

E 자리에는 문자 C를 할당해요.

i를 +1 해서 3가 돼요.

여기서도 스왑하기 위한 for 문이에요.

i가 3부터 Data[3]을 temp2에 임시저장

Data[3] 자리에 위에서 할당한 temp E를 할당

temp 에는 [21행]에서 저장한 E를 할당해요.

i가 4인 경우도 동일하게 반복하면 B,A,C,D,E 로 배열이 정렬돼요.

[27행]은 배열을 출력하라는 구문이기에, BACDE 가 출력돼요.

정답은

4

BACDE 가 돼요.



[문제 유형] 연산자

[기출 예제] 23년 2회 실기 (난이도: ★★☆☆)

▶ 정답 $n[(i+1) \% 5]$

다음 C언어로 구현된 프로그램을 실행시킨 결과가 "43215"일 때, <처리조건>을 참고하여 괄호에 들어갈 알맞은 식을 쓰시오.

입력값이 54321일 경우 출력값이 43215로 출력되어야 한다.

<처리조건>

괄호의 식에 사용할 문자는 다음으로 제한한다.

- n, i
- +, -, /, *, %
- 0~9, (,), [,]

★ 모듈로(modulo) 연산자에 대한 설명은 위 페이지를 참고해주세요!

```

1 #include <stdio.h>
2
3 int main(void) {
4     int n[5];
5     int i;
6
7     for (i = 0; i < 5; i++) {
8         printf("숫자를 입력해주세요 : ");
9         scanf("%d", &n[i]);
10    }
11
12    for (i = 0; i < 5; i++) {
13        printf("%d", (
14            ) );
15    }
16    return 0;
17 }
```

n이라는 이름의 정수 배열을 선언했어요. 크기는 5예요.

i라는 정수 변수도 선언했어요. 이건 반복문에서 사용할 거예요.

for 반복문을 사용해서 사용자로부터 5개의 숫자를 입력받아요.

반복할 때마다 "숫자를 입력해주세요 : "라는 메시지를 출력해요.

scanf 함수를 사용해서 사용자가 입력한 숫자를 n 배열에 저장해요.

다른 for 반복문을 사용해서 n 배열에 저장된 숫자들을 출력해요.

이때, 괄호 안에는 우리가 출력하고 싶은 순서대로 숫자를 가져와야 해요.

n[0] n[1] n[2] n[3] n[4]

	n[0]	n[1]	n[2]	n[3]	n[4]
	5	4	3	2	1

위의 배열처럼 값이 저장되어 있고, 우리는 43215를 출력해야해요.

이 순서를 잘 보면, 인덱스가 1, 2, 3, 4, 0 순으로 변하고 있어요.

첫 번째 인덱스는 1이고,

다음 인덱스는 이전 인덱스에 1을 더한 값이에요.

그런데 인덱스가 4 다음에는 0으로 "회전"하고 있어요.

이걸 코드로 표현하면 $(i+1) \% 5$ 가 되는 거예요.

i는 0부터 4까지 변하는 반복문의 변수예요.

i+1은 1부터 5까지 변하겠죠.

여기에 % 5를 하면, 값의 범위는 0부터 4가 돼요.

 $(0+1) \% 5 = 1$ $(1+1) \% 5 = 2$ $(2+1) \% 5 = 3$ $(3+1) \% 5 = 4$ $(4+1) \% 5 = 0$



[기출 예제] 23년1회 실기 (난이도: ★★★)

다음은 2진수 101110을 10진수로 변환하는 C언어 프로그램이다. 프로그램을 분석하여 괄호(a~b)에 들어갈 알맞은 답을 쓰시오.

▶ 정답

a. %
b. 10

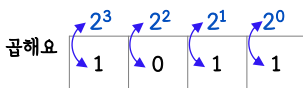
```

1  #include <stdio.h>
2
3  int main() {
4
5      int input = 101110;  → 2진수 101110을 10진수로 변환하고자 하는 입력값 input을 정의하고 있어요.
6      int di = 1;          → 2진수의 각 자리수를 10진수로 변환할 때 사용할 배수 di를 정의합니다.
7      int sum = 0;         2진수의 가장 오른쪽 자리부터 시작하여, 2의 거듭제곱을 나타내는 값이에요.
8                          변환된 10진수 값을 누적할 변수 sum을 0으로 할당해요.
9      while (1) { 무한 루프를 시작합니다.
10
11          if (input == 0) {    input이 0이 되면 모든 자리를 처리했으므로 반복문을 빠져나옵니다.
12              break
13          } else {
14
15              sum = sum + (input (a)(b)) * di; → 이 부분이 중요해요.
16              di = di * 2;      여기서는 input의 가장 오른쪽 자리수를 가져와야 합니다.
17              input = input / 10; 이를 위해 input을 10으로 나눈 나머지를 사용하면
18                                2진수의 가장 오른쪽 숫자를 얻을 수 있죠.
19                                그래서 괄호 (a)에는 '%'를, (b)에는 '10'을 넣어야 해요.
20
21                                di를 2배 증가시키는데, 이는 2진수의 다음 자리가 10진수로
22                                2배 더 큰 가치를 가지기 때문이에요.
23
24                                input을 10으로 나눠서 다음 자리수로 넘어갑니다.
25                                이는 2진수를 한 자리씩 오른쪽으로 이동시키는 효과가 있죠.
26      }
27
28      printf("%d", sum);
29
30      return 0;
31  }

```

예를 들어, 101110 을 10으로 나누면, 10111 이 되면서 한 자리 오른쪽 이동이 되었어요.

★ 2진수 계산은 이렇게 할 수 있어요



2진수를 계산할 때, 각 자리수의 2의 제곱 * 값(1 또는 0)이에요.

예에서는 $(2^3 \times 1) + (2^2 \times 0) + (2^1 \times 1) + (2^0 \times 1)$ 으로 표현할 수 있고, 1011의 10진수는 $8+0+2+1=11$ 이 돼요.



★ 연산자 종류

관계	==	같다	>	크다	<	작다
	!=	같지 않다	>=	크거나 같다	<=	작거나 같다
비트	&	and	모든 비트가 1이면 1			
	^	xor	모든 비트가 같으면 0, 하나라도 다르면 1			
		or	모든 비트 중 하나라도 1이면 1			
	~	not	각 비트의 부정 (0이면 1 산출)			
	<<	-	비트를 왼쪽으로 이동			
	>>	-	비트를 오른쪽으로 이동			
논리	!	not	&&	and		or

★ 연산자는 외워두면 좋아요

왼쪽 표를 먼저 숙지하고, 문제를 푸는 게 좋아요.

외울 게 많아 보이지만, 현실에서의 수학 기호와 비슷해서 외우는데 크게 어렵지 않을 거예요!

[기출 예제] 22년1회 실기 (난이도: ★★★)

다음은 정수를 역순으로 출력하는 C언어 프로그램이다. 예를 들어 1234의 역순은 4321이다.

단, 1230 처럼 0으로 끝나는 정수는 고려하지 않는다. 프로그램을 분석하여 괄호(①~③)에 들어갈 알맞은 연산자를 쓰시오.

```

1  #include <stdio.h>
2
3  int main() {
4
5      int number = 1234;
6      int div = 10;
7      int result = 0;
8
9      while (number ( ① ) 0) {
10         result = result * div;
11         result = result + number ( ② ) div;
12         number = number ( ③ ) div;
13     }
14
15     printf("%d", result);
16     return 0;
17 }
18

```

5 → 뒤집을 숫자가 저장되어 있어요 (예: 1234).
 6 → 10으로 설정돼 있어요. 이걸 나중에 숫자를 자릿수별로 나누는 데 사용돼요.
 7 → 최종 결과를 저장하기 위해 0으로 할당되어 있어요.
 9 → while 반복문을 사용해서 **number가 0이 될 때까지 반복**해요.
 10 → result는 10배(div) 증가시키고 (result * div)
 11 → number의 마지막 자릿수를 추가해요 (number % div).
 12 → 10으로 나눠서 한 자리씩 줄여나가요 (number / div).

▶ 정답

- ① >
- ② %
- ③ /

★ 이런 식으로 유추할 수 있어요.

먼저, 주어진 정보는 [10행]에서 result에 10을 곱한다는 거예요.

이 말은 1이라면 10이 되겠고, 25라면 250이 될 거예요.

즉, 왼쪽으로 한 칸 이동시키겠다는 << 비트연산자와 유사해요.

그럼 처음 result에 들어간 값을 계속 왼쪽으로 이동한다는 의미이고,

우리의 number 1234 중에서는 4가 result에 먼저 들어가야 계속해서 왼쪽으로 이동해서 결국에는 역순이 될 수 있겠죠?

number (②) div = 4가 나오려면, % 연산자를 사용해야 해요.

숫자를 대입하면, 1234 (②) 10 = 4인 건데, 10으로 나눈 나머지가 필요하기 때문이에요.

두 번째로, number에는 4는 필요없어졌으니, number의 4를 지울 필요가 있겠죠?

즉, number를 오른쪽 이동 시켜야 하는데, 이때는 10을 나눠주면 돼요.

숫자를 대입하면, 1234 (③) 10 = 123이 돼야하는데, 그래서 / 연산자가 필요해요.

마지막으로, (①)를 찾아야 해요.

[12행]에서 number가 계속 10씩 나누기 한다는 사실을 알았고,

0이 되면, result에 넣을 number 값도 없겠죠.

즉, 자연스럽게 (①) 은 > 이 돼요.

number가 0 보다는 커야 역순 처리를 할 값이 존재할테니까요!



[기출예제] 24년1회 실기 (난이도: ★★★)

다음 C언어 코드에서 알맞는 출력 값을 작성하십시오.

★ 삼항 연산자

삼항 연산자 ?는 조건식 ? 참일 때 값 : 거짓일 때 값 형태로 동작합니다.

True ? "a" : "b" -> a 반환

False ? "c" : "d" -> d 반환

▶ 정답
151

```

1 #include <stdio.h>
2
3 int main() {
4     int v1 = 0, v2 = 35, v3 = 29;
5
6     if(v1 > v2 ? v2 : v1) {
7         v2 = v2 << 2;
8     } else {
9         v3 = v3 << 2;
10    }
11
12    printf("%d", v2+v3);
13 }
```

세 개의 정수형 변수 v1, v2, v3가 선언되고
각각 0, 35, 29로 초기화돼요.

if(v1 > v2 ? v2 : v1) 는
if(0 > 35 ? 35 : 0) 으로 해석되고,
if(false ? 35 : 0) 이므로,
if(0) 이 됩니다.

if 문 안에 0이 들어가면 거짓이기 때문에 else 가
수행되게 됩니다.

v3 = v3 << 2; 는 비트 연산을 수행합니다.

v3를 왼쪽으로 두 비트 시프트합니다.

V3는 29이고, 2진수로 11101입니다.

왼쪽으로 2칸 이동하면 1110100 이고

1110100 은 10진수로 116입니다.

116을 v3에 덮어쓰게 됩니다.

[12행] 은 v2 + v3 = 35 + 116 = 151 이 출력되게 됩니다.

삼항 연산자, 비트 연산자의 개념을 모르면 풀기 어려울 수 있는 문제예요.

하지만, 한번만 이해했다면 다시 만나면 쉽게 풀 수 있어요!

★ Key Point

구조체와 포인터, 그리고 비트 AND(&) 연산, 16진수에 대해 이해하고 있어야 해요.

개념만 알면 쉽지만, 이전에 나오지 않은 스타일이기에 어려울 수 있어요.

- 비트 AND 연산 (&): dec 함수 내의 enc & 0xA5는 enc와 0xA5의 각 비트를 비교하여, 두 비트가 모두 1인 경우에만 1을 반환합니다.
- 16진수: 코드에 0xA0, 0xA5와 같이 0x로 시작하는 값은 16진수를 의미합니다. 비트 연산을 위해서는 이를 2진수로 변환하여 계산해야 합니다.

▶ 정답
908

[기출예제] 25년1회 실기 (난이도: ★★★)

다음 C언어 코드에서 알맞는 출력 값을 작성하십시오.

```

1 #include <stdio.h>
2
3 typedef struct student {
4     char* name;
5     int score[3];
6 } Student;
7
8 int dec(int enc) {
9     return enc & 0xA5;
10 }
11
12 int sum(Student* p) {
13     return dec(p->score[0]) + dec(p->score[1]) + dec(p->score[2]);
14 }
15
16 int main() {
17     Student s[2] = { "Kim", {0xA0, 0xA5, 0xDB}, "Lee", {0xA0, 0xED, 0x81} };
18     Student* p = s;
19     int result = 0;
20
21     for (int i = 0; i < 2; i++) {
22         result += sum(&s[i]);
23     }
24     printf("%d", result);
25     return 0;
26 }
```

Student 구조체를 정의해요.

학생 이름을 가리킬 포인터
3개의 정수형 점수를 저장할 배열

정수(enc)를 받아 0xA5 (10100101)와 비트 AND 연산을 한 결과를 반환해요.

enc를 2진수로 변환하고, 같은 자리 수의 비트가 모두 1이어야 1을 반환, 그 외에는 0을 반환해요.

★ 0xA5 2진수

16진수는 1,2,3,4,5,6,7,8,9,A(10),B(11),C(12),D(13),E(14),F(15) 로 표현해요.

A는 10을, 5는 5을 의미해요. 2진수로 표현하면 1010, 0101 이에요.

합쳐서 10100101 이에요.

Kim	0xA0 (2진수: 10100000)	0xA5 (2진수: 10100101)	0xDB (2진수: 11011011)
	0xA5 (2진수: 10100101)	0xA5 (2진수: 10100101)	0xA5 (2진수: 10100101)
	10100000 (10진수: 160)	10100101 (10진수: 165)	10000001 (10진수: 129)

Lee	0xA0 (2진수: 10100000)	0xED (2진수: 11101101)	0x81 (2진수: 10000001)
	0xA5 (2진수: 10100101)	0xA5 (2진수: 10100101)	0xA5 (2진수: 10100101)
	10100000 (10진수: 160)	10100101 (10진수: 165)	10000001 (10진수: 129)

160 + 165 + 129 + 160 + 165 + 129 = 908 이 result에 저장돼요.

908이 출력돼요.



[문제 유형] 단순 while문

★ while 구문

while 구문은 '조건을 만족하는 동안 계속 반복해' 라는 뜻이에요. 마치 '이 책 재미있으면 계속 읽어'라고 말하는 것처럼, 프로그램에게 어떤 조건이 참인 동안은 특정한 일을 계속 하라고 지시하는 거죠.

```
int i = 0; // i를 0으로 시작해요.
while (i < 5) { // i가 5보다 작은 동안에는, 아래 명령을 반복해요.
    printf("%d\n", i); // i의 값을 화면에 보여줘요.
    i++; // i를 1씩 증가시켜요. 이걸 '증감'이라고 해요.
}
```

이 프로그램은 i를 0에서 시작해서 1씩 증가시키면서, i의 값이 5보다 작을 때까지 i의 값을 화면에 출력해요. i가 5가 되면, 조건 'i < 5'가 거짓이 되고, 그래서 'while' 반복문을 끝내게 됩니다. 그래서 화면에는 0부터 4까지의 숫자가 차례로 보여지겠죠.

▶ 정답 0

[기출예제] 20년 3회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

```
1  #include <stdio.h>
2
3  void main() {
4      int i = 0, c = 0; → i와 c라는 두 개의 정수 변수가 선언되어 있고, 둘 다 0으로 할당되어 있어요.
5      while (i < 10) { → i가 10보다 작을 때까지 계속 반복됩니다.
6          i++; → 반복문 내에서, 먼저 i를 1씩 증가시켜요 (i++).
7          c*=i; → c에 i를 곱한 값을 다시 c에 할당해요 (c *= i). 이것은 c = c * i와 같은 의미예요.
8      }
9      printf("%d", c); → 정답은 0이에요.
10 }
```

★ 0에다 어떤 숫자를 곱해도 0이에요

```
i = 1
c = c * i = 0 * 1 = 0
i = 2
c = c * i = 0 * 2 = 0
i = 3
c = c * i = 0 * 3 = 0
i = 4
c = c * i = 0 * 4 = 0
i = 5
c = c * i = 0 * 5 = 0
i = 6
c = c * i = 0 * 6 = 0
i = 7
c = c * i = 0 * 7 = 0
i = 8
c = c * i = 0 * 8 = 0
i = 9
c = c * i = 0 * 9 = 0
```



[문제 유형] 재귀 함수

★ 재귀 함수

함수 내부에서 자기 자신을 다시 호출하는 함수예요. 여기서 $f(n)$ 은 n 과 $f(n-1)$ 의 곱으로 정의되고 있어요.

재귀 함수를 풀 때는, [4행] 처럼 종료조건(더 이상 재귀하지 않고 반환하는 조건) 까지 함수를 나열했다가 거꾸로 값을 계산하면서 최초 함수까지 올라오면 돼요.

이런 식으로 함수가 자기 자신을 호출하면서 문제를 더 작은 문제로 나누어 풀어요.

[기출 예제] 23년 3회 실기 (난이도: ★★★)

다음 C 프로그램 출력 결과 구하기

▶ 정답 5040

```

1  #include <stdio.h>
2
3  int f(int n) {
4      if(n<=1) return 1;
5      else return n*f(n-1);
6  }
7
8  int main() {
9      printf("%d", f(7));
10 }
```

정수 n 을 매개변수로 받아요.

만약 n 이 1보다 작거나 같으면 1을 반환해요.

그렇지 않으면 n 과 $f(n-1)$ 의 곱을 반환해요.

이 함수는 재귀 함수라고 불러요.

$f(7)$ 을 호출하고 그 결과를 출력하고 있어요.

$$\begin{aligned}
 f(7) &= 7 \times f(6) \\
 &= 7 \times 6 \times f(5) \\
 &= 7 \times 6 \times 5 \times f(4) \\
 &= 7 \times 6 \times 5 \times 4 \times f(3) \\
 &= 7 \times 6 \times 5 \times 4 \times 3 \times f(2) \\
 &= 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times f(1) \\
 &= 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1
 \end{aligned}$$

즉, 수학에서의 팩토리얼 계산을 구현한 것이예요.

팩토리얼은 1부터 어떤 양의 정수까지의 모든 양의 정수의 곱이예요.

예를 들어, 7은 $1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 = 5040$ 이 되는 거죠.

그래서 정답은 5040이 돼요!

[기출 예제] 22년 1회 실기 (난이도: ★★★)

▶ 정답 120

다음 C언어로 구현된 프로그램을 분석하여 5를 입력했을 때 그 실행 결과를 쓰시오.

(단, 출력문의 출력 서식을 준수하시오.)

```

1  #include <stdio.h>
2
3  int func(int a) {
4      if (a <= 1) return 1;
5      return a * func(a - 1);
6  }
7
8  int main() {
9      int a;
10     scanf("%d", &a);
11     printf("%d", func(a));
12 }
```

하나의 정수 a 를 매개변수로 받아요.

a 가 1 이하일 경우 1을 반환해요. (종료 조건)

a 에 $func(a-1)$ 의 결과를 곱해서 반환해요.

이 부분이 재귀 호출이예요.

사용자로부터 정수 a 를 입력받아요.

$func(a)$ 를 호출해서 그 결과를 출력해요.

여기서는 5를 넣어서 호출할거예요

정답은 $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ 이예요.

$$120 == func(5)$$

$$5 \times 24 == func(4)$$

$$4 \times 6 == func(3)$$

$$3 \times 2 == func(2)$$

$$2 \times 1 == func(1)$$

$$\begin{aligned}
 f(5) &= 5 \times f(4) \\
 &= 5 \times 4 \times f(3) \\
 &= 5 \times 4 \times 3 \times f(2) \\
 &= 5 \times 4 \times 3 \times 2 \times f(1) \\
 &= 5 \times 4 \times 3 \times 2 \times 1
 \end{aligned}$$



[문제 유형] 정렬

▶ 정답 50 75 85 95 100

[기출 예제] 20년1회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

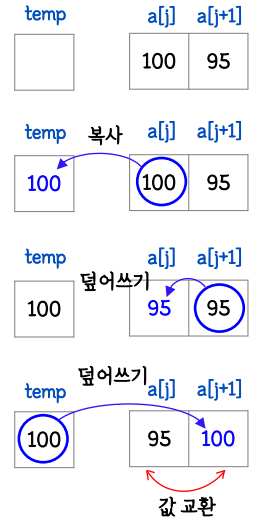
★ key point

버블 정렬이든, 선택 정렬이든 정렬 종료에 상관없이 정렬 문제인걸 캐치했다면, 오름차순인지 내림차순인지만 정확하게 파악하고 순서대로 출력하면 돼요!
꼭 손으로 다 할 필요는 없어요!

a[5]라는 정수 배열이 선언되어 있고,
[75, 100, 95, 50, 85] 5개의 요소로 할당되어 있어요

★ swap

[11행]의 로직 동작 과정이에요.



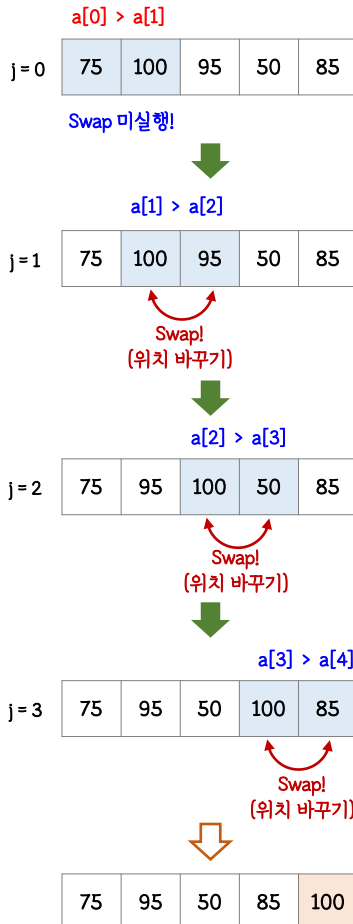
```

1  #include <stdio.h>
2
3  void main() {
4      int i,j;
5      int temp;
6      int a[5] = { 75, 100, 95, 50, 85 };
7
8      for(i=0; i<4; i++){
9          for(j=0; j<4-i; j++){
10             if(a[j] > a[j+1]){
11                 temp=a[j];
12                 a[j] = a[j+1];
13                 a[j+1] = temp;
14             }
15         }
16     }
17
18     for(i=0; i<5; i++){
19         printf("%d ", a[i]);
20     }
21 }

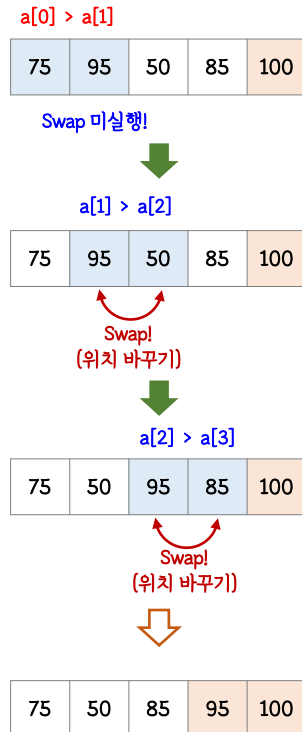
```

★ 버블 정렬

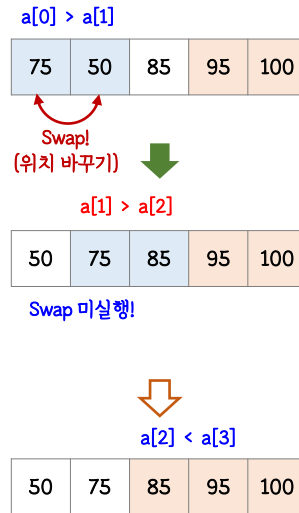
i=0 ▶ j<4
(j=0,1,2,3 실행)



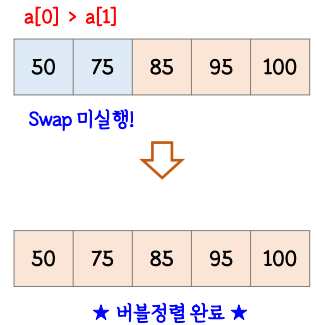
i=1 ▶ j<3
(j=0,1,2 실행)



i=2 ▶ j<2
(j=0,1 실행)



i=3 ▶ j<2
(j=0, 실행)





[기출 예제] 23년1회 실기 (난이도: ★★★)

다음은 버블 정렬을 이용하여 배열에 저장된 수를 오름차순으로 정렬하는 프로그램이다. 프로그램을 분석하여 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오.

```

1  #include <stdio.h>
2
3  void swap(int* a, int idx1, int idx2) {
4      int t = a[idx1];           → 배열의 idx1 위치에 있는 원소를 임시 변수 t에 저장합니다.
5      a[idx1] = a[idx2];         → idx2 위치에 있는 원소를 idx1 위치로 이동합니다.
6      a[①] = t;                 → 처음 idx1 위치에 있던 원소를 idx2 위치로 이동합니다. 여기서 괄호 (①)에는 idx2가 들어가야 합니다.
7  }
8
9  void sort(int* a, int len) {   → 배열 a를 버블 정렬 방식으로 정렬합니다. len은 배열의 길이입니다.
10     for (int i = 0; i < len-1; i++) → 바깥쪽 for 반복문은 정렬할 배열의 길이만큼 반복합니다. 배열의 모든 원소를 확인해야 하니까요.
11         for (int j = 0; j < len-1-i; j++) → 안쪽 for 반복문은 정렬된 원소를 제외하고 나머지 원소들을 비교합니다.
12             if (a[j] > a[j+1]) → 인접한 두 원소의 크기를 비교하고, 앞쪽 원소가 더 크면 위치를 바꿉니다.
13                 swap(a, j, j+1); → [3행] swap 함수를 호출하여 j번째와 j+1번째 원소의 위치를 바꿉니다.
14 }
15
16 void main() {
17     int a[] = { 85, 75, 50, 100, 95 }; → 정렬할 배열을 할당합니다.
18     int nx = 5; → 배열 a의 원소 개수를 저장하는 변수입니다.
19     sort(a, ②); → [9행] sort 함수를 호출하여 배열 a를 정렬합니다.
20 }

```

여기서 괄호 (②)에는 배열의 길이인 nx가 들어가야 합니다.

▶ 정답
① idx2
② nx

★ key point

여기서는 오름차순 정렬이라는 힌트를 문제에서 줬어요.
메소드명 swap이고, swap은 일반적으로 두 요소의 위치를 바꾸는 걸 의미해요. 우리는 그럼 두 요소의 위치를 바꾸는 코드를 작성해주면 될 거라는 걸 유추해볼 수 있어요.

idx1의 값을 t라는 변수에 임시 저장했고, idx1 요소는 idx2로 덮어썼고
idx2의 요소에 idx1을 넣어야 서로 위치가 바뀌는 효과가 있겠죠?
그래서, ①에는 idx2가 들어가요.

너무 어렵다면, 바로 위 페이지의 ★ swap을 참고해주세요!

★ key point

원래 정상적인 문제 풀이의 흐름으로는 배열의 끝까지 반복해야하니 ②의 값은 nx가 된다고 생각하는게 올바른 풀이예요.

하지만, 실제 시험장에서 코드 해석이 어려워 찍어야 하는 상황에서는 다음처럼 유추해서 정답을 맞출 수도 있어요. sort에 들어갈 수 있는건 main문의 변수가 a 배열과 nx 두개만 존재하는 상황에 첫번째 매개변수로 a가 이미 들어간 상황에서는 추가로 넣어줄 변수는 nx밖에 없어 nx가 정답이 될 거라는 생각이 들 수 있어요. 이 방법은 답을 아예 모르는 상황에서 조금이라도 정답에 가까운 답을 적기 위한 방법이지 절대 옳은 방법이 아니예요!

[기출 예제] 23년2회 실기 (난이도: ★★★)

다음은 데이터를 오름차순으로 정렬하는 선택 정렬 알고리즘을 C 언어 프로그램으로 구현한 것이다.

프로그램을 분석하여 괄호에 들어갈 알맞은 연산자를 쓰시오.

<처리조건>

괄호의 식에 사용할 문자는 다음으로 제한한다.

- +=, -=, *=, /=
- ==, !=, >, >=, <, <=
- &&, ||

```

1  #include <stdio.h>
2  int main() {
3      int E[] = {64, 25, 12, 22, 11}; → 정렬할 데이터가 저장된 배열 E를 선언하고 할당하고 있어요.
4      int n = sizeof(E) / sizeof(E[0]); → 배열 E의 전체 크기를 첫 번째 요소의 크기로 나눠서 배열에 몇 개의 요소가 있는지를 계산해요.
5      int i = 0; → 즉, 배열 E의 길이 n을 구하는 거죠.
6      do {
7          int j = i + 1; → 내부 루프에서 비교할 두 번째 요소의 시작점을 설정해요. i 다음 요소부터 시작하죠.
8          do {
9              if (E[i] ( ① ) E[j]) { 이 부분이 중요한데요, 여기서는 E[i]가 E[j]보다 클 때 두 요소의 위치를 바꿔줘야 해요.
10                 int tmp = E[i]; → 오름차순 정렬을 하려면 현재 선택된 요소(E[i])가 비교 대상 요소(E[j])보다 클 경우에만 위치를
11                 E[i] = E[j]; → 바꾸면 되거든요.
12                 E[j] = tmp;
13             }
14             j++;
15         } while (j < n);
16         i++;
17     } while (i < n-1); → i가 배열의 길이보다 하나 작을 때까지 반복해요.
18     for(int i=0; i<=4; i++)
19         printf("%d ", E[i]);
20 }

```

▶ 정답
① >

★ key point

정렬 문제에서 각 요소들의 정렬 조건에 들어갈 빈칸이니, 두 요소의 비교가 커거나(<), 작거나(>), 커거나 같거나(<=), 작거나 같거나(>=) 만 들어갈 수 있음을 유추할 수 있어요.

정렬 종류에 상관없이 현재 오름 차순으로 정렬하려고 하고, 작은 순으로 정렬이 되어야할 때, 왼쪽의 요소가 오른쪽 요소보다 클 때 바꿔줘야 작은 순으로 정렬될 수 있어요.

j는 i보다 +1이 크기때문에, i는 왼쪽 요소 j는 오른쪽 요소예요. 왼쪽이 오른쪽보다 크면 두 요소를 바꿔야하니 >이 돼요. >=는 왜 정답이 아니냐면, 틀린 방법은 아니지만 같을 때 굳이 두 요소를 변경할 필요가 없기 때문이에요!

따라서 괄호에 들어갈 알맞은 연산자는 > 입니다.
이 조건은 '만약 E[i]가 E[j]보다 크다면'을 의미하죠.



[기출 예제] 24년1회 실기 (난이도: ★★★)

다음은 C언어에 대한 문제이다. 알맞는 출력 값을 작성하시오.

이 문제는 문자열을 뒤집고 나서 특정 위치의 문자들을 출력하는 문제예요.
문제를 풀 때, **main** 을 먼저 살펴보세요.

▶ 정답
GECA

```

1  #include <stdio.h>
2  #include <string.h>
3
4  void reverse(char* str){
5      int len = strlen(str);
6      char* p1 = str;
7      char* p2 = str + len - 1;
8      while(p1 < p2){
9          char t = *p1;
10         *p1 = *p2;
11         *p2 = t;
12         p1++;
13         p2--;
14     }
15 }
16
17 int main() {
18     char str[100] = "ABCDEFGH";
19
20     reverse(str);
21
22     int len = strlen(str);
23
24     for(int i=1; i<len; i+=2) {
25         printf("%c", str[i]);
26     }
27
28     printf("\n");
29
30     return 0;
31
32 }
```

strlen(str)을 사용하여 문자열의 길이를 구해요.

p1은 문자열의 시작을,

p2는 문자열의 끝을 가리켜요.

p1의 값을 t에 임시로 옮기고,

*p1에 *p2의 값을 넣고,

*p2에 임시로 넣어둔 t의 값을 넣어요. (이러면 양끝의 값이 바뀌어요)

p1++과 p2--를 통해, 양 극단에서 한 칸씩 가까워지면서

그 두 값을 바꾸고 p1이 p2보다 같거나 커지면 그만뒤요.

결과적으로는, 파라미터 str의 값이 뒤집어져서

HGFEDCBA가 돼요.

str 배열을 선언하고 "ABCDEFGH"로 초기화하고 있어요.

reverse 함수를 호출하여 str에 대해서 거꾸로 뒤집어요.

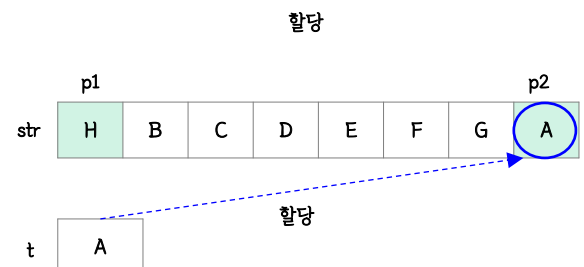
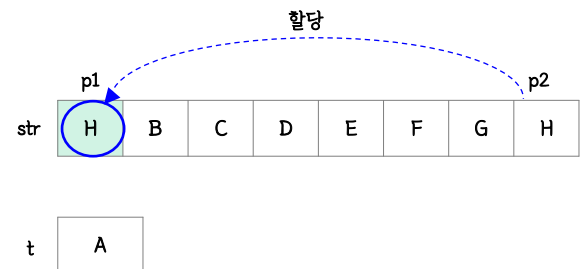
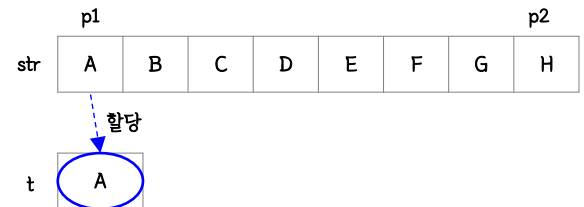
str의 길이를 len에 할당했어요.

i가 0 이 아닌 1 부터 2씩 증가하면서 len(끝)까지 반복해요.

i 번째 index 값을 출력해요. len은 8이므로, 1부터 8미만까지 동작해요.

1 -> 3 -> 5 -> 7 index의 문자를 출력해요.

str은 HGFEDCBA 이므로, GECA가 정답이 돼요.





[기출 예제] 24년 2회 실기 (난이도: ★★★) 오래 걸리지는 않지만, 실수하기 좋은 문제예요

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

▶ 정답
-13

```

1  #include <stdio.h>
2
3  void swap(int a, int b) { → C에서 기본적으로 함수 매개변수는 '값에 의한 전달' 방식을 사용해요.
4      int t = a;           이는 함수 내에서 매개변수 값을 변경해도 호출한 곳의 원래 변수에는 영향을 주지 않는다는 뜻이에요.
5      a = b;               포인터로 주소를 전달하지 않는 한 swap 함수가 실제로 a와 b의 값을 바꾸지 않아요.
6      b = t;
7  }
8
9  int main() {
10
11     int a = 11; → a와 b라는 두 개의 정수 변수를 만들고 각각 11과 19로 초기화해요.
12     int b = 19;
13     swap(a, b); → swap 함수를 호출해요. 하지만 이 호출은 위에서 설명한 것처럼 a와 b의 값을 실제로 바꾸지 않아요.
14
15     switch(a) { → a가 11이므로 case 11부터 시작해서 b에 2를 더하고,
16         case 1:      b += 1; 그 다음 default로 넘어가 b에 3을 더 더해요.
17         case 11:     break; 가 없기 때문이에요.
18         case 11:     b += 2; b는 24가 돼요.
19         default:    b += 3;
20         break;
21     }
22
23     printf("%d", a-b); → 11 - 24 이므로 -13 이 출력돼요.
24
25 }
26

```

★ key point

swap 방식은 꾸준히 출제된 유형의 문제예요.

그래서, 포인터의 개념을 아는 지 출제 의도를 넣어 꼬아서 낸 문제예요.

단순히, swap 방식으로 문제를 풀었다면 틀렸을 거예요.



[문제 유형] 포인터

★ 포인터

포인터는 매우 중요하지만 어려울 수 있는 개념이에요. 포인터는 메모리 주소를 저장하는 변수로, 특정한 데이터의 위치(주소)를 가리킵니다. 이를 통해 직접적으로 메모리에 접근하고, 데이터를 효율적으로 관리할 수 있습니다. $P = \&a$ 라고 할당하면, p 는 변수 a 의 주소를 가리키게 됩니다. $*p$ 를 사용하면 p 가 가리키는 주소에 있는 데이터, 즉 a 의 값에 접근할 수 있습니다.

포인터 문제는 정말 많이 반복해서 출제되지만, **문제의 형식을 계속 유사하게 출제**됩니다. **포인터 개념을 이해하고 가면 정말 좋지만 너무 어렵다면 문제의 유형별 답을 찾는 방법만 암기하시는 것도 방법**이에요!

▶ 정답

Art
A
A
Art
Art

[기술 예제] 23년1회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```
1 #include <stdio.h>
2
3 int main(){
4     char a[] = "Art";
5     char* p = NULL;
6     p = a;
7
8     printf("%s\n", a);
9     printf("%c\n", *p);
10    printf("%c\n", *a);
11    printf("%s\n", p);
12
13    for(int i = 0; a[i] != '\0'; i++)
14        printf("%c", a[i]);
15
16 }
17
```

역슬래시 $\backslash$ 는 Enter와 비슷해요. 한 줄 내림해요.

배열에 바로 문자열을 저장하는 경우, 그 끝을 알 수 없기 때문에 역슬래시 $0 (= \text{null} = \text{w0} = \backslash 0$ 전부 같은 표현이예요 운영체제나 환경별로 표시되는 형식만 다를뿐이예요.) 이 배열의 끝에 추가돼요. 문자열 "Art"를 저장하는 문자 배열 a 를 선언하고 할당하고 있어요.

이 배열에는 'A', 'r', 't', 그리고 문자열의 끝을 나타내는 널 문자('w0')까지 총 4개의 문자가 들어 있죠.

p 라는 이름의 포인터를 선언하고 NULL로 할당해요.

$p = a$; → 포인터 p 에 배열 a 의 주소를 할당해요. 이제 p 는 문자열 "Art"가 저장된 배열 a 를 가리키고 있죠.

문자열 a 를 출력하는 코드예요. $\%s$ 는 문자열을 출력하는 형식 지정자이며, "Art"가 출력됩니다.

p 가 가리키는 위치의 문자를 출력해요. $\%c$ 는 단일 문자를 출력하는 형식 지정자이며, a 의 첫 번째 요소를 가리키므로 'A'가 출력됩니다.

배열 이름 a 를 포인터처럼 사용해서 첫 번째 문자를 출력해요. 여기서도 'A'가 출력돼요.

포인터 p 가 가리키는 문자열을 출력해요. p 는 "Art"를 가리키므로 "Art"가 출력됩니다.

문자열 a 의 각 문자를 하나씩 출력하는 반복문이에요.

널 문자('w0')를 만날 때까지 각 문자를 순서대로 출력합니다.

결과적으로 "Art"가 한 글자씩 출력되죠.

$a[0]$	$a[1]$	$a[2]$	$a[3]$
A	r	t	w0

p
 $a[0]$

[7행] A라는 값이 아닌 배열 a 의 $a[0]$ 메모리 주소를 가리키고 있어요.

★ ASCII 코드

컴퓨터가 문자를 숫자로 표현하는 방법이에요.

문자	10진	문자	10진
A	65	N	78
B	66	O	79
C	67	P	80
D	68	Q	81
E	69	R	82
F	70	S	83
G	71	T	84
H	72	U	85
I	73	V	86
J	74	W	87
K	75	X	88
L	76	Y	89
M	77	Z	90

★ key point

어렵다면, 아래 표현에 대한 정답을 찾는 방법을 외우고 풀어보세요. 계속 반복돼요.

```
char a[] = "Art"
char* b = "Good"
```

```
printf("%s", a) → "Art"
printf("%c", a[0]) → "A"
printf("%c", *a) → "A"
printf("%c", *(a+2)) → "t"
printf("%s", a+1) → "rt"
```

```
printf("%s", b) → "Good"
printf("%s", b+1) → "ood"
printf("%c", *b) → "G"
printf("%c", *(b+3)) → "d"
```

마지막으로 어려울 수 있는데, G 에 $+3$ 을 하면 J 가 돼요. ASCII 코드는 문자를 숫자로 표현하는 방법이에요. A는 65부터 시작해서 Z는 90으로 +1씩 증가해서 매핑되어있어요. *b는 G가 출력되고, 문자 G의 ASCII 코드는 71이에요. 거기에 +3은 74이고 그래서 J가 출력돼요. 어렵다면, 출력된 문자에 a,b,c,d... 순서대로 옆으로 이동한다 생각해 보세요.

가령, $*b+3$ 은 $G + 3$ 이고, abcd순서대로 나열하면 3칸 옆 알파벳은 $G \rightarrow H \rightarrow I \rightarrow J$ 니까 J를 구할 수도 있어요.

```
printf("%c", *b+3) → "J"
```



[기출 예제] 20년 4회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

```
1 #include <stdio.h>
```

```
2
3 int main(){
```

```
4 char *p = "KOREA";
```

```
5 printf("%s \n", p);
```

```
6 printf("%s \n", p+3);
```

```
7 printf("%c \n", *p);
```

```
8 printf("%c \n", *(p+3));
```

```
9 printf("%c ", *p+2);
```

```
10 }
```

→ p라는 포인터 변수가 "KOREA"라는 문자열을 가리키고 있어요. 이 때 p는 문자열의 첫 글자인 'K'의 주소를 가지고 있죠.
→ p가 가리키는 전체 문자열 "KOREA"를 출력해요.
→ p의 주소에서 3만큼 더한 위치, 즉 "KOREA"에서 4번째 글자 'E'부터 시작하는 부분 문자열 "EA"를 출력해요.
→ p가 가리키는 위치의 문자, 즉 'K'를 출력해요.
→ p에서 3만큼 더한 위치의 문자, 즉 'E'를 출력해요.
→ p가 가리키는 문자 'K'의 ASCII 값에 2를 더한 결과를 출력해요.
'K'의 ASCII 값은 75이므로, $75 + 2 = 77$ 이고, 이는 ASCII 테이블에서 'M'에 해당해요.

★ 문자열 저장 방법

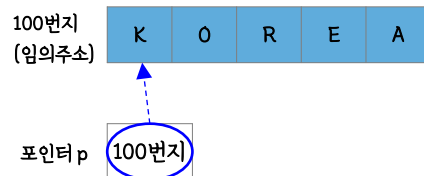
- 배열
- ex) `char arr[5] = "KOREA"`
- 포인터
- ex) `char *p = "KOREA"`

▶ 정답

KOREA
EA
K
E
M

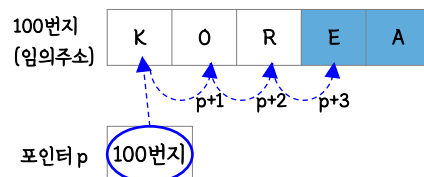
[4행] `printf("%s \n", p);`

문자열 포인터 p 를 출력



[5행] `printf("%s \n", p+3);`

문자열 포인터 p 에 저장된 값에 3 을 더한 후 출력



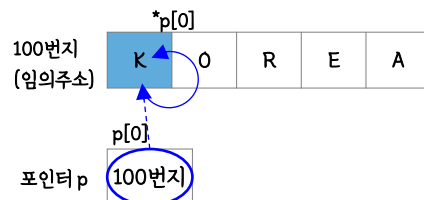
[6행] `printf("%c \n", *p);`

문자열 포인터 p 가 가리키고 있는 주소 영역에 접근하여 값을 가져와서 출력

p 는 p[0]의 주소를 가리키고, 따라서, *p는 *p[0] 이에요.

*(억참조 연산자)는 해당 주소에 존재하는 값을 가져오는 연산자예요.

*p[0]은 p[0] 주소 영역에 존재하는 값을 반환하기 때문에 K만 출력돼요.

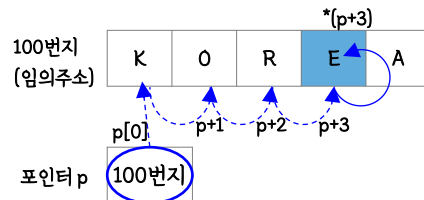


[7행] `printf("%c \n", *(p+3));`

문자열 포인터 p 에 저장된 값에 3을 더한 후, 해당 값이 가리키고 있는 주소 영역에 접근하여 값을 가져와서 출력

p 는 p[0]의 주소를 가리키고, (p+3)은 [5행]과 동일하게 3칸 이동해요.

그 다음, *(억참조 연산자) 를 통해 p+3 주소에 존재하는 값을 반환하기 때문에 E만 출력돼요.



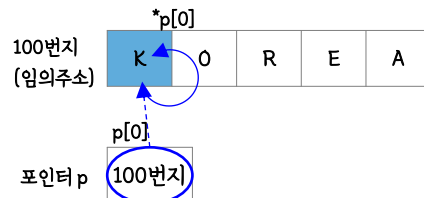
[8행] `printf("%s \n", *p+2);`

*p 의 값(K=75) 을 가져온 후 2를 더하여 출력
위의 아스키 코드를 참고하세요!

A의 아스키코드가 65 인 것만 알아두면 돼요.

▶ $75 + 2 = 77 = M$

75	K
76	L
77	M





[기출예제] 23년 3회 실기 (난이도: ★★★)

▶ 정답 K O R E A O R E A K E O

다음 C 프로그램 출력 결과 구하기

★ 중요해요

여러 년도에서 자주 반복 출제되고 있는 문제예요!

```
1 #include <stdio.h>
2
3 int main() {
4     char* p = "KOREA";
5     printf("%s ", p);
6     printf("%s ", p+1);
7     printf("%c ", *p);
8     printf("%c ", *(p+3));
9     printf("%c ", *p+4);
10 }
```

p라는 char 포인터 변수를 선언하고 있어요. 이 포인터는 문자열 "KOREA"의 첫 번째 문자인 'K'의 주소를 가리키고 있어요.

%s 형식 지정자를 사용하여 p가 가리키는 문자열 전체를 출력해요. 따라서 "KOREA"가 출력될 거예요.

p+1이 가리키는 문자열을 출력해요. p+1은 p가 가리키는 주소에서 한 칸 옆으로 이동한 주소예요.

즉, 'O'의 주소를 가리켜요. 따라서 "OREA"가 출력될 거예요.

%c 형식 지정자를 사용하여 *p의 값을 문자로 출력해요. *p는 p가 가리키는 주소에 저장된 값, 즉 'K'예요. 따라서 'K'가 출력될 거예요.

*(p+3)의 값을 문자로 출력해요. p+3은 p가 가리키는 주소에서 세 칸 옆으로 이동한 주소, 즉 'E'의 주소예요. 따라서 'E'가 출력될 거예요.

*p+4의 값을 문자로 출력해요. *p는 'K'의 ASCII 코드 값이에요. 거기에 4를 더하면 'O'의 ASCII 코드 값이 돼요. 따라서 'O'가 출력될 거예요.

그래서, 전체 출력되는 정답은 K O R E A O R E A K E O 이예요.

[기출예제] 21년 2회 실기 (난이도: ★★★)

▶ 정답 8

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

(단, 출력문의 출력 서식을 준수하시오.)

```
1 #include <stdio.h>
2
3 int main(){
4
5     int ary[3];
6     int s = 0;
7     *(ary+0) = 1;
8     ary[1] = *(ary + 0) + 2;
9     ary[2] = *ary + 3;
10    for(int i = 0; i < 3; i++) {
11        s = s + ary[i];
12    }
13
14    printf("%d",s);
15 }
```

정수 배열에 값을 할당하고, 그 값들을 모두 더해서 출력하는 코드예요.

ary라는 이름의 정수형 배열이 있어요. 배열 크기는 3이에요.

s라는 변수는 합계를 저장하기 위해 사용되고, 0으로 할당되어 있어요.

*(ary+0) = 1;은 ary 배열의 첫 번째 요소(ary[0])에 1을 할당해요.

ary[1] = *(ary + 0) + 2;은 두 번째 요소(ary[1])에 첫 번째 요소의 값(ary[0] 즉, 1)에 2를 더한 값, 즉 3을 할당해요.

ary[2] = *ary + 3;은 세 번째 요소(ary[2])에 첫 번째 요소의 값에 3을 더한 값, 즉 4를 할당해요.

for(int i = 0; i < 3; i++) { s = s + ary[i]; } 배열 ary의 모든 요소를 순회하면서 그 값을 변수 s에 더해요.

printf("%d",s); s의 최종 값을 출력해요. 정답은 1 + 3 + 4 = 8이에요

★ 배열의 이름은 배열의 0번째 주소예요

ary == &ary[0] 이고, (메모리의 주소)

*ary = ary[0] 이예요. (값)

0번째에 값을 넣고 싶으면,

*ary = 1

ary[0] = 1

*ary = 1

다 가능해요.

[7행]에서는 *(ary+0)=1 라고 적혀있는데, *(ary) = 1과 동일한 표현이에요.

★ 배열의 이름에 괄호 없이 더하는 건 포인터 이동이 아니예요.

[8행]과 [9행]의 차이는 무엇일까요?

[8행]은 괄호를 사용했고, [9행]은 괄호 없이 + 3을 했죠.

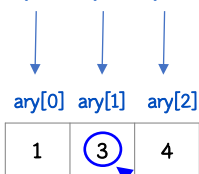
물론, [8행]도 괄호 안에 0을 더해서 주소의 이동은 없으나, 배열의 0번째 위치에서 N번째 위치의 값을 가져오고 싶을 때는 *(ary + n)으로 표현해야 해요.

*ary + n은 *ary의 값을 가져온 뒤에 정수 n을 더하는 표현이에요.

그러므로, [9행]은 *ary 먼저 가져온 뒤(1), + 3을 진행하여 4가 나와요.

그 값을 ary[2]에 할당해요.

주소 ary + 0 ary + 1 ary + 2



주소 앞 *을 붙이면 주소의 값을 가져와요
여기서는 *(ary+1)로 표현할 수 있어요



[기출예제] 21년 3회 실기 (난이도: ★★★)

▶ 정답 37

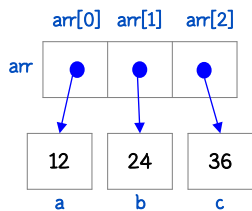
다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

```

1  #include <stdio.h>
2
3  int main(){
4      int *arr[3];  → arr이라는 이름의 포인터 배열이 있어요. 이 배열은 정수에 대한 포인터를 저장할 수 있어요.
5      int a = 12, b = 24, c = 36;  → a, b, c라는 세 개의 정수 변수가 있어요. 각각 12, 24, 36으로 할당되어 있죠.
6
7      arr[0] = &a;  → 각 배열 요소인 arr[0], arr[1], arr[2]는 각각 a, b, c의 메모리 주소를 저장하고 있어요.
8      arr[1] = &b;
9      arr[2] = &c;
10
11     printf("%d\n", *arr[1] + **arr + 1);  → *arr[1] + **arr + 1을 계산해서 출력해요.
12 }

```

여기서 *arr[1]은 arr[1]이 가리키는 주소에 있는 값, 즉 b의 값 24를 의미해요.
**arr는 arr[0]이 가리키는 주소에 있는 값, 즉 a의 값 12를 의미해요.
따라서, 이 계산은 24 + 12 + 1이 되겠죠. 이 계산의 결과, 즉 37을 출력해요.



a의 값이 아니라, a의 메모리 주소가 arr[0]에 들어있어요.

즉, arr[0]은 a 메모리 주소고 *arr[0]을 해야 실제 메모리 주소 a에 들어있는 값 12를 가져옵니다.
그러므로 *arr[1]은 24입니다.

배열 이름 arr 자체가 배열의 첫 번째 요소, 즉 arr[0]의 주소를 나타내요.

*arr은 arr[0]로 표현할 수 있고, arr[0]에 저장된 값을 의미해요.

arr[0]에 저장된 값은 지금 a의 주소를 저장하고 있으니([7행])

한번 더 역참조 연산자(*)로 a에 저장된 값을 가져오기 위해서 **arr(*arr[0])을 사용한 거예요.

그러므로 **arr은 12입니다.

[기출예제] 23년 1회 실기 (난이도: ★★★)

▶ 정답 qwe

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

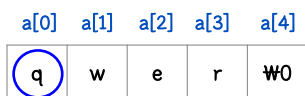
```

1  #include <stdio.h>
2
3  int main() {
4
5      char* a = "qwer";  → 문자열 "qwer"를 가리키는 포인터 a를 선언하고 값을 할당해요.
6      char* b = "qwtety";  → 문자열 "qwtety"를 가리키는 포인터 b를 선언하고 값을 할당해요.
7
8      for(int i = 0; a[i] != '\0' ; i++){  → 문자열 a를 처음부터 끝까지(널 문자가 나타날 때까지) 확인하는 반복문이에요.
9          for(int j = 0; b[j] != '\0' ; j++){  → 여기서 i는 문자열 a의 각 문자를 순서대로 가리키는 인덱스예요.
10             if(a[i] == b[j]) printf("%c", a[i]);  → 내부 반복문은 문자열 b를 처음부터 끝까지 확인해요.
11         }  → j는 문자열 b의 각 문자를 순서대로 가리키는 인덱스죠.
12     }  → a의 i번째 문자와 b의 j번째 문자가 같은지 확인해요.
13  }  → 만약 같다면 그 문자를 출력합니다.
14

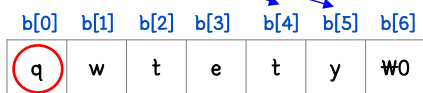
```

★ key point

문자열 배열의 끝이 null(\0인 것만 알고 있으면, 크게 어렵지 않은 문제예요.



하나씩 같은 문자인지 비교 후 출력



출력!

문자열 a와 b를 비교해서 두 문자열에 공통으로 있는 문자를 찾아서 화면에 출력합니다.

예를 들어, 'q', 'w', 'e', 'r'과 'q', 'w', 't', 'e', 't', 'y'를 비교하면, 공통 문자 'q', 'w', 'e'가 있고, 이 문자들이 출력되는 거죠.

그래서 정답은 qwe가 돼요.



[기출 예제] 22년 2회 실기 (난이도: ★★★)

▶ 정답 10

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```

1  #include <stdio.h>
2
3  int len(char*p);
4
5  int main() {
6      char* p1 = "2022";
7      char* p2 = "202207";
8
9      int a = len(p1);
10     int b = len(p2);
11
12     printf("%d", a + b);
13 }
14
15 int len(char*p){
16     int r = 0;
17     while(*p != '\0'){
18         p++;
19         r++;
20     }
21     return r;
22 }
```

문자열의 길이를 계산하고, 두 문자열 길이의 합을 출력하는 문제예요.
전혀 어렵지 않아요!

len 함수를 위한 함수 선언이에요.
len 함수는 문자열의 길이를 반환하는 함수로, 문자열은 char* 포인터로 전달돼요.

두 개의 문자열 p1과 p2를 선언하고 할당합니다.

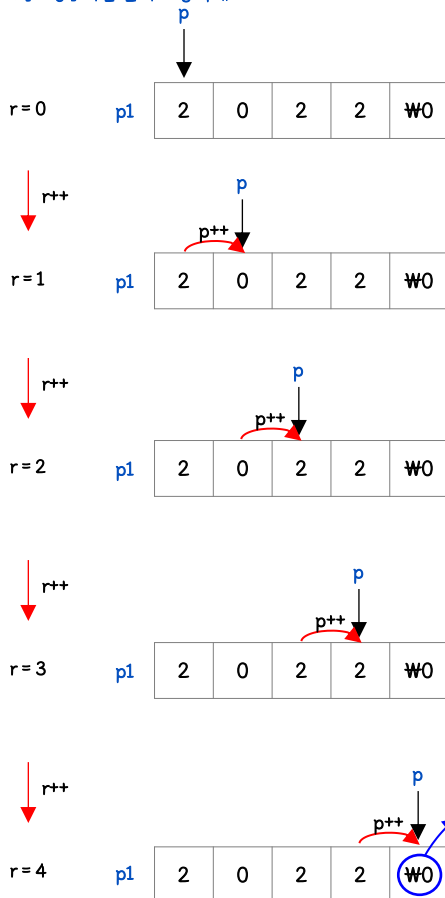
함수를 호출하여 각 문자열의 길이를 계산하고, 그 결과를 변수 a와 b에 저장합니다.

a = 4, b = 6으로 10이 출력돼요.

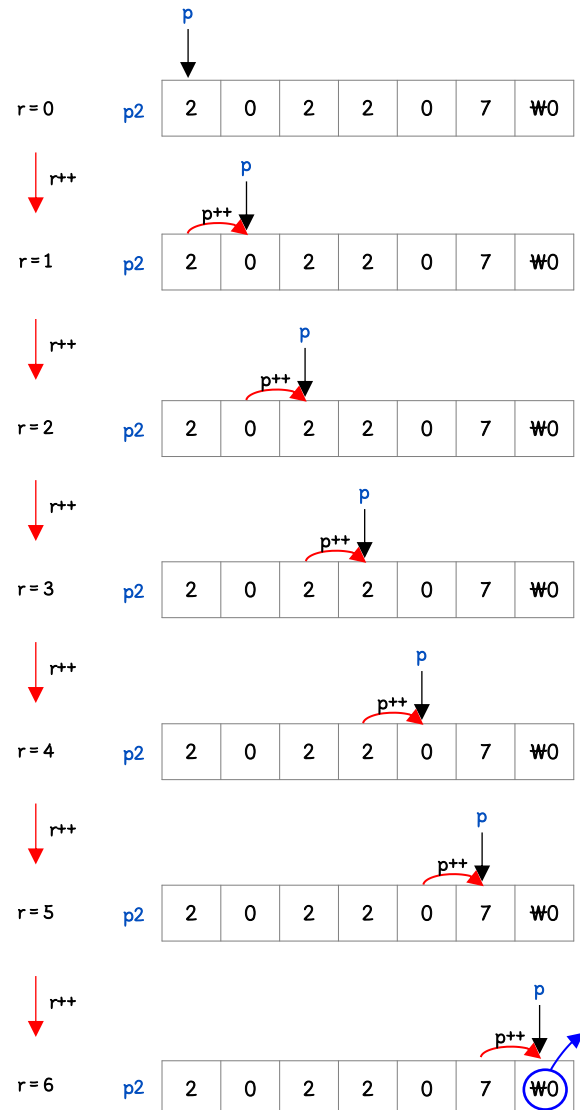
★ key point
후위 연산자 ++: 변수++ 는 변수를 먼저 반환하고, 변수에 변수 + 1를 할당을 뜻해요.
배열의 주소(포인터 p)에 +1을 하면, 다음 문자를 가리킨다는 개념을 알고 있으면 돼요.
위에서 *a+1의 출력하라 와 같은 문제처럼 주소 이동에 대한 개념을 묻는 문제예요.
문제 형태는 다르지만, 문제에서 묻고자하는 본질은 같아요

문자열의 길이를 저장할 변수 r을 0으로 할당합니다.
*p가 널 문자가 아닐 때까지 반복합니다. *p는 포인터 p가 가리키는 문자를 나타냅니다.
p++; 포인터 p를 다음 문자로 이동시키고,
r++; 길이 카운터 r을 1 증가시킵니다.
계산된 문자열 길이 r을 반환합니다.

★ [17행] 다음 순서로 동작해요



`while(*p != '\0')`
이동 후 [17행] *p가 \0이므로,
while문을 빠져나와요.
그러므로, p1의 len은 4를 반환해요.





▶ 정답
박영희
박영희
박영희

[기출 예제] 23년 2회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 "홍길동", "김철수", "박영희"를 차례로 입력했을 때 그 실행 결과를 쓰시오.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  char n[30];
5  char* getNames() {
6      printf("입력하세요 : ");
7      gets(n);
8      return n;
9  }
10
11 int main() {
12     char * test1;
13     char * test2;
14     char * test3;
15
16     test1 = getNames();
17     test2 = getNames();
18     test3 = getNames();
19
20     printf("%s\n", test1);
21     printf("%s\n", test2);
22     printf("%s", test3);
23 }
```

여기서는 'n'이라는 이름의 문자 배열을 만들고 있어요.
이 배열은 최대 30개의 문자를 저장할 수 있어요.
사용자로부터 이름을 입력받아 그 이름을 다시 돌려주는 역할을 해요.
만약, test1, test2, test3이 같은 메모리 주소가 아니라 별도로 메모리를 할당하고 싶다면, [4행]을 지우고, [6행] 아래에 char* n = (char*)malloc(30 * sizeof(char)); 로 대체하면 돼요.

사용자의 입력을 받아 'n'이라는 배열에 저장하는 일을 하고 있어요.
'n' 배열에 저장된 값을 함수를 호출한 곳으로 돌려주는 역할을 해요.

'test1', 'test2', 'test3'이라는 이름의 포인터 변수를 선언하고 있어요.
포인터는 메모리 주소를 저장하는 변수로, 여기서는 문자열의 주소를 저장하게 될 거예요.

'getNames' 함수를 호출해 사용자로부터 입력받은 이름을 'test1', 'test2', 'test3' 이 가리키게 해요.
이 변수들은 전부 같은 포인터 (같은 메모리 주소)를 바라보고 있어요.
이 말은 같은 메모리 주소를 전부 바라 보고 있다면, 마지막으로 그 메모리 주소에 수정한 내용으로 test1, test2, test3도 보이게 될 거예요.

처음에 메모리 x번 위치에 홍길동이라고 저장한 뒤, x번 위치라는 정보를 test1에 저장 했어요.
두번째로 메모리 (위와 동일한) x번 위치에 김철수 라고 저장한 뒤, x번 위치를 test 2에 저장 했어요.
현재까지, test1도 같은 메모리 주소를 바라보고 있으니 test1 값을 출력해도 김철수라고 보일거예요.

마지막으로 동일한 x번 위치에, 박영희라고 저장한 뒤, test3에 포인터를 저장했는데, test1, test2, test3 전부 같은 메모리 주소를 바라보고, 마지막으로 그 메모리에는 박영희가 저장되어있으니, 전부 박영희로 출력돼요!

★ 포인터

메모리 주소를 다루기 위해 포인터라는 개념을 사용해요.
포인터는 변수가 아닌, 변수의 주소를 저장하는 변수예요.
예를 들어, 'char*'는 문자의 주소를 가리키는 포인터라고 할 수 있어요.

[16행] 에서
홍길동이라고 쓰고
test1이
n[0]을가리켜요.

n[0]	n[1]	n[2]	...	n[29]
홍	길	동		

[17행] 에서
김철수라고 덮어쓰고
test1과 test2가
n[0]을가리켜요.

n[0]	n[1]	n[2]	...	n[29]
김	철	수		

[18행] 에서
박영희라고 덮어쓰고
test1, test2, test3가
n[0]을가리켜요.

n[0]	n[1]	n[2]	...	n[29]
박	영	희		



[기출 예제] 22년 2회 실기 (난이도: ★★★)

▶ 정답

22

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```

1  #include <stdio.h>
2
3  int main() {
4      int a[4] = {0, 2, 4, 8};
5      int b[3] = {};
6      int i = 1;
7      int sum = 0;
8      int *p1;
9
10     for (i; i < 4; i++) {
11         p1 = a + i;
12         b[i-1] = *p1 - a[i-1];
13         sum = sum + b[i-1] + a[i];
14     }
15
16     printf("%d", sum);
17
18     return 0;
19 }
```

a 배열은 {0, 2, 4, 8}으로 할당돼요.
b 배열은 크기가 3인데, 모든 요소가 0으로 할당돼요.

1부터 시작해서 3까지 반복해요. (1,2,3)
a 배열의 i번째 요소를 가리키게 돼요. 즉, a[i]와 같아요.
b 배열의 (i-1)번째 요소에 a 배열의 i번째 요소에서 i-1번째 요소를 뺀 값을 저장해요.
i = 1 → b[0] = *p1 - a[0] = 2 - 0 = 2
i = 2 → b[1] = *p1 - a[1] = 4 - 2 = 2
i = 3 → b[2] = *p1 - a[2] = 8 - 4 = 4
으로, 값이 할당돼요.

sum을 출력해요. 정답은 22예요.

sum에는 이전 자기 자신 sum, b[i-1]과 a[i]의 합을 더해요.
i = 1 → sum = 0 + b[0] + a[1] = 0 + 2 + 2 = 4
i = 2 → sum = 4 + b[1] + a[2] = 4 + 2 + 4 = 10
i = 3 → sum = 10 + b[2] + a[3] = 10 + 4 + 8 = 22
으로, 값이 할당돼요.

a[0] a[1] a[2] a[3]

0	2	4	8
---	---	---	---

b 배열의 (i-1)번째 요소에 a 배열의 i번째 요소에서 i-1번째 요소를 뺀 값을 저장해요.

i = 1 ▶ b[0] = *p1 - a[0] = a[1] - a[0] = 2 - 0 = 2 (← *p1 = a[i] 이니까요!)

i = 2 ▶ b[1] = *p1 - a[1] = a[2] - a[1] = 4 - 2 = 2

i = 3 ▶ b[2] = *p1 - a[2] = a[3] - a[2] = 8 - 4 = 4

으로, 값이 할당돼요.



b[0] b[1] b[2]

2	2	4
---	---	---

sum에는 이전 자기 자신 sum, b[i-1]과 a[i]의 합을 더해요.

i = 1 ▶ sum = 0 + b[0] + a[1] = 0 + 2 + 2 = 4

i = 2 ▶ sum = '직전 sum 값' + b[1] + a[2] = 4 + 2 + 4 = 10

i = 3 ▶ sum = '직전 sum 값' + b[2] + a[3] = 10 + 4 + 8 = 22 (최종 결과값)

으로, 값이 할당돼요.



▶ 정답
Nd sc 1

[기출 예제] 24년1회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```

1  #include<stdio.h>
2  #include<ctype.h>
3
4  int main(){
5      char*p = "It is 8";
6      char result[100];
7      int i;
8
9      for(i=0; p[i]!='\0'; i++){
10         if(isupper(p[i]))
11             result[i] = (p[i]-'A'+5)% 26 + 'A';
12         else if(islower(p[i]))
13             result[i] = (p[i]-'a'+10)% 26 + 'a';
14         else if(isdigit(p[i]))
15             result[i] = (p[i]-'0'+3)% 10 + '0';
16         else if(!(isupper(p[i]) || islower(p[i]) || isdigit(p[i])))
17             result[i] = p[i];
18     }
19
20     result[i] = '\0';
21     printf("%s\n",result);
22
23     return 0;
24 }
```

문자열 p의 각 문자를 순회하면서 다음 조건에 따라 변환합니다.

대문자인 경우, (isupper 함수는 문자가 대문자인지 여부를 확인)
 $p[i] - 'A'$ 현재 문자의 ASCII 값을 기준으로 'A'의 ASCII 값을 빼서
 'A'를 0으로, 'B'를 1로, ..., 'Z'를 25로 만들어요.
 거기서, +5을 해 5만큼 이동시켜요.
 모듈러 연산자 %를 이용해 0~25 범위를 벗어나지 않도록 해요.
 모듈러 연산자는 나머지를 뜻해요. 즉, 26로 나눈 나머지 값이에요.
 + 'A'을 더해 A로부터 몇만큼 떨어져 있는 수를 구해요.

계산을 해보면, I는 ASCII 값으로 73 A는 65입니다.
 $(73 - 65 + 5) \% 26 + 65 = 78$ 이 되고, 78은 ASCII 값으로 N이 돼요.

이렇게 하나하나 로직을 풀어보면 어렵지만,
 결과적으로는 현재 문자열의 5번째 뒤에 있는 문자를 구하는
 방법이에요.

I → J(1) → K(2) → L(3) → M(4) → N(5)

모든 문자열에 대해서 변환을 거치면,

I → N
 t → d
 공백 → 공백
 i → s
 s → c
 공백 → 공백
 8 → 1

즉, Nd sc 1로 변환되어 출력돼요.

ASCII 코드와 **반복되는 변환 규칙**을 파악해야 풀 수 있는 문제였는데요.

이런 류의 문제를 만나면 로직의 유사성을 눈치채고, 어떤 구조로 변환시키는 지
 파악하는 게 중요해요.

만약, 실제 시험장에서 로직 해석이 어렵다면 일단 넘기고 다른 문제를 다 풀고
 돌아와서 천천히 파악 후 풀어보는 걸 추천드려요.

소문자인 경우, (islower 함수는 문자가 소문자인지 여부를 확인)
 대문자와 동일하지만 + 10이므로, 현재 문자 기준으로 10칸
 떨어진 소문자를 반환해요.

t → u(1) → v(2) → w(3) → x(4) → y(5) → z(6) → a(7) → b(8) → c(9) → d(10)
 i → j(1) → k(2) → l(3) → m(4) → n(5) → o(6) → p(7) → q(8) → r(9) → s(10)
 s → t(1) → u(2) → v(3) → w(4) → x(5) → y(6) → z(7) → a(8) → b(9) → c(10)

즉,
 t는 d로
 i는 s로
 S는 c로 변환돼요.

숫자인 경우, (isdigit 함수는 문자가 숫자인지 여부를 확인)
 소문자, 대문자와 동일하지만 + 3이므로, 현재 숫자 기준으로 3칸 떨어진
 숫자를 반환해요.

즉, 8 + 3으로 11이 되지만, 모듈러 연산자로 나머지만 1이 반환돼요.

그 외의 경우에는 문자 그대로 반환해요.



[기출예제] 24년 2회 실기 (난이도: ★★★)

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
1 #include <stdio.h>
2
3 int main() {
4     int arr[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9}; → 3행 3열의 2차원 배열 arr을 선언하고 값을 초기화해요.
5     int* parr[2] = {arr[1], arr[2]}; → 크기가 2인 포인터 배열 parr은 선언하고 arr 두번째 행, arr의 3번째 행의 주소를 저장해요.
6     printf("%d", parr[1][1] + *(parr[1]+2) + **parr); 즉, arr[1]은 4, 5, 6의 시작 주소, arr[2]는 7, 8, 9의 시작 주소를 저장해요.
7
8     return 0;
9 }
```

parr[1] = arr[2] 이고,
arr[2]에서 2번째(arr[2][1])인
8을 가져와요.

parr[1] = arr[2]에서
두 칸 이동 = arr[2][2]
9를 가져와요.

*parr = parr[0]주소 = arr[1] = 4, 5, 6
**parr = arr[1][0] = 4가 돼요.

즉, $8+9+4 = 21$ 이 출력돼요.

arr	[0][0]	[0][1]	[0][2]
[0][0]	1	2	3
[1][0]	4	5	6
[2][0]	7	8	9

parr	[0]	[1]
[0]	●	●
[1]	●	●

arr[1][0], arr[2][0]의
시작 주소를 저장

▶ 정답
21

[기출예제] 24년 2회 실기 (난이도: ★★★)

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 void sumFn(char* d, char* s) { → 두 개의 문자열 포인터를 받아요.
5     while (*s) { → *s가 가리키는 곳에 글자가 있는 동안 계속해라"라는 뜻이에요.
6         *d = *s; → s가 가리키는 글자를 d가 가리키는 곳에 할당해요.
7         d++; → d를 다음 위치로 옮겨요.
8         s++; → s를 다음 위치로 옮겨요.
9     }
10    *d = '\0'; → 복사가 끝난 후, 문자열의 끝을 나타내는 null 문자를 추가해요.
11    }
12    즉, s를 받아 d에 옮기는 작업을 하는 함수예요.
13
14 int main() {
15     char* str1 = "first"; → 두 개의 문자열을 만들어요.
16     char str2[50] = "teststring"; str1은 "first"라는 고정 문자열을, str2는 "teststring"이라는 내용으로 시작하는 50자리 배열이에요.
17     sumFn(str2, str1); → str2에 str1의 값을 덮어써요.
18
19     for (int i = 0; str2[i] != '\0'; i++) { → str2의 각 문자에 대해 반복하면서 result에 인덱스 값을 더해요.
20         result += i;
21     }
22     printf("%d", result);
23
24     return 0;
25 }
```

결과적으로 "first"의 길이인 5에 대해 $0+1+2+3+4$ 의 합인 10을 출력해요.

[7행] 포인터 이동

	t	e	s	t	s	t	r	i	n	g	\0
[6행] 덮음	f	i	r	s	t	\0					

▶ 정답
10

[기출예제] 24년 3회 실기 (난이도: ★★★)

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
1 #include <stdio.h>
2
3 void func(int** arr, int size) { → 이중 포인터(arr)과 배열 크기(size)를 매개변수로 받아요.
4     for (int i = 0; i < size; i++) { → 배열의 크기만큼 반복하면서 요소에 접근해요.
5         *(*arr + i) = (*(*arr + i) + i) % size;
6     }
7 }
8
9 int main() {
10     int arr[] = {3, 1, 4, 1, 5};
11     int* p = arr;
12     int** pp = &p;
13     int num = 6; → 그냥 num 변수 선언 과정이에요.
14
15     func(pp, 5);
16     num = arr[2]; → arr[2]에는 1이 할당되게 되고,
17     printf("%d", num); 출력은 1이 돼요.
18
19     return 0;
20 }
```

func 함수 호출

pp = &p ← arr = p ← arr + 0 arr + 1 arr + 2 arr + 3 arr + 4
(arr의 0번째 주소를 담고 있는 메모리의 주소) (arr의 0번째 주소)

★ key point

포인터와 2중 포인터의 동작 원리를 아는지 체크하는 문제예요.
배열의 주소와 포인터 간의 관계를 잘 아는 게 중요해요.

▶ 정답
1

3	1	4	1	5	\0
(3+0)%5	(1+1)%5	(4+2)%5	(1+3)%5	(5+4)%5	
3	2	1	4	4	\0



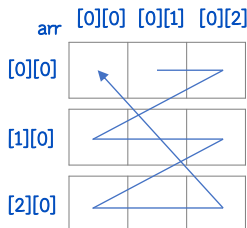
[가출 예제] 25년1회 실기 (난이도: ★★★)

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void set(int** arr, int* data, int rows, int cols) { → 아래 main 흐름을 먼저 읽어주세요.
5      for (int i = 0; i < rows * cols; ++i) { → for 문은 전체 원소 개수(rows * cols)만큼 반복해요. (3×3 = 9번 반복)
6          arr[((i + 1) / rows) % rows][((i + 1) % cols) % cols] = data[i]; → ((i+1)/row) % row는 ((i+1)/3) % 3 이고, i+1을 나눈 몫을 3으로 나눴을 때 나머지를 뜻해요.
7      }
8  }
9      즉, 첫번째 행부터 채우겠다는 의미예요. 0 → 0 → 1 → 1 → 1 → 2 → 2 → 2 → 0
10     (i+1)%cols는 i+1을 3으로 나눈 나머지를 뜻해요. 즉, 열 1,2,0 이 반복돼요. (i를 넣어보면 알 수 있어요)
11
12 int main() {
13     int rows = 3, cols = 3, sum = 0; → 3행 3열의 2차원 배열 arr을 선언하고 값을 초기화해요.
14     int data[] = {5, 2, 7, 4, 1, 8, 3, 6, 9}; → 3×3짜리 배열에 채울 9개의 숫자가 배열에 할당되어 있어요.
15     int** arr; → 이중 포인터를 써서 2차원 배열을 가리킬 준비를 해요.
16     arr = (int**) malloc(sizeof(int*) * rows); → 행의 개수(rows)만큼 포인터를 담을 포인터 배열을 생성해요.
17     for (int i = 0; i < cols; i++) {
18         arr[i] = (int*) malloc(sizeof(int) * cols); → 각 행마다 cols만큼 정수 저장 공간을 따로 할당해, 3×3 배열을 완성해요.
19     }
20
21     set(arr, data, rows, cols); → arr 이차원 배열이 data 값으로 채워져요.
22
23     for (int i = 0; i < rows * cols; i++) { → 0부터 8(9미만)까지 순회해요.
24         sum += arr[i / rows][i % cols] * (i % 2 == 0 ? 1 : -1); → i / rows는 몇 번째 행(row)인지, i % cols는 몇 번째 열(column)인지 정해요.
25     }
26     for(int i=0; i<rows; i++) {
27         free(arr[i]); free는 동적할당 된 메모리를 해제해요.
28     }
29     free(arr);
30     printf("%d", sum); → sum은 13이기에 13이 출력돼요.
31 }

```



arr

	[0][0]	[0][1]	[0][2]
[0][0]	9	5	2
[1][0]	7	4	1
[2][0]	8	3	6

▶ 정답
13

arr

	[0][0]	[0][1]	[0][2]
[0][0]			
[1][0]			
[2][0]			

★ malloc

동적 메모리 할당 함수입니다.

자바에서는 List를 통해 동적으로 쉽게 데이터를 삽입/삭제 할 수 있지만 C에서는 조금 더 복잡하게 size 바이트만큼 연속된 메모리 공간을 힙 영역에서 할당해요.

cols의 크기인 배열을 프로그램이 동작하는 시점에 읽고 생성한다고 생각하면 돼요. **단순하게 malloc를 통해 이차원 배열을 생성했다** 라고 생각하면 됩니다.

★ key point

어려운 문제로 이중 포인터, malloc 함수를 알아야 풀 수 있는 문제예요. 배열 할당의 규칙도 중구난방이고, 복잡해서 실수하기 너무 좋은 문제요. 이런 때는 값을 미리 넣지 말고 **값 할당의 순서를 미리 그려보고** 채워 넣으면 실수를 줄일 수 있어요.



[문제 유형] 정적 변수

★ 정적변수

1. 메모리에 **한 번만 할당**돼요.

- 정적 변수는 프로그램이 실행될 때 메모리에 할당되고, 프로그램이 종료될 때까지 그 값을 유지해요.
- 그래서 함수가 여러 번 호출되더라도 새로 초기화되지 않아요.

2. **초기화는 한 번만 이루어져요.**

- 정적 변수를 선언할 때 초기값을 주면, 프로그램 실행 시 한 번만 초기화돼요.
- 이후에는 이전에 저장된 값을 그대로 사용해요.

[가출 예제] 24년 3회 실기 (난이도: ★★☆☆)

▶ 정답 20

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```

1  #include <stdio.h>
2
3  int func() {
4      static int x = 0;  → 정적 변수는 함수가 끝나도 메모리에 남아 값이 유지돼요. 처음에 x는 0으로 초기화돼요.
5      x += 2;            → 함수가 호출될 때마다 x += 2;로 값이 2씩 증가해요.
6      return x;          → 최종적으로 x의 값을 반환(return)해요.
7  }
8
9  int main() {
10     int x = 1;  → 이 x는 메인 함수 안에서만 사용되는 지역 변수이고, func 함수의 x와는 다른 거예요.
11     int sum = 0; → 최종적으로 계산된 값을 저장하는 용도로 사용돼요.
12     for (int i = 0; i < 4; i++) { → 0부터 시작해서 4번 반복해요.
13         x++; → 메인 함수의 지역 변수 x를 1 증가시켜요 (헷갈리게 하기 위한 변수 - 지역 변수 x는 사용되는 곳이 없어요.)
14         sum += func(); → func 함수를 호출하고, 반환된 값을 sum에 더해요.
15     }
16     printf("%d", sum);
17
18     return 0;
19 }
```

func 내부의 정적 변수 x는 호출될 때마다 2씩 증가하니까, 반환값이 계속 바뀌어요:

- 첫 번째 호출: 2 반환
- 두 번째 호출: 4 반환
- 세 번째 호출: 6 반환
- 네 번째 호출: 8 반환

결과적으로 sum은 $2 + 4 + 6 + 8 = 20$ 이 돼요.

★ key point

정적 변수(static)의 특성과 지역 변수와의 차이, 그리고 변수의 스코프(scope)를 이해하고 있는지 확인하는 문제입니다. 특히, 함수 내부의 정적 변수가 호출될 때 어떻게 값이 유지되는지를 알아보는 게 목적이예요.



[문제 유형] 구조체

★ 구조체

구조체란, 여러 가지 다른 종류의 변수를 하나로 묶어 새로운 형태의 '데이터 뭉치'를 만드는 것이예요.
마치 일상에서 서로 다른 물건들을 하나의 상자에 담아 정리하는 것처럼요!

예를 들어, 친구 한 명을 생각해볼게요. 이 친구에 대해 이름, 나이, 전화번호 등 다양한 정보가 있잖아요?
C언어의 구조체를 사용하면 이런 정보들을 한데 모아 '친구'라는 하나의 구조체로 만들 수 있어요.

```
struct Friend {
    char name[50];
    int age;
    char phone[15];
};
```

구조체 생성

```
struct Friend myFriend;
```

직접 접근

myFriend.name, myFriend.age, myFriend.phone처럼 점(.)을 사용해 해당 친구의 정보에 바로 접근할 수 있어요.

구조체 배열로 생성하기

```
struct Friend myFriends[5];
```

배열 접근

myFriends[0].name 이렇게 배열의 인덱스를 사용하여 접근하면 돼요. 여기서 0은 배열의 첫 번째 친구를 의미합니다.

구조체 포인터로 생성하기

```
struct Friend *ptrFriend;
ptrFriend = &myFriend;
```

포인터 접근

ptrFriend->name, ptrFriend->age, ptrFriend->phone처럼 화살표(->)를 사용해 포인터가 가리키는 구조체의 정보에 접근하면 돼요.

[기출 예제] 22년 2회 실기 (난이도: ★★★)

▶ 정답 2

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```
1  #include <stdio.h>
2
3  struct A { → struct A라는 새로운 데이터 타입을 정의합니다.
4      int n;      이 타입에는 n과 g라는 두 개의 int 타입의 필드(멤버 변수)가 있어요.
5      int g;
6  };
7
8  int main() {
9      struct A a[2]; → struct A 타입의 객체 두 개를 담을 수 있는 배열 a를 선언합니다.
10     for(int i = 0; i < 2; i++) { → 0부터 시작하여 2 미만인 경우에 대해 반복하는 루프입니다.
11         a[i].n = i; → 배열 a의 i번째 원소의 n 필드에 i 값을 할당합니다.
12         a[i].g = i+1; → 배열 a의 i번째 원소의 g 필드에 i+1 값을 할당합니다.
13     }
14     printf("%d", a[0].n + a[1].g); a[0].n의 값과 a[1].g의 값을 더한 결과를 출력합니다.
15 }
```

루프가 두 번 반복되므로,

a[0].n에는 0이, a[0].g에는 1이 저장되고,

a[1].n에는 1이, a[1].g에는 2가 저장됩니다.

따라서, 마지막에 출력하는 a[0].n + a[1].g는 0 + 2이므로 결과는 2가 됩니다.

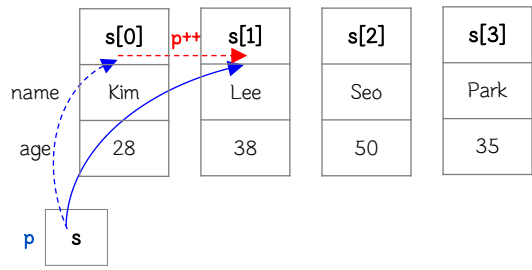
	a[0]	a[1]
n	0	1
g	1	2



[기출 예제] 21년 1회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

```
1 #include <stdio.h>
2
3 struct good {
4     char name[10];  good이라는 이름의 구조체가 정의되어 있어요.
5     int age;        이 구조체에는 name과 age라는 두 필드가 있어요.
6 };
7
8 void main() {  s라는 good 구조체 배열이 할당되어 있어요. 배열은 "Kim", 28, "Lee", 38, "Seo", 50, "Park", 35 이렇게 4명의 정보를 포함하고 있어요.
9     struct good s[] = {{"Kim", 28}, {"Lee", 38}, {"Seo", 50}, {"Park", 35}};
10
11
12     struct good *p;  → p라는 포인터를 선언하고,
13     p = s;          → good 구조체 배열 s의 첫 번째 원소를 가리키게 설정해요.
14     p++;            → p++를 통해 포인터를 다음 구조체 원소로 이동시켜요. p = s이니, s+1과 동일하게, 두번째 요소를 가리키도록 움직여요.
15                     즉, p는 이제 배열의 두 번째 원소, 즉 "Lee"와 38을 가리키고 있어요.
16     printf("%s\n", p->name);
17     printf("%d\n", p->age);
18 }
```



▶ 정답
Lee
38

★ key point

구조체와 포인터가 둘 다 쓰였어요.
구조체 배열의 첫 번째 주소를 p에 할당했어요.
주소를 +1([14행]) 시키면 다음 요소를 반환한다는 개념은 동일해요.

[기출 예제] 23년 3회 실기 (난이도: ★★★)

C언어에서 구조체의 멤버에 접근하기 위해 괄호안의 기호를 작성하시오.

출력결과

Lee
41

▶ 정답 →

```
1 #include <stdio.h>
2
3 struct insa {  → insa라는 구조체를 만들고 있어요. 이름과 나이를 저장할 수 있고, impl_a와 impl_b라는 포인터도 있어요.
4     char name[10];  포인터는 다른 insa 구조체를 가리키는데 사용돼요.
5     int age;
6     struct insa* impl_a;
7     struct insa* impl_b;
8 };
9
10 int main() {
11     struct insa p1 = { "Kim", 28, NULL, NULL };  → 'Kim', 'Lee', 'Park'이라는 이름을 가진 세 명의 insa 구조체를 만들어요.
12     struct insa p2 = { "Lee", 36, NULL, NULL };  각각 나이도 설정하고 있고,
13     struct insa p3 = { "Park", 41, NULL, NULL };  impl_a와 impl_b 포인터는 아직 가리키는 곳이 없어서 NULL로 설정했어요.
14
15     p1.impl_a = &p2;  → p1의 impl_a 포인터가 p2를 가리키게 하고,
16     p2.impl_b = &p3;  p2의 impl_b 포인터가 p3를 가리키게 설정하고 있어요.
17
18     printf("%s\n", p1.impl_a( )name);  → p1의 impl_a 포인터가 가리키는 구조체의 이름을 출력하고 (p2의 이름이겠죠?)
19     printf("%d", p2.impl_b( )age);     p2의 impl_b 포인터가 가리키는 구조체의 나이를 출력해요 (p3의 나이겠죠?).
20 }
```

구조체 포인터의 내부 변수를 접근할 때는 ->를 써요.

기억이 안난다면, 위 페이지의 ★ 구조체에 대한 설명 다시 한번 보시는 걸 추천드려요!

★ key point

구조체의 내부 변수에 접근하는 방법은 일반 변수로 선언했다면, (점) 구조체 포인터로 선언했다면 ->로 접근이 가능해요.
[6행] [7행]에서 구조체의 주소를 받도록 선언되어 있으니, 바로 정답 -> 를 유추 할 수 있어요.



[기출예제] 21년 3회 실기 (난이도: ★★★)

▶ 정답 501

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.
(단, 출력문의 출력 서식을 준수하시오.)

```

1  #include <stdio.h>
2
3  struct jsu {
4      char name[12];
5      int os, db, hab, hhab;
6  };
7
8  int main() {
9      struct jsu st[3] = {"데이터1", 95, 88}, {"데이터2", 84, 91}, {"데이터3", 86, 75};
10     struct jsu* p;
11     p = &st[0];
12     (p+1)->hab = (p+1)->os + (p+2)->db;
13     (p+1)->hhab = (p+1)->hab + p->os + p->db;
14     printf("%d\n", (p+1)->hab + (p+1)->hhab);
15 }

```

st라는 이름의 jsu 구조체 배열이 3개의 원소로 할당되어 있어요. 각 원소에는 이름과 os, db 값이 들어가 있어요.
hab과 hhab은 할당되지 않아서 기본값은 0이에요.

st[0]의 주소

p는 st[0], (p+1)은 st[1], (p+2)는 st[2]를 의미해요.
즉, st[1]의 hab를 st[1]의 os + st[2]의 db으로 할당해요.
이 값은 데이터2의 os 값(84)와 데이터3의 db 값(75)를 더한 159예요.

[14행] st 배열의 두 번째 원소의 hab 값을 계산해요.
[15행]은 다음과 같아요.
st[1]->hhab = st[1]->hab + st[0]->os + st[0]->db
데이터2의 hab 값(159)과 데이터1의 os(95)와 db(88) 점수를 모두 더한 값 342예요.
출력은 159 + 342 인 501이 정답입니다.

	st[0]	st[1]	st[2]
name	데이터1	데이터2	데이터3
os	95	84	86
db	88	91	75
hab		① 159	
hhab		② 342	

p = st[0]
p + 1 = st[1]
p + 2 = st[2]

[14행] : (p+1)->hab = (p+1)->os + (p+2)->db
 ① st[1]의 hab 값 = st[1]의 os 값 + st[2]의 db 값 = 159
 84 75

[15행] : (p+1)->hhab = (p+1)->hab + p->os + p->db
 ② st[1]의 hhab 값 = st[1]의 hab 값 + st[0]의 os 값 + st[0]의 db 값 = 342
 159 95 88

[17행] : (p+1)->hab + (p+1)->hhab
 st[1]의 hab 값 + st[1]의 hhab 값 = 501 (최종 출력값)
 159 342



▶ 정답
9981 and 2795.10

[기출예제] 24년1회 실기 (난이도: ★★★)

다음 C언어의 알맞은 출력값을 작성하시오.

```

1  #include <stdio.h>
2
3  typedef struct{           → BankAcc라는 이름의 이 구조체는 계좌 번호(accNum)와 잔액(bal)을 가집니다.
4      int accNum;
5      double bal;
6  } BankAcc;
7
8  double sim_pov(double base, int year) {   base의 year만큼의 거둬제곱 값을 계산하여 반환해요
9      int i;
10     double r = 1.0;           → r은 1.0으로 할당되어 시작해요.
11
12     for (i = 0; i < year; i++) {
13         r = r * base;           → r에 base를 yaer만큼 곱한 뒤 반환해요.
14     }
15
16     return r;
17 }
18
19 void initAcc(BankAcc *acc, int x, double y){ acc 객체와 x, y를 받아
20     acc -> accNum = x;           → acc.accNum에 x를 할당
21     acc -> bal = y;             → acc.bal에 y를 할당해요.
22 }
23
24 void update_balance(BankAcc *acc, double *en){
25     if (*en > 0 && *en < acc -> bal) {           → 주어진 금액 *en이 양수이고 계좌 잔액보다 작으면
26         acc -> bal = acc -> bal - *en;           → 빼고(출금),
27     }else{
28         acc -> bal = acc -> bal + *en;           → 그렇지 않으면 더해요. (입금)
29     }
30 }
31
32 void apply_interest(BankAcc *acc){
33     acc -> bal = acc -> bal * sim_pov((1+0.1),3) ;           → acc의 잔액을 1.1(1+0.1)으로 3제곱한 값으로 업데이트해요.
34 }
35
36 int main(){
37     BankAcc myAcc;           → myAcc라는 이름의 BankAcc 구조체 변수를 선언해요.
38     initAcc(&myAcc, 9981, 2200.0);
39     double amount = 100.0;
40     update_balance(&myAcc, &amount);
41     apply_interest(&myAcc);
42     printf("%d and %.2f", myAcc.accNum, myAcc.bal);
43     return 0;
44 }
```

initAcc 함수를 호출하여 계좌 번호(accNum) 9981과 잔액(bal) 2200으로 초기화해요.
amount 변수를 100으로 초기화해요.
[40행] 함수를 호출하여 100을 출금하여 잔액을 2100으로 만듭니다.
(잔액보다 작은 금액을 넘겼기 때문에 2100으로 차감돼요.)
[41행] 함수를 호출하여 잔액을 업데이트합니다.
결과적으로 잔액은 2100 * 1.331(1.1*1.1*1.1) = 2795.10이 됩니다.

계좌 번호와 잔액을 출력하여 9981 and 2795.10이 정답이 돼요.

★ key point

정말 어려워보이지만,
구조체 개념만 잘 알고 천천히 코드를 해석한다면
크게 어려움 없이 풀 수 있어요.



▶ 정답
20

[기출예제] 24년 2회 실기 (난이도: ★★☆☆)

다음은 C언어의 구조체에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

```
1 #include <stdio.h>
2
3 struct node {
4     int n1;
5     struct node *n2;
6 };
7
8 int main() {
9
10    struct node a = {10, NULL};
11    struct node b = {20, NULL};
12    struct node c = {30, NULL};
13
14    struct node *head = &a;
15    a.n2 = &b;
16    b.n2 = &c;
17
18    printf("%d\n", head->n2->n1);
19
20    return 0;
21 }
```

→ 'node'라는 구조체를 정의해요. 이 구조체는 정수 n1과 다른 node를 가리키는 포인터 n2를 가지고 있어요.

→ 세 개의 node 구조체 a, b, c를 만들고 초기화해요.

→ 각각의 n1은 10, 20, 30으로 설정되고, n2는 NULL(아무것도 가리키지 않음)로 설정돼요.

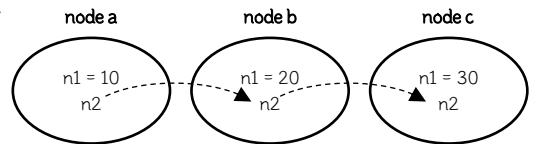
→ head라는 포인터를 만들고 a의 주소를 저장해요. 이제 head는 a를 가리켜요.

→ a의 n2가 b를 가리키도록, b의 n2가 c를 가리키도록 설정해요.

→ head는 a를 가리키고, a의 n2는 b를 가리켜요.

→ b의 n1은 [11행]에서 20으로 할당했어요.

→ 20이 출력돼요.



▶ 정답
312

[기출예제] 24년 3회 실기 (난이도: ★★☆☆)

다음은 C언어의 구조체에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

```
1 #include <stdio.h>
2
3 struct Node {
4     int value;
5     struct Node* next;
6 };
7
8 void func(struct Node* node) {
9     while (node != NULL && node->next != NULL) {
10         int t = node->value;
11         node->value = node->next->value;
12         node->next->value = t;
13         node = node->next->next;
14     }
15 }
16
17 int main() {
18     struct Node n1 = {1, NULL};
19     struct Node n2 = {2, NULL};
20     struct Node n3 = {3, NULL};
21
22     n1.next = &n3;
23     n3.next = &n2;
24
25     func(&n1);
26
27     struct Node* current = &n1;
28
29     while (current != NULL) {
30         printf("%d", current->value);
31         current = current->next;
32     }
33
34     return 0;
35 }
```

Node라는 구조체를 정의해요.

- value: 값을 저장하는 변수.
- next: 다음 노드의 주소를 저장하는 포인터

→ 현재 노드가 존재하고, 다음 노드도 존재하는 동안 반복해요.

→ 현재 노드의 값을 임시 변수 t에 저장해요.

→ 현재 노드의 값을 다음 노드의 값으로 바꿔요.

→ 다음 노드의 값을 임시 변수 t에 저장된 값으로 바꿔요.

→ 두 노드씩 이동해요.

→ 노드 n1, n2, n3을 생성하고, 값은 1, 다음 노드는 없음(NULL)으로 설정해요.

→ n1(1) -> NULL, n2(2) -> NULL, n3(3) -> NULL

→ n1(1) -> n3(3) -> n2(2) -> NULL 의 형태로 연결했어요.

→ 첫 번째 반복:

node는 n1(1)이고, node->next는 n3(3)이에요.

교환: n1.value = 3, n3.value = 1

node는 n2로 이동.

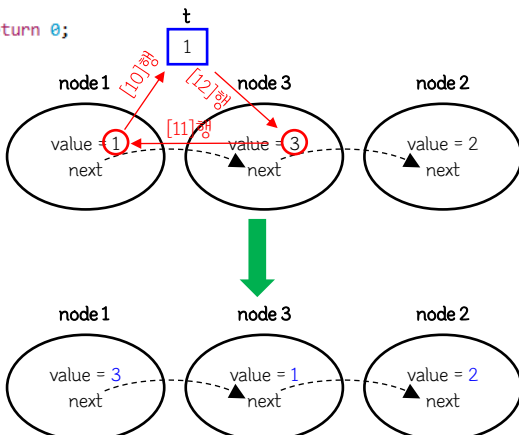
결과: n1(3) -> n3(1) -> n2(2)

→ 두 번째 반복:

node는 n2이고, node->next는 NULL이므로 반복 종료.

→ 최종 리스트:

n1(3) -> n3(1) -> n2(2) -> NULL





[기출예제] 25년1회 실기 (난이도: ★★★)

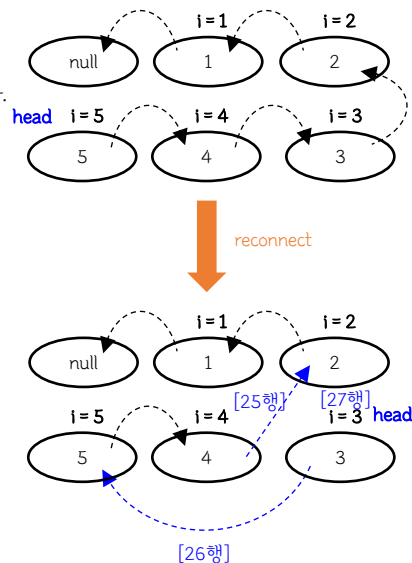
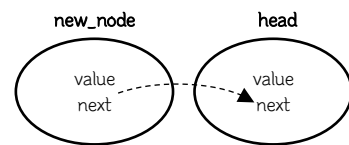
▶ 정답
35421

다음은 C언어의 구조체에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력 값을 작성하시오.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Data { → 'Data'라는 구조체를 정의해요.
5      int value;        값을 저장할 value와 다음 노드를 가리킬 포인터 next를 변수로 가지고 있어요.
6      struct Data *next;
7  } Data;
8
9  Data* insert(Data* head, int value) {
10     Data* new_node = (Data*)malloc(sizeof(Data)); → 새로운 노드를 만들어요. (malloc는 동적으로 메모리를 size만큼 생성해요. - 자바의 new와 유사)
11     new_node->value = value;
12     new_node->next = head; → 그 노드의 값을 value로 할당하고, 새로운 노드의 next에 기존 head 노드를 가리키게 합니다.
13     return new_node; → 새로운 노드를 반환해요. (즉, 새로운 노드가 리스트의 맨 앞에 위치하게 돼요.)
14 }
15
16 Data* reconnect(Data* head, int value) { → [38행]을 확인하고, 어떤 값이 들어오는 지 확인합니다. head = reconnect(head, 3); 코드가 실행됩니다.
17     if (head == NULL || head->value == value) return head; → 만약 리스트가 비었거나(head==NULL), 이미 맨 앞이면 그냥 리턴.
18     Data *prev = NULL, *curr = head;
19     while (curr != NULL && curr->value != value) { → while 반복문이 실행되어 값이 3인 노드(curr)와 그 이전 노드(prev)를 찾습니다.
20         prev = curr; → prev는 i=4인 노드를 가리킵니다.
21         curr = curr->next; → curr는 i=3인 노드를 가리킵니다.
22     }
23
24     if (curr != NULL && prev != NULL) {
25         prev->next = curr->next; → i = 4 노드의 next 포인터를 i = 3 노드가 가리키던 2번 노드로 변경합니다.
26         curr->next = head; → i = 3 노드의 next 포인터를 현재 head인 i = 5 노드로 변경합니다.
27         head = curr; → 리스트의 head를 i = 3 노드로 변경합니다.
28     }
29     return head;
30 }
31
32 int main() {
33
34     Data *head = NULL, *curr;
35     for (int i = 1; i <= 5; i++) {
36         head = insert(head, i);
37     }
38     head = reconnect(head, 3);
39     for (curr = head; curr != NULL; curr = curr->next) {
40         printf("%d", curr->value); → head (i=3) 부터 시작하여 null까지 출력합니다.
41     }
42     return 0;
43 }

```





[문제 유형] 자료 구조

▶ 정답 213465

[기출 예제] 23년 2회 실기 (난이도: ★★★)

다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오.

```

1 #include <stdio.h>
2 #define MAX_SIZE 10  스택의 최대 크기를 10으로 정의하는 부분이에요. MAX_SIZE는 스택에 들어갈 수 있는 최대 요소의 수를 나타냅니다.
3
4 int isWhat[MAX_SIZE];  스택을 저장할 배열 isWhat를 선언합니다. 여기서 최대 10개의 정수를 저장할 수 있어요.
5 int point = -1;  스택의 현재 위치를 나타내는 포인터 point를 선언하고, 초기값으로 -1을 설정합니다. 스택이 비어 있음을 의미해요.
6
7 void into(int num) {  스택에 데이터를 삽입하는 함수예요.
8     if (point >= 10) printf("Full");  만약, 스택이 가득 차 있으면 "Full"을 출력하고, 아니면 주어진 숫자 num을 스택에 삽입합니다.
9     else isWhat[++point] = num;  point에 +1은 먼저 한 뒤에 반환해요.
10 }
11
12 int take() {  스택에서 데이터를 꺼내는 함수입니다.
13     if (isEmpty() == 1) printf("Empty");  스택이 비어 있으면 "Empty"를 출력하고, 아니면 스택의 가장 위에 있는 데이터를 꺼냅니다.
14     else return isWhat[point--];  스택에서 제거하면서, 그 값을 반환해요.
15     return 0;  point를 먼저 반환한 뒤, -1을 더해줍니다.
16 }
17
18 int isEmpty() {  스택이 비어 있는지 확인하는 함수입니다. 비어 있으면 1을, 아니면 0을 반환합니다.
19     if (point == -1) return 1;  [31행]에서 스택이 전부 빌 때까지 반복해요.
20     return 0;
21 }
22
23 int isFull() {  스택이 비어 있는지 확인하는 함수입니다. 비어 있으면 1을, 아니면 0을 반환합니다.
24     if (point == 10) return 1;  여기서 사용되지 않아요.
25     return 0;
26 }
27
28 int main(int argc, char const *argv[]) {
29     int e;
30     ① into(5); ② into(2);
31     while(!isEmpty()) {
32         ③ printf("%d", take());
33         ④ into(4); ⑤ into(1); ⑥ printf("%d", take());
34         ⑦ into(3); ⑧ printf("%d", take()); ⑨ printf("%d", take());
35         ⑩ into(6); ⑪ printf("%d", take()); ⑫ printf("%d", take());
36     }
37     return 0;
38 }

```



